GEXLens — Manuál pro správce a vývojáře

Verze 1.11 · říjen 2026 · interní dokumentace — není dostupná v aplikaci

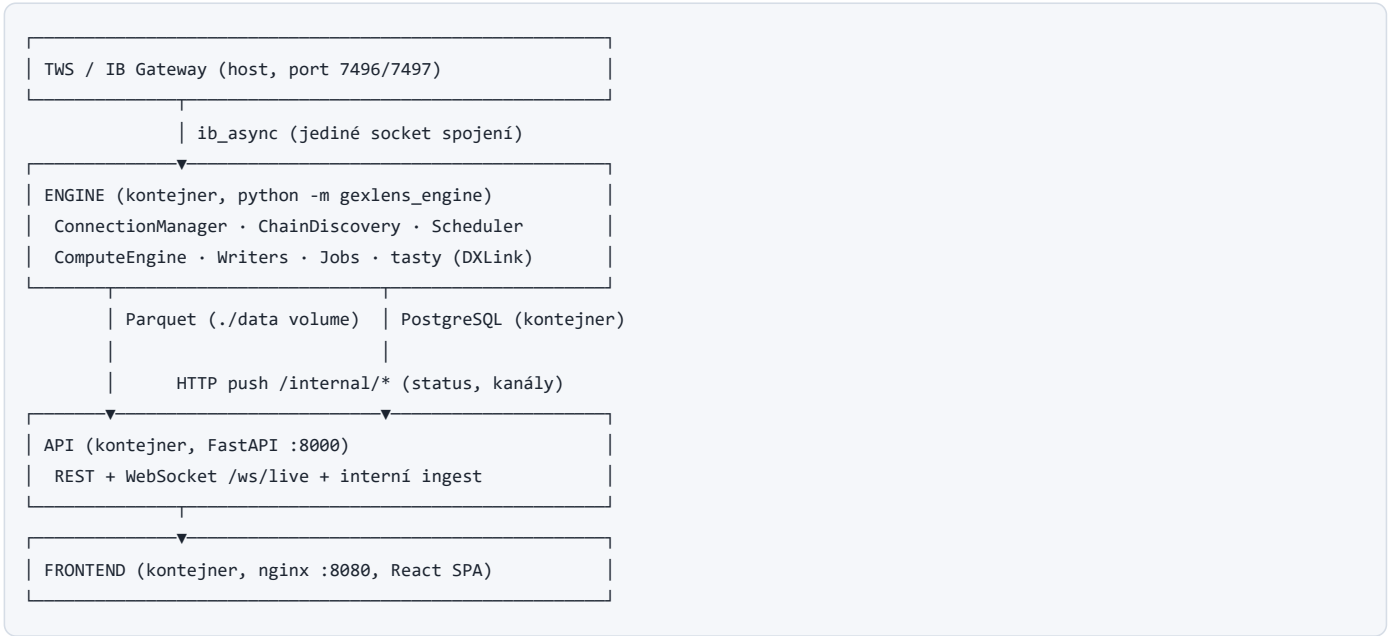
Technický popis architektury, provozu, konfigurace a vývoje aplikace GEXLens. Uživatelská příručka: `UZIVATELSKY-MANUAL.md`. Zdroj pravdy funkčních požadavků: `docs/SPEC.md` (v2.0); architektonická rozhodnutí v `docs/adr/`.

Obsah

1. Architektura
2. Struktura repozitáře
3. Provoz (docker compose)
4. Konfigurace — kompletní reference
5. Engine — datová pipeline
6. Datové formáty a persistence
7. API reference
8. Frontend
9. Vývojové prostředí
10. Testy a CI
11. Známé limity účtu a otevřené body
12. Diagnostika a údržba
13. Zprovoznění od nuly — IBKR účet, TWS/Gateway
14. Bezpečnost a nasazení na server

---

1. Architektura



Klíčové vlastnosti:

- **Engine a API jsou oddělené procesy.** Engine počítá a zapisuje; API jen čte storage a přeposílá live push z enginu (interní HTTP ingest → StatusStore + LiveHub → WebSocket klientům).
- **Vše lokální** — API CORS povoluje jen `localhost` / `127.0.0.1`; žádná telemetrie.
- Engine se z kontejneru připojuje na TWS na hostiteli přes `host.docker.internal`.

2. Struktura repozitáře

```
GEX/
├─ engine/                  Python 3.12 balík gexlens_engine
│  └─ src/gexlens_engine/
│     ├─ config.py          Pydantic Settings (.env, GEXLENS_*)
│     ├─ ibkr/              connection, discovery, scheduler,
│     │                     underlying (bary+pacing), mock (pro testy)
│     ├─ compute/           gex, levels, heatmap, walls, cumdelta, profile
│     ├─ storage/           parquet_store, oi_archive, retention, meta
│     ├─ adapters.py        produkční ib_async adaptéry + HttpPublisher
│     ├─ runtime.py         EngineRuntime — minutový cyklus (testovatelný)
│     └─ __main__.py        vstupní bod: discovery→archiv→smyčka
├─ api/                     Python balík gexlens_api (FastAPI)
│  └─ src/gexlens_api/      main (routy+WS), data, heatmap (vektorizace),
│                           live (hub), status, crud, alerts, meta_repo
├─ frontend/                React + TypeScript + Vite
│  └─ src/                  components/, heatmap/, replay/, panels/,
│                           profile/, annotations/, state/, api/
├─ docs/                    SPEC.md, adr/, manual/
├─ docker/                  Dockerfiles + nginx.conf
├─ compose.yml              celý stack
├─ scripts/                 bootstrap, start skript pro plochu
└─ Makefile                 test / run / run-api / run-frontend / run-engine
```

Pravidla vývoje jsou v `CLAUDE.md`: práce po GitHub issues, golden testy výpočtů, IBKR se v CI nikdy nevolá živě (mock vrstva `engine/ibkr/mock.py`), komentáře česky / identifikátory anglicky.

3. Provoz (docker compose)

```
docker compose up -d --build     # start / rebuild
docker compose ps                # stav služeb
docker compose logs -f engine    # živé logy enginu
docker compose stop              # zastavení (data zůstávají)
docker compose down              # odstranění kontejnerů (volume pgdata zůstává)
```

Služba	Port (host)	Poznámka
frontend	**8080**	nginx, SPA + `/manual/` wiki
api	**8010** (kontejner 8000)	FastAPI, OpenAPI na `/docs`; loopback jen pro nástroje na hostiteli, prohlížeč jde přes nginx
postgres	**55432**	⚠ záměrně ne 5432/5433 — na vývojovém PC běží nativní PostgreSQL na obou
engine	—	bez portu; TWS přes `host.docker.internal:7496`

Nasazení, pin revize image a návrat (#1393, ADR-0047 bod 1)

- **Kdy:** po merge do `main` jen v pauze Globexu (16:00–17:00 CT = 23:00–24:00 SELČ i SEČ; když mají USA letní čas a Evropa ne — od 2. neděle v březnu do poslední neděle v březnu a od poslední neděle v říjnu do 1. neděle v listopadu — 22:00–23:00 SEČ; `scripts/lib/GlobexClock.ps1`), mimo ni jen s výslovným svolením vlastníka a vždy po dotazu (AGENTS.md, rozhodnutí 3b v #1385). Na produkčním daemonu se nebuildí; image staví CI (kap. 9).
- **Revize image:** compose bere `ghcr.io/kechicz/gex-python` a `gex-frontend` s tagem `${GEXLENS_<SLUŽBA>_TAG:-${GEXLENS_IMAGE_TAG:-latest}}`. Bez proměnných běží `latest` (aktuální main). Tagy `sha-<7>` staví workflow *Images* pro každý merge, který mění kód. Služby: `GEXLENS_ENGINE_TAG`, `GEXLENS_API_TAG`, `GEXLENS_NEWS_TAG`, `GEXLENS_FRONTEND_TAG`.
- **Návrat celé aplikace jedním krokem:** do `.env` (končí-li newline) `GEXLENS_IMAGE_TAG=sha-<rev>`, pak `docker compose pull engine api news-engine frontend; docker compose up -d --no-deps engine api news-engine frontend` (výčet služeb, aby se nestáhl a nerestartoval `postgres`). Jen jedna služba: `GEXLENS_ENGINE_TAG=sha-<rev>`, `docker compose pull engine` a `docker compose up -d --no-deps engine`. Zpět na aktuální main = řádek z `.env` odebrat a znovu `pull` + `up -d` týchž služeb. Digest (`sha-<rev>@sha256:…`) jde jen do proměnné jedné služby — `gex-python` a `gex-frontend` mají různé digesty.
- **`deploy-engine-offhours.ps1`** nasazuje HEAD main; při nastaveném pinu (`GEXLENS_IMAGE_TAG`, `_ENGINE_`, `_API_`, `_FRONTEND_TAG`) skončí chybou, než cokoli restartuje — jinak by compose spustil pinovanou revizi a skript by čekal na `latest` marně. `start-prod.ps1 -Build` (nouzový lokální build) při pinu postaví HEAD pod pinovaný tag — před ním pin zruš.
- **Ověření po nasazení nebo návratu** (ne HTTP 200): `docker inspect --format '{{.Name}} {{.Config.Image}} {{index .Config.Labels "org.opencontainers.image.revision"}} {{.State.StartedAt}}' gex-engine-1 gex-api-1 gex-news-engine-1 gex-frontend-1` — revize musí odpovídat zamýšlenému commitu a čas startu nasazení.

Data: Parquet v `./data` (bind mount, sdílené engine↔API), PostgreSQL ve volume `pgdata`. Zálohovat stačí `./data` + `pg_dump` (hlavně tabulku `oi_eod`, která se nikdy nemaže).

Provozní detaily kontejnerů

- Kontejnery běží pod **UID 10001** (`docker/entrypoint.sh`) — bind-mount adresáře musí být zapisovatelné pro tento UID.
- **nginx frontendu proxuje `/api`** na službu API — port API se ven nepublikuje. Adresa služby `api` se **resolvuje za běhu** přes Docker DNS (`resolver 127.0.0.11 valid=10s` + proměnná v `proxy_pass`, #993, v1.3): se statickým `proxy_pass http://api:8000/` si nginx IP vyřešil jednou při startu a po restartu Docker Desktop, kdy api dostal jinou adresu, vracel 502 na všechno (REST i WS), dokud se frontend ručně nerestartoval. Nově se proxy zotaví do ~10 s bez zásahu. S proměnnou nefunguje odříznutí prefixu koncovým lomítkem, proto je v konfiguraci `rewrite ^/api/(.*)$ /$1 break`.
- **Cache hlavičky** (#858): `index.html`, `/manual/` a SPA fallback jdou s `Cache-Control: no-cache`, hashované `/assets/` jako `immutable`. Do té doby si prohlížeč sám určoval, jak dlouho `index.html` podrží, a zastaralá kopie držela uživatele na starém bundlu i po deployi — hard reload (Ctrl+Shift+R) je od té doby jen záchranná brzda, ne standardní krok.
- **`scripts/` je v image enginu** (#960, `COPY` až za `uv sync`, aby úprava skriptu neshazovala cache vrstvy se závislostmi) — dokumentované kroky po nasazení (`docker compose exec engine python scripts/…`) fungují bez `docker cp`.
- **Start skripty ukazují skutečný výstup dockeru** (#975): `start-prod.ps1` i `start-dev.ps1` pouští docker přes sdílený `scripts/_docker.ps1` — výstup teče živě a při selhání se k chybě přiloží posledních 25 řádků; preflight rozliší „docker není v PATH" od „démon neběží". Dřív každé selhání skončilo domněnkou „běží Docker Desktop?".
- Shellové skripty mají v `.gitattributes` vynucené `eol=lf` — checkout na Windows je nesmí konvertovat na CRLF (kontejner by je nespustil.)
- **Zaručený exit enginu** (#779, v1.2): fatální výjimka v `main()` končí okamžitým `os._exit(1)` — kontejner spadne a `restart: unless-stopped` ho zvedne. Dřív po pádu zůstal zombie proces (kontejner „Up", mrtvý engine); při podezření na viselce: `docker kill -s USR1 gex-engine-1` vypíše zásobníky všech vláken (#771).
- **Broker news pásky** (#734, v1.2): kořeny se změřeným Error 200 (`BRFUPDN`, `DJ` — `DEAD_TAPES` v `ibkr/newsticks.py`) se přeskakují; Dow Jones broad tape neexistuje v žádné variantě, reálně tečou BRFG a DJNL.
- **Vyhodnocení shadow porovnání**: `scripts/feed_comparison_report.py [--days N] [--symbol ES] [--sessions 2026-08-14,...]` — agregace v DB, rozpad per podklad, filtr čistých seancí (vstup prahů #614). **Surová historie do 22. 8. je od 1. 9. zhuštěná** do `feed_comparison_daily` (#965, varianta B): 25,7 M řádků / 2,5 GB = 79 % databáze se zálohovalo při každém

dumpu. Percentily nejsou skládatelné, proto tabulka nese dvě úrovně — denní řádky (`session_date`) a **celkové řádky spočítané ze surových dat** (`session_date IS NULL`), takže čísla citovaná v #614, #616 a #517 A zůstávají ověřitelná; nový řez (jiné okno, jiná podmnožina seancí) už nejde. Postup `scripts/feed_comparison_compact.py --build` → `--verify` → `--drop` (drop odmítne, když ověření neprojde). Databáze 3 169 → 653 MB; živý crosscheck na tabulce nezávisí (počítá se z `compare_minute`).

---

4. Konfigurace — kompletní reference

Zdroj: proměnné prostředí `GEXLENS_*` a `.env` (viz `.env.example`). Validuje se při startu — nevalidní hodnota = engine odmítne nastartovat se srozumitelnou chybou.

Proměnná	Default	Význam
`GEXLENS_IBKR_HOST`	127.0.0.1	V compose přepsáno na `host.docker.internal`
`GEXLENS_IBKR_PORT`	7496	7496 live / 7497 paper (TWS); 4001/4002 (Gateway)
`GEXLENS_IBKR_CLIENT_ID`	1	
`GEXLENS_MARKET_DATA_TYPE`	1	1=live; delayed engine odmítá
`GEXLENS_CONNECT_TIMEOUT_S`	10	
`GEXLENS_RECONNECT_BACKOFF_BASE_S` / `_MAX_S`	2 / 60	Exponenciální reconnect
`GEXLENS_HEARTBEAT_INTERVAL_S` / `_TIMEOUT_S`	30 / 15	Heartbeat spojení; agresivnější hodnoty vedly k falešným reconnectům během sweep dávek
`GEXLENS_RECONNECT_STALL_ALERT_S`	300	Watchdog reconnectu (#770): po tolika sekundách bez spojení alert `connection_stall` do zvonečku, pak eskalací (#1317, `stall_alert_stage` : 1×, 3× a 12× prahu, dál à 48× — při 300 s tedy 5 min, 15 min, 1 h, à 4 h), s textem podle zálohy tastytrade (`spot_source = tasty` aspoň u jednoho instrumentu → „data jedou ze zálohy“, jinak „sběr dat stojí“) a zprávou „IBKR spojení je zpět“ po obnovení — jen při otevřeném trhu, jinak log (#1307); `/status.connection_offline_for_s` nese délku výpadku (klíč chybí, když spojení drží)
`GEXLENS_SYMBOLS`	ES	Základní sada futures podkladů (čárkami); watchlist z DB se přidává za běhu (ADR-0003)
`GEXLENS_MAX_INSTRUMENTS`	3	Strop souběžných instrumentů (rozpočet market data lines)
`GEXLENS_FRONT_ROLL_DAYS`	8	Roll front kontraktu (#1189, ADR-0039): kontrakt je front, dokud má do expirace VÍC než N dní (CME roll date = 8 d před expirací). Platí pro IBKR pipeline, tasty streamer i IV rank. 0 = původní chování (nejbližší nepropadlý kontrakt). Discovery cache front kontrakt po rollu zahodí.
`GEXLENS_WATCHLIST_POLL_CYCLES`	5	Watchlist + runtime nastavení (strike_range_points) se čtou z DB každý k-tý cyklus

Proměnná	Default	Význam
`GEXLENS_OI_ARCHIVE_EXPIRIES`	5	Ranní OI archiv pokrývá N nejbližších expirací (základ ΔOI vs. včera)
`GEXLENS_SWEEP_NEXT_EXPIRY`	true	Sekundární sweep následující expirace (positioning příští seance)
`GEXLENS_NEXT_EXPIRY_SWEEP_EVERY`	3	Kadence sekundárního sweepu (každá k-tá minuta)
`GEXLENS_STRIKE_RANGE_POINTS`	200	Výchozí denní obálka spot ± X (ADR-0002); od mechaniky v6 začne aktivní řetěz po rollu v settle v obálce rozšířené o striky sekundáru (ADR-0057 doplněk 20. 10.)
`GEXLENS_STRIKE_RANGE_EXPAND_THRESHOLD`	0.25	Rozšíření při přiblížení k okraji
`GEXLENS_STRIKE_RANGE_MAX_POINTS`	800	Strop šířky obálky (≥ 2× base)
`GEXLENS_BATCH_SIZE`	80	Dávka rotačních subskripcí
`GEXLENS_BATCH_TIMEOUT_S`	4	Čekání na kompletní data kontraktu
`GEXLENS_WINGS_SWEEP_EVERY`	3	Křídla každý k-tý cyklus
`GEXLENS_ATM_SWEEP_WIDTH`	30	ATM ± N strikes každý cyklus
`GEXLENS_REPAIR_MAX_ATTEMPTS`	3	Retry repair fronty za sweep
`GEXLENS_MARKET_DATA_LINES`	100	Kapacita market data lines — **strop účtu je 100**: přidělení IBKR pro API (TWS „Maximum Allowed", 9. 10. 2026, #1479); původní odhad „≥ 150" z ADR-0001 neplatil. `batch_size` nikdy nezvyšovat
`GEXLENS_DATABASE_URL`	postgres localhost	V compose směřuje na službu `postgres`
`GEXLENS_DATA_DIR`	data	Kořen Parquet partic
`GEXLENS_RETENTION_DAYS`	**90**	Purge okno (ADR-0022, odchylka od R3). Nemaže se: `oi_eod` ani žádná `bars/` partice
`GEXLENS_KEEP_BARS_FOREVER`	true	Bary podkladu se z purge vyjímají — základ historických výpočtů (ADR-0028)
`GEXLENS_DISK_LIMIT_GB`	2	Alert při překročení
`GEXLENS_API_BASE`	http://127.0.0.1:8000	Kam engine pushuje (v compose `http://api:8000`)

| `GEXLENS_PG_PASSWORD` | — | **Povinné** — compose bez něj nenastartuje (generuje `scripts/init-secrets.ps1`) | | `GEXLENS_API_TOKEN` | — | **Povinné** — sdílené tajemství `/internal/*` a `/backup/postgres` | | `GEXLENS_BIND_ADDR` | 127.0.0.1 | Na jaké adrese publikují porty (server: ponechat loopback + reverse proxy) | | `GEXLENS_ALLOWED_ORIGINS` | — | CORS whitelist API | | `GEXLENS_NEWS_API_TOKEN` | — | Token news-engine → API push | | `GEXLENS_TASTY_ENABLED` | true | **Trvalá** tastytrade větev (#613, #763): session, DXLink stream, chain mapa, křížová kontrola (#517 A), oba fallbacky (#614), OI fill (#664). Bez tajemství se stejně nespustí, proto default `true` | | `GEXLENS_TASTY_COMPARISON_WRITE` | true | **Dočasný** zápis porovnávacích řádků do `feed_comparison` (#613). Vypnout po vyhodnocení M7 fáze 2 — tally pro detektor a fallbacky běží dál, takže se tím NEztrácí odolnost proti výpadku IBKR. Surová historie do 22. 8. je zhuštěná do `feed_comparison_daily` (#965, kap. 3) | | `GEXLENS_TASTY_MAX_ENTRIES` / `GEXLENS_TASTY_ADHOC_RESERVE_ENTRIES` | 25 000 / 2 000 | **Rozpočet DXLink subskripce** (#982, ADR-0027): server počítá položky `symbol × event` na spojení a strop je 25 000 (změřeno sondou `tasty_probe.py sizecap`, 2. 9.) — produkce 6 236 symbolů × 4 eventy = 24 944, takže ad-hoc pohled přetekl a odmítnuté symboly tiše mlčely. `tasty/budget.py` odebírá **per účel jen eventy, které se čtou** (wide Quote+Summary — jde o OI; extended a ad-hoc Quote+Greeks+Summary; řetěz a podklad vše),

produkce ≈ 18 400 položek; wide/extended smí jen po `strop − rezerva`. Deterministický ořez při přetečení: extended od nejvzdálenější expirace/striku, pak wide od okraje — řetěz/podklad/ad-hoc nikdy. `/status.tasty_budget` nese využití, ořez a `size_exceeded` (server nahlásil `subscription size is too big`; heal se na to nespouští) | | `GEXLENS_TASTY_SHADOW` | — | **ZASTARALÉ (#763)**, nechávej nenastavené. Hlídalo obojí naráz, takže „vypínám doběhnuté měření" tiše bralo i fallbacky. Když je nastaven, řídí `GEXLENS_TASTY_ENABLED` a engine to při startu ohlásí varováním | | `GEXLENS_TASTY_CLIENT_SECRET` / `_REFRESH_TOKEN` | — | OAuth2 grant **výhradně scope** `read` (ADR-0025); dev grant patří do `.env.dev` pod standardními názvy. Obsah `.env` se nikdy nevypisuje do konzole | | `GEXLENS_CROSSCHECK_ENABLED` | true | Křížová kontrola IBKR × tasty (#517 fáze A) — pasivní, bez requestů a linek navíc. Bez zapnuté tasty větve se tiše nezapne | | `GEXLENS_CROSSCHECK_SHARE_THRESHOLD` / `_MINUTES` / `_COOLDOWN_MINUTES` | 0.70 / 3 / 15 | Prahy **měřené** na shadow historii, ne odhadnuté — viz níže. `_MINUTES` pod 3 = falešný poplach každou třetí minutu | | `GEXLENS_CROSSCHECK_CHANGE_THRESHOLD` | 0.30 | Rozlišovač „tichý trh × mrtvá záloha" (#764): podíl kontraktů s **měnícími se** IBKR hodnotami, od kterého je trh živý a mlčící tasty porucha → alert `feed_backup_dead`. Měřeno nad 4 833 min (13.–19. 8.): pauza CME má medián změn ~0, aktivní trh p5 ≈ 0,38–0,42; s prahem 0,30 vyšlo **0 planých poplachů** (bez rozlišovače 6 epizod, všechny v pauze CME 21–22 UTC) | | `GEXLENS_DISK_FREE_WARN_GB` / `_CRIT_GB` / `GEXLENS_DB_SIZE_ALERT_GB` | 15 / 5 / 4 | Dohled nad volným místem (#773): alert `disk_space` do zvonečku (varování/kriticky) + výpis největších tabulek. Měří se SKUTEČNÉ volné místo disku s datovým adresářem (bind mount ukazuje čísla hostitele) a velikost PG přes `pg_database_size`. Obsazení hostitelského disku WSL vhdx (u nás `D:\Programy\Docker\DockerDesktopWSL`, PG volume uvnitř) z kontejneru změřit nejde — protože ale vhdx leží na témže disku jako data, jeho růst měřené volné místo ukusuje přímo; práh na velikost DB je časná výstraha na tahouna růstu. Vhdx se po úklidu sám nezmenší — kompaktace vyžaduje zastavený Docker (`wsl --shutdown` + `Optimize-VHD`/diskpart). Stejná čísla plní patičku UI (`Disk X / Y`), která do té doby ukazovala `– / –` | | `GEXLENS_PROBE_ENABLED` | true | Aktivní IBKR sonda (#517 fáze B): na alert `ibkr_suspect` jednorázový snapshot front future → rozliší výpadek farmy („čekej") od potichu mrtvých subskripcí (cílená obnova bez reconnectu). Neběží periodicky; vlastní cooldown 10 min a pouští se jen s rezervou ≥ 2 market data lines | | `GEXLENS_TASTY_SPOT_FALLBACK` | true | Cena podkladu z tastytrade, když IBKR přestane posílat ticky (#614 fáze 2a) | | `GEXLENS_TASTY_SPOT_STALE_AFTER_S` / `_RECOVER_AFTER_S` / `_MAX_AGE_S` | 30 / 60 / 30 | Hystereze spotu: kdy převzít, kdy vrátit, max stáří tasty kotace. Návrat je delší než převzetí schválně — kmitání zdroje stojí víc než o půl minuty pozdější návrat | | `GEXLENS_TASTY_CHAIN_FALLBACK` | true | Fallback **celého opčního řetězu** (#614 fáze 2b). Spouští ho verdikt křížové kontroly, takže dědí její měřené prahy | | `GEXLENS_TASTY_CHAIN_RECOVER_MINUTES` | 5 | Kolik čistých minut v řadě vrátí řetěz na IBKR. Delší než zapínací série (`_CROSSCHECK_MINUTES`): přepnutí překreslí celý profil | | `GEXLENS_TASTY_CHAIN_MAX_AGE_S` | 120 | Max stáří tasty hodnoty, aby kontrakt vstoupil do fallbackového řetězu | | `GEXLENS_STARTUP_CONNECT_WAIT_S` | 60 | Jak dlouho se při startu čeká na IBKR, než engine rozjede zbytek i bez něj (#756). **Není to timeout spojení** — supervisor se pokouší dál. 0 = nečekat | | `GEXLENS_TASTY_OI_FILL` | true | Díry denního OI archivu doplní tasty `Summary` (#664) — typicky 0DTE ráno, než CME publikuje | | `GEXLENS_NEWS_LLM_ENABLED` | false | LLM klasifikace zpráv (Gemini). **Zakonzervováno** (#740 fáze 0): v `news_weights` neprošla Wilson gate ani jednou (0/20 řádků, hit rate 0,484 proti 0,516 u pravidel). Od ADR-0036 (#1150) váha z Wilson LB nikdy není 0 (rozsah 0,25–2,0, mince = 1,0) a event se váží řádkem svého predictoru (`sentiment_source`) | | `GEXLENS_NEWS_EXPLAIN_ENABLED` | false | **Vysvětlení zprávy** (#1126 3d, v1.18): tlačítko „Vysvětlit" u karty v News → `POST /news/{id}/explain` → Gemini free tier přes REST (klíč `GEXLENS_NEWS_GEMINI_API_KEY` výše, api kontejner přes `env_file`; bez klíče 503, cache jede dál). Jiný účel než klasifikace: porozumění, ne predikce — do SentIndexu, vah ani signálů nic neteče (R4). Odpovědi navždy v PG `news_explanations` (event_id, model, text, tokeny, created_at) | | `GEXLENS_NEWS_EXPLAIN_MODEL` | gemini-3.8-flash,gemini-3.6-flash,gemini-3.5-flash | Řetěz modelů oddělený čárkou — při 5xx „high demand" se hned zkusí další (free tier je přetěžovaný, 15. 9.: 3.8 200/503/503, 3.7 503). **Pinovat konkrétní verze**, ne alias `*-latest` (#738); `thinkingLevel: low` (`minimal` odmítá 400). Jedno vysvětlení ≈ 600 tokenů; uložený řádek nese model, který odpověděl | | `GEXLENS_NEWS_EXPLAIN_DAILY_TOKENS` | 300000 | Denní strop prompt+output tokenů za UTC den, po překročení 429 (0 = bez stropu); free tier má navíc vlastní denní kvótu requestů (429 od Google se hlásí stejně). Spotřeba je v logu api (`Vysvětlení zprávy N: model, in+out tokenů (dnes X/strop)`) | | `GEXLENS_PUSH_TELEGRAM_TOKEN` | — | **Push na Telegram** (#1175, v1.18): přihlašovací řetězec bota z @BotFather; api kontejner ho dostane přes `env_file`. Bez něj (nebo bez chat id) se nic neposílá, `GET /push/status` hlásí `configured: false`. Tytéž klíče čte z `.env` i `scripts/lib/OpsAlert.ps1` pro upozornění, když neběží Docker (#1279); ten se navíc řídí hlavním vypínačem a přepínačem Noční údržba selhala z `GET /push/status` (#1284) | | `GEXLENS_PUSH_TELEGRAM_CHAT_ID` | — | Id soukromého chatu s botem (napiš botovi /start, id je v odpovědi metody `getUpdates` Bot API, pole `chat.id`) | | `GEXLENS_PUSH_QUIET_HOURS` | 23:00-06:00 | Tiché hodiny v Europe/Prague (`HH:MM-HH:MM`, prázdné = žádné); provozní přepínače (kategorie `ops` v `PUSH_TOPICS`) jdou i v nich | | `GEXLENS_PUSH_DAILY_CAP` | 200 | Denní strop odeslaných zpráv (0 = bez stropu); dedup per (kind, symbol, začátek textu) 10 min | | `GEXLENS_PUSH_SETUP_MIN_CONFIDENCE` | 0 | Minimální confidence vzniklého setupu pro push, **v procentech 0–100** (stejná škála, jakou setup posílá v alertu; do #1285 byl rozsah 0–1 a práh nic

nefiltroval) | | *(serverová nastavení)* `push_telegram_enabled`, `push_telegram_topic_<key>` | true / podle kategorie | **Hlavní vypínač a přepínače per druh upozornění** (#1284, Settings → Notifikace), jen bool — jiná hodnota nebo neznámý klíč v `PUT /settings` = 422. Jediný zdroj pravdy je tabulka `PUSH_TOPICS` v `api/src/gexlens_api/push_telegram.py`: 31 přepínačů ve dvou skupinách (`market` = Setupy a burza, `app` = Chování aplikace), každý slučuje druhy (`kind`) se stejnou dřívější kategorií `setup` / `ops` / `news` / `info`; z ní plyne výchozí hodnota (`info` vypnuto, ostatní zapnuto), výjimka z tichých hodin (`ops`) a emoji zprávy. Z tichých hodin je navíc vyjmutý přepínač s `quiet_exempt` — jen „Zprávy před otevřením po víkendu" (`news_preopen`, #1291): aktualizace 15 min před nedělním otevřením padá na 23:45. Efektivní hodnota se počítá při čtení (`effective()`): vlastní klíč → dřívější `push_telegram_setup/ops/news/info` (#1175, už jen čtení, zápis 422) → výchozí; nic se nemigruje ani nepřepisuje. Hlavní vypínač vypnutý = na Telegram neodejde nic, hodnoty přepínačů zůstanou. Druh bez záznamu v `PUSH_TOPICS` na Telegram neodejde (WARNING v logu api jednou za druh) — nový alert = doplnit tabulku i `PUBLISHED_KINDS` v `api/tests/test_push_telegram.py`. Zdroj alertů = kanál `alerts` `LiveHub` (posluchač `TelegramPush`), takže engine, news-engine i provozní alerty api jdou jednou cestou; zvonek na přepínačích nezávisí. Odeslání běží v daemon vlákně, 3 pokusy, 4xx = chyba konfigurace bez opakování; stav a přepínače v `GET /push/status` | | `GEXLENS_SCENARIO_AUTO_MINUTES_BEFORE_OPEN` | 15 | **Automatický scénář dne** (#1173 A): kolik minut před US openem (9:30 ET) engine per symbol sestaví scénář z verdiktu dne (`compute/dayverdict`). Verdikt převezme od zapisovače `DayVerdictWriter` z téže minuty (kap. 5 „Verdikty dne", #1595 d1) — API se za minutu čte jednou (`scenario_auto.gather_context`: `/candles`, `/bars`, `/instruments/{sym}/days`, `/gammacliff`, `/oidelta`, `/sentiment/state`, `/news/upcoming`; levels, tendence a tenká mapa z runtime). Jeden scénář per seance v okně [open − N, open), vznikne v **první minutě okna s úplným verdiktem** (neúplný — výpadek čtení API, chybí levels nebo tendence — ho odloží na další minutu; do #1595 jeden pokus v první minutě okna i při výpadku; při N > 15 před open − 15 min jen na pětiminutových značkách, kdy zapisovač počítá), **jen v obchodní den** (`settle.is_trading_session`: po–pá mimo svátky NYSE, ADR-0046, #1309); `source='auto'`, `rationale` = hlasy + chybějící vstupy; verdikt none/wait_news = bez scénáře (log), verdikt se uloží i tak. Snímek doplní frontend přes `PUT /scenarios/{id}/image` (jednou, jinak 409). 0 = vypnutý scénář, **ne** zápis verdiktu dne (zapisovač má vlastní kadenci) | | *(scénáře dne, #1173)* | — | PG tabulka `scenarios` (řádek per scénář: symbol, den a čas vzniku, termín + `deadline_ts` = settle termínu, vstup, cíle, cesta, `annotation_id`, `image_path`, `image_bytes`, `evaluated_at`, `result`), snímky `data/scenarios/{sym}/{den}/{id}.png` (max 4 MB, jen PNG). API: `POST /scenarios` (jen dopředu — termín ≥ aktuální seance, do 60 dní a v obchodní den po–pá, jinak 422; `created_at` razítkuje server), `GET /scenarios?symbol=`, `GET /scenarios/{id}/image`, `GET /scenarios/stats` (`preliminary` do n < 30; automatické scénáře jen z aktuální verze pravidel verdiktu `rationale.rules_version`, v odpovědi `rules_version`, ruční vždy — #1588), `GET /scenarios/disk` (součet z DB, ne rglob), `DELETE /scenarios/images?older_than_days=N` (jen PNG, řádky zůstávají), `DELETE /scenarios/{id}`. Engine `ScenarioCollector` per symbol jednou po settle (+15 min) **obchodní seance** (o víkendu neběží, #1309): scénáře po termínu bez výsledku → bary `derived/{sym}/bars/` od `created_at` do `deadline_ts`, EM z `em_respect` seance vzniku → `result` + alert `scenario_result` (přepínač Telegramu „Scénář dne"); scénář s termínem mimo obchodní seanci (víkendové řádky z doby před #1309) se uzavře bez výsledku a bez alertu a `GET /scenarios/stats` ho nezapočítá — bez ručního zásahu do dat. Nad 1 GB snímků alert `scenario_disk` (hranově, přepínač Telegramu „Dochází místo na disku"); žádné automatické mazání | | `GEXLENS_NEWS_ALPACA_KEY_ID` / `_SECRET` | — | Alpaca News API — živý stream i historický backfill (#743, #744). Stačí paper účet a **Trading API**, ne Broker API | | `GEXLENS_NEWS_UA_CONTACT` | — | Kontakt (jméno a e-mail) do hlavičky User-Agent pro zdroje, které bez identifikace vrací 403 — dostane ho **jen BLS** (ADR-0059 bod 7, #1516), ostatní zdroje posílají prohlížečovou hlavičku. Převádí se na ASCII (diakritika pryč). Bez proměnné se BLS nespustí a log hlásí `BLS nenakonfigurován` (není to porucha). Nahrazuje `GEXLENS_NEWS_SEC_CONTACT`; čte ho i sonda `scripts/news_candidates_probe.py` | | `GEXLENS_NEWS_REDDIT_RSS_ENABLED` / `_INTERVAL_S` / `_FEED_DELAY_S` | true / 300 / 15 | Reddit nativní RSS (#578). Reddit limituje **anonymní přístup per IP napříč subreddity** (#941, měřeno 29. 8.: 5 s → 429, 15 s → 200, dva feedy za sebou i tak 429 — potřebuje ~150 s klidu). Kolektor proto jede **round robin — jeden subreddit za cyklus** (každý à 10 min, pro crowd sentiment stačí), s jedním retry při 429; chyby feedů se logují se status kódem |

Od #696 jde do kontejnerů celý `.env` (`env_file`), ne ruční výčet — každý klíč z `.env.example` po `docker compose up -d <služba>` skutečně platí. `environment:` v compose nese jen odvozené hodnoty (DATABASE_URL, adresy služeb, /app/data) a ty mají přednost. **Dev stack navíc čte volitelný `.env.dev`** (přepisy jen pro dev: vlastní tasty grant, symboly…; viz `.env.dev.example`). Pozn.: změna PG hesla v `.env.dev` vyžaduje reseed dev volume.

Frontend build-time (build arg `docker/frontend.Dockerfile`): `VITE_API_BASE` (výchozí `/api`), `VITE_GEXLENS_ENV` (`dev` = badge DEV, prod nic) a `VITE_BUILD_SHA` (commit z CI workflow *Images* do exportu měřiče výkonu #1409; lokální build `dev`). Kvůli

`VITE_BUILD_SHA` se bundle přestaví při každém běhu *Images*, i u commitu jen do backendu. Frontend se ale nasazuje jen výslovným pull, takže cache prohlížeče se zneplatní jen při nasazení frontendu.

5. Engine — datová pipeline

Minutový cyklus (`runtime.EngineRuntime.run_cycle`):

1. **Sweep** — `SubscriptionScheduler` projede řetězec v dávkách (ATM±30 každý cyklus, křídla každý 3.), nekompletní kontrakty přes repair frontu, výsledek do in-memory cache.
2. **Snapshot** — cache → `SnapshotRow` (OI z ranního archivu) → atomický zápis Parquet.
3. **Výpočty** — GEX per strike (naivní dealer model, vyměnitelná strategie) → levels (flip interpolovaně, walls, centroid) → zápis do `derived/levels` .
4. **Cum Δ** — `begin_cycle` jednou za cyklus zafixuje verdikt minuty (ADR-0057 bod 3: běží trade větev, nebo mezera). Bar větev rozkládá přírůstek objemu (printvol, pokrytí). Od v6 je přírůstek objemu (tracker, Δ Flow C/P a `opt_vol` setupů, hlas `delta_flow` tendence, replay) proti **maximu seance** (`compute/volume.py` , #1602, ADR-0057 doplněk 22. 10.). Pokles i návrat k maximu dají 0 a pokles zaloguje varování jednou za kontrakt a seanci. Kontrakt po cyklu bez objemu jen založí stav s referencí max(reference, hodnota). Pod v5 se přírůstek počítá proti poslední hodnotě. Znaménko toku pod mechanikou v5 (zdroj `midpoint`) dává minutový midpoint test, od v6 (zdroj `aggressor`) jen tisky dxFeed `TimeAndSale` se stranou (`add_dx_trade` , aktivní i sekundární řetěz přes trasy tisků). Od v6 se tisky řežou v okamžiku vzorku trade větve (`seal`), takže řádek minuty nese tisky mezi dvěma vzorky. Tisky nové seance přijaté před rollem trackeru čekají stranou a nese je první řádek seance. Minuta v mezeře má pod v6 CumΔ NULL. `close_minute` → `derived/flow` (sekundární řetěz minutu uzavře kvůli netu a řádek zahodí).
5. **Bary podkladu** — 5s reqRealTimeBars agregované na 1min → `derived/bars` .
6. **Push do API** — `/internal/status` + kanály `levels.*` , `flow.*` , `price.*` .

Multi-instrument orchestrátor (ADR-0003)

`__main__` řídí **pipeline per podklad** (`instruments.InstrumentPipeline`): cílová sada = `GEXLENS_SYMBOLS` ∪ watchlist z DB — změny chodí okamžitě přes PostgreSQL `LISTEN/NOTIFY` (kanál `gexlens_watchlist` , #207: API po zápisu notifikuje, orchestrátor se probudí ze sleep a nový symbol startuje do sekund; svíčky dne doplní backfill z #221), poll à `WATCHLIST_POLL_CYCLES` zůstává jako fallback pro backendy bez NOTIFY. Probuzení uprostřed minuty spustí plný cyklus jen pro nové pipeline — běžící by duplikovaly zápisy. Sweepy instrumentů běží **sekvenčně** — špička market data lines je vždy jedna dávka. Multiplikátor a burza se čtou z contract details. Ne-futures symbol → alert `instrument_error` + cooldown 30 cyklů. Pád cyklu jednoho instrumentu neshodí ostatní; status se agreguje (součty Greeks/repair, pole `symbols`).

Každá pipeline navíc drží **sekundární runtime následující expirace** (`secondary=True`): sweep v kadenci `NEXT_EXPIRY_SWEEP_EVERY` , zapisuje jen snapshots + levels své expirace (flow/bary patří výhradně aktivnímu řetězu — soubory jsou per symbol).

Roll expirace v settle (#1331, ADR-0039 dodatek): hranice je settle EXPIRACE z jediného helperu `compute/settle.expiry_settle(expiry)` — 16:00 ET (DST přes IANA zónu, #511), kvartální expirace SOQ 9:30 ET. Ptají se ho roll (`instruments.expiry_expired(expiry, now)`), discovery i cache (`CachedDiscovery.unexpired(now)` , `DiscoveryCache.load(now=…)`), hlídka Greeks, setupy (timeout, čas do expirace, invariant `born_after_settle`), sondy T9 a Dyn profil. Orchestrátor pipeline s vypršelou expirací zastaví **po** cyklu minuty settle: ten cyklus je poslední nad starým řetězem — moduly v něm uzavřou, co k settle patří (timeout sond T9 a setupů z živé dávky, paper ordery, agregát `map_state` ze vzorků v paměti), nové setupy ani sondy v něm nevzniknou (`born_after_settle`). Příští minutu plán pipeline založí znovu a `create_pipeline` vybere první expiraci se settle v budoucnu (IBKR vrací dnešní expiraci i po settle — bez filtru by pipeline co minutu rolovala na tutéž). Mezera ve sběru 1–2 min, error 354 z resubskripce kryje grace `SUBSCRIPTION_ERROR_ROLLOVER_GRACE_S` . Do #1331 se rolovalo s novým UTC dnem (3–4 h nad mrtvým řetězem). Audit modulů „jednou po settle“: setupy (otevřené z DB, cesta z partic — nezávislé na staré pipeline), sondy T9 (jen v paměti → timeout v cyklu settle), paper (čte DB každou minutu, settle seance), `map_state` (zápis v cyklu settle, ne až po grace; nová pipeline bez vzorků nic nepřepíše), gamma útes, `volregime` , `em_respect` , verdikty briefingu, vyhodnocení scénářů a IV rank (po settle + grace v nové pipeline, nad partice/DB, idempotentní), T6 a automatický scénář (ráno, rollu se

netýkají), tendence a max pain (OI archivu dne pro novou expiraci). Nová pipeline začíná s prázdnou historií `SetupEngine` — detekce se nad novým řetězem rozjede po zahřátí, úrovně dvou řetězů se nemíchají. Frontend vybírá výchozí expiraci podle settle (`AppState.defaultExpiry` nad `expirySettleUtc`) a automaticky zvolenou přepne v okamžiku settle (#1367); ruční volbu ani doskok #946 nemění. **Kvartální datum podle trading class (#1366, ADR-0039 dodatek):** SOQ 9:30 ET patří jen standardní kvartální třídě (trading class = kořen produktu, `settle.settles_at_soq`); týdenní série nového kontraktu na totéž datum (EW3/QN3), kterou pipeline po rollu front kontraktu sbírá, má settle 16:00 ET. Třídu nese `EngineRuntime.trading_class` / `.settle()` (roll, discovery i cache přes `ExpiryInfo.trading_class` , hlídka Greeks, Dyn profil, setupy, sondy) a `context.trading_class` setupu i sondy (čtení historie, timeout po restartu); řádky před #1366 se čtou podle data. Přepočet setupů a backtest třídu dovozují z tickeru (`settle.history_expiry_settle` : kořenový ticker = týdenní série, pinovaný expirující kontrakt = SOQ), frontend stejně (`settlesAtSoq`).

Další joby: **OI archiv** při startu + retry à 30 min dokud den nemá data (alert `oi_missing` , hranově, jen při otevřeném trhu v obchodní den CME po publikačním okně — kap. 5 „Provozní upozornění při zavřeném trhu“); pokrývá `OI_ARCHIVE_EXPIRIES` nejbližších expirací — základ ΔOI vs. včera. První archiv po založení pipeline (start enginu, roll v settle) čte celý průchod naráz na pozadí (#1208). **Retry, čtení nového UTC dne a obnova po publikaci z minutového cyklu jdou po dávkách mezi cykly** (#1646): `run_minute` průchod jen otevře, dávky (`batch_size` , nikdy souběžně se sweepem — 100 lines) čte `advance_oi_archives` na konci iterace hlavní smyčky, a to jen když se dávka do startu další minuty vejde (`batch_timeout_s` + rezerva 5 s). Zápis a finalita (dvě shodná čtení + pokrytí `oi_final_min_coverage`) přijdou až po celém průchodu; FA validace, kalibrace α, sebekontrola setupů a Forward GEX pak běží na pozadí (IBKR nečtou). Průchod ES (~~1 400 kontraktů, 18 dávek à ~3,6 s) se rozloží do 2–3 minut a minutový cyklus nezdrží. Rozpracovaný průchod se zahodí s novým UTC dnem, při zavření trhu (denní pauza, pátek; čtení nového UTC dne otevřené v zavřeném trhu doběhne) a se stopem pipeline (roll). Pipeline se čtou postupně (ES, pak NQ): o půlnoci UTC má ES IBKR OI nového dne (runtime čte OI podle UTC dne) až po ~2 min a NQ po ~3–4 min, mezitím díry doplní tasty~~ `Summary` ~~ (#664,~~ `oi_filled` ~~), jinak zůstane viditelná mezera; dřív při čtení v cyklu vypadla celá minuta. Log: „OI archiv ES …: průchod N kontraktů po dávkách mezi cykly“, „přečteno n/N kontraktů, pokračuje v další minutě“, „N zapsáno, M chybí“.~~ **~~Diagnostika chybějících kontraktů~~** ~~(#1650, jen log INFO, chování archivu beze změny): každá dávka (i v prvním archivu po startu a rollu) píše řádek „OI archiv ES : dávka 161–240/1344 (EW4 20261023 0/80) — přečteno 0, bez ticku 80, tick bez OI 0, nekvalifikováno 0, chyba 0; 4.3 s, lag smyčky max 0.02 s“ — pozice dávky v průchodu, přečteno/celkem po sériích (trading class a expirace) v pořadí průchodu, důvody chybění a délka dávky. *Bez ticku* = po~~ `reqMktData` ~~ do~~ `batch_timeout_s` ~~ (4 s) nedorazil žádný tick: request neodešel včas (throttle ib_async 45 zpráv/s), smyčka stála, nebo IBKR mlčí. *Tick bez OI* = kotace nebo greeks chodí, ale tick 101 se stranou kontraktu ne (IBKR ho posílá pozdě). *Nekvalifikováno* = sec-def farma kontrakt nevrátila. *Chyba* = výjimka (traceback „fetch_snapshot selhal“). *Lag smyčky* je nejdelší přesah~~ `asyncio.sleep(0.25)` ~~ sondy souběžné s dávkou — jak dlouho nejvýš smyčka nezpracovávala příchozí ticky (CPU práce v témže vlákně, GIL držený vláknem z~~ `to_thread` ~~). Čtení: lag ≥ ~1 s u vypadlé dávky = zablokovaná smyčka; „bez ticku“ bez lagu = zahlcené odesílání nebo IBKR; „tick bez OI“ = pozdní tick 101; vypadávají jen série, které čte i sweep (aktivní a sekundární řetěz) = sdílený~~ `Ticker` ~~ ib_async podle conId — u nich „tick bez OI“ hypotézu 3 nevylučuje, protože~~ `ticker.time` ~~ posouvají i kotace sweepu (rozlišení platí pro obnovu po dávkách, která se sweepem souběžně neběží; první archiv po startu a rollu ano, #1649). Řádek průchodu nese~~ **~~důvod nefinality~~**~~: „N zapsáno, M chybí — pokrytí P % (na desetiny), změněno C, chybí dřív archivovaných A“. Finální je jen průchod po publikaci s C = 0 (žádná hodnota se proti archivu dne nezměnila; nový kontrakt i celý první snímek dne se počítají jako změna), A = 0 (chybějící kontrakt, který archiv dne už má, nejde potvrdit — #494) a P ≥~~ `oi_final_min_coverage` ~~; pak následuje „OI archiv … je finální (dvě shodná čtení)“. Do #1646 se obnova četla v cyklu celá (~~100 s) a každých ~31 min od 12:30 UTC chyběla minuta ES i NQ. **POZOR: OI se čte přes generic tick 101 i pro FOP** (tick 588 na FOP nedodává nikdy — ADR-0001 v3; hodnota se čte podle strany kontraktu, opačná strana je validní 0.0). **Auto-rozšíření obálky strikes** (grow-only, capped → alert) + runtime změna `strike_range_points` ze Settings UI (překlopí pipeline). **OI zdi** (#851): `compute/oiwalls.py` počítá maximum OI per strana nad širokým archivem (ne nad snapshoty omezenými obálkou) s cache na verzi archivu expirace dne (`OIEodRepository.archive_version` = max `captured_ts` IBKR a max `tasty_captured_ts` , #1605) — archiv se přes dopoledne dopisuje a tasty řádky mimo obálku se po publikaci CME obnovují, takže klíč jen na den nebo jen na `captured_ts` by zamrzl jako Max Pain (#826); vlastní řada `oiwalls/` , LEVELS_SCHEMA se nerozšiřuje (ADR-0008). Týž klíč má cache Max Pain v setupech a v tendenci.

Široký OI archiv z tasty (#828, oprava #1605): striky aktivní expirace, které IBKR nezměřilo, archivuje do `oi_eod` tasty `Summary.open_interest` (`wide_oi_once` → `tasty/wideoi.wide_oi_pass` v kadenci extended smyčky; subskripce účelu `wide` v pásmu

`GEXLENS_TASTY_NEAR_BAND_PCT` kolem ceny; vypíná `GEXLENS_TASTY_WIDE_OI`). Za pokrytý se bere jen strike s IBKR řádkem dne (vyplněný `captured_ts`, přes všechny série). Tasty řádky se obnovují celý den a zapisuje se jen změněná hodnota (porovnání per PK proti tasty řádku), takže minutový běh obvykle nezapíše nic a po publikaci CME (07:00 CT) nesou tasty řádky novou publikaci. IBKR řádek je pro tasty nedotknutelný: stráž přímo v SQL (`upsert_many`: `ON CONFLICT … DO UPDATE … WHERE oi_eod.captured_ts IS NULL AND oi_eod.oi != excluded.oi`, jen sloupce `oi` a `tasty_captured_ts`) drží i při souběhu s IBKR archivací a nevynuluje greeks. Obě cesty zapisují řádky seřazené podle PK (stejné pořadí zámků → bez uváznutí na PG). Když IBKR strike doarchivuje (rozšíření obálky, `_archive_new_strikes`), tasty řádek přepíše a ten pak nese oba časy. Detektor finality (#463, `snapshot_by_class`) i srovnání OI IBKR × tasty (#664, `snapshot(…, ibkr_only=True)`) berou jen IBKR řádky — tasty řádek není předchozí IBKR čtení a kontrakt, který IBKR nikdy nezměřilo, finalitu neblokuje. Log enginu: „Široký OI z tasty (#828): N řádků zapsáno/obnoveno". Do #1605 bral zapisovač za pokryté i vlastní řádky: zápis z 00:00 UTC (před publikací) se do konce dne neobnovil, ~55 % Σ OI 0DTE řetězu jelo na předchozí publikaci a publikace dne expirace mimo obálku se do archivu (R4) nezapsala. Dny před nasazením opravy to v archivu nesou dál, znovu je pořídit nejde.

Nastavení připojení ze Settings UI (#446, #950, #992): orchestrátor čte watchlist + runtime nastavení z DB každý `WATCHLIST_POLL_CYCLES`-tý cyklus nebo po `LISTEN/NOTIFY`; od #992 posílá NOTIFY i `PUT /settings/{key}` (stejný kanál `gexlens_watchlist` — po probuzení se čte obojí jedním průchodem) a **bez spojení k IBKR se DB čte každý cyklus** (`runtime_settings.should_poll_settings`), takže změna portu platí do sekund i v reconnect smyčce (dřív až za ≤ 5 min). Hodnota uložená v DB **přebíjí** `.env` — je to záměr #446, ale po `docker compose up -d engine` s přepsaným `.env` to jinak nešlo poznat (2. 9.: connect na 4001 a o sekundu později skok zpět na 7496); engine to od #992 při prvním cyklu hlásí `WARNING`em s návodem, co změnit. **Ruční přepojení** (#950): `POST /engine/reconnect {target: ibkr|tasty|both}` zapíše serverem generované razítko `reconnect_request_*` do `settings` (klíče schválně nejsou ve `WRITABLE_SETTINGS`, přes `PUT /settings` je podvrhnout nejde); engine reaguje na **změnu** razítka (`pending_reconnects`), výchozí stav si načte `seed_reconnects` jednou před hlavní smyčkou — chybějící klíč se pamatuje jako `None`, takže první požadavek po startu neshoří (#957). IBKR: `ib.disconnect()` + supervisor; tasty: `DxLinkStream.force_reconnect()` (zavře socket, standardní `run` smyčka udělá reconnect i resubskripci). Přepojení = 1–2 min díra ve sběru, UI si vyžádá potvrzení. **Denní roll expirace**: vypršelá pipeline se zastaví a další cyklus založí novou s čerstvou discovery (bezobslužný přechod přes víkend). **Retention purge** jednou denně v zavřeném trhu (#1337, `storage/retention.purge_day_due` nad `compute/marketclock.is_market_closed`): v pracovní dny v denní pauze CME 16:00–17:00 CT (léto 21:00–22:00 UTC, zima 22:00–23:00 UTC), o víkendu kdykoli; start jen s rezervou 15 min do otevření a jednou za kalendářní den CT. Dřívější klíč `GEXLENS_RETENTION_PURGE_TIME_UTC` (pevných 21:30 UTC = v zimě 15:30 CT, otevřený trh) zrušen, v `.env` se ignoruje. **Měření disku (#773) i purge běží na pozadí** (`background.BackgroundJob`, vlákno přes `asyncio.to_thread`): cyklus je jen odpálí a v další minutě vyzvedne doběhlý výsledek, nečeká na ně (měření přes bind mount až 121 s, purge ~2,5 min dřív vynechávaly minutové cykly); druhý běh se nespustí, dokud předchozí neskončil, výjimka se loguje jako `ERROR` „… selhal na pozadí". Alert `disk_space` / `disk_limit` tak přichází o cyklus později.

Bary podkladu (#221): **Backfill 1min barů** při startu pipeline (aktuální den + retention okno, reqHistoricalData pod pacing guardem, upsert podle ts_min — živý stream a backfill se nedublují; od #1055 (v1.6) nese doplněný bar `source = ibkr_hist` a **změřenou minutu nepřepíše** — přednost původu `bar_source_rank`: měřený > `ibkr_hist` > `tasty_candle`, živý zápis historickou hodnotu naopak nahradí vždy). **Hlídání tiché ztráty barů** (`BarsStallDetector`): když ≥ `BARS_STALL_ALERT_MINUTES` (default 3) při otevřeném trhu nedorazí žádný 5s bar, odejde alert `bars_stalled` — i když stojí i spot (#1347: 3. 9. stál stream i spot 72 min bez alertu; text rozliší „spot přitom žije" / „stojí i spot") (typicky mrtvé TWS farmy po noční přestávce — pomáhá restart TWS); po návratu streamu alert `bars_recovered` + automatický re-backfill dnešního dne doplní díru. Při zavřeném trhu podle rozvrhu CME (pre-open, denní pauza) se detektor ani nekrmí (#1307). **Rekonstrukce děr z dxFeed Candle** (#617, v1.3): jednou po startu pipeline (`_candle_gap_backfill`, jen s běžící tasty větví) se pro aktuální seanci spočítají minuty, které IBKR historical nedodal, a doplní se z dxFeed `Candle` (historie od `fromTime`, bez pacing limitu) přes vlastní krátké spojení mimo živou datovou cestu — sdílený handshake `tasty/dxlink.py`. `backfill_gaps` výsledek ještě jednou filtruje na chybějící minuty, takže měřená minuta se nemá jak přepsat; selhání se jen zaloguje. ADR-0024 platí dál pro opční vrstvu (Greeks zpětně neexistují). Past z ADR-0027: streamer symbol se nesestavuje (`/ESU6:XCME` s hlubokým `fromTime` vrací rok 2016), bere se hotový z chain endpointu. UI doplněné minuty hlásí bannerem (sbírá se ze všech barů dne, ne jen z těch na ose snapshotů — večerní minuty Globexu na osu opcí nepadnou, #974). **Hlídka Greeks po settle** (#959): `greeks_watch_applies(expiry, now)` vypne `greeks_stalled` pro expirující řadu po jejím settle (`compute/settle.py`, DST-korektně) a detektor se nekrmí — vypořádaný řetěz se přestane kotovat legitimně (sekundární řada měla v téže vteřině plný počet) a pipeline nad ním běží až do půlnoci, kdy `expiry_expired`

překlápí podle kalendářního dne. V tomhle okně nevznikají setupy (#1324): `SetupEngine._detect_new` se ptá `compute/setups.born_after_settle(runtime.expiry, now)`, jediného predikátu invariantu „setup se vztahuje k živé expiraci". Hranice je settle expirace podle ADR-0039 bod 2 (`setup_settle_ts` → `expiry_settle_ts` : 16:00 ET, kvartální SOQ 9:30 ET), táž jako timeout setupu. Kromě okna šablony T3 (`minutes_to_expiry`) detektory expiraci nehlídají. Do rollu na settle tak na otevřeném Globexu v okně settle → 00:00 UTC 1–3 h denně žádný setup nevznikne (léto 20:00–24:00 UTC, zima 21:00–24:00 UTC, bez denní pauzy CME); skutečná oprava, roll expirace v settle, je #1331 — pipeline s ADR-0039 bod 2 zatím v souladu není (`expiry_expired` roluje běžnou expiraci podle UTC dne, sondy T9 `probes.probe_settle` a čas do expirace Dyn profilu počítají se `settle_ts`). Nečitelná `runtime.expiry` invariant vypne jen s `WARNING` v logu (1× za expiraci). Týž predikát vyřazuje historické řádky při čtení (`SetupsRepository` : `realized_since` pro brzdy a bránu, `closed_for_calibration` , `closed_since` pro sebekontrolu, `closed_between` pro kouče; souhrn je spočítá do `after_settle_count`) a replay `scripts/backtest_setups.replay` (backtest i walk-forward) v minutě po settle vlastní expirace nového kandidáta nevytvoří (minuty dál přehrává, otevřené setupy se vyhodnocují jako dřív; hranice `setup_settle_ts` místo dřívějších pevných 20:00 UTC). Zatím je nevyřazuje gamma útes `next_setups` (#1331). V DB řádky zůstávají.

Vyhodnocení setupů po cestě ceny (#1320, ADR-0004 dodatek 2026-09-30, čistá funkce `compute/setups.walk_setup_path` , sdílí ji živý `SetupEngine` i `scripts/recompute_setup_outcomes.py`): otevřený setup se uzavírá jen nad minutovými bary v pořadí, bez vynechané minuty; spot se nehodnotí nikdy (3. 9. 14:17–15:30 UTC stál stream i spot, engine 72 min „viděl" rovnou čáru a první živý bar zapsal jako cíl místo stopu). **Cesta začíná za barem vstupu** — barem, jehož close je entry (poslední bar dávky cyklu: obvykle N−1, opožděný cyklus nese i bar N, NQ sbírá bary až po ES). Jeho ts jde do `context.entry_bar_ts` , odkud cestu po restartu začne engine i offline přepočet; řádky bez něj (před #1320, resp. vzniklé nad spotem do #1346) začínají za minutou N−1. **Dávka cyklu bez baru setup nevytvoří** (#1346): minuta jen vyhodnotí otevřené setupy, do historie detektoru ani feature logu nejde a toky (přírůstky volume) připadnou další minutě s barem; opožděný bar přijde v další dávce a detekce proběhne nad agregátem dávky (vstup = close posledního baru). Dřív se minuta sestavila ze spotu (O = H = L = C) a při zamrzlém spotu vznikaly setupy nad cenou, kterou trh neměl (ES 1004, NQ 1048, ES 1049). Takové historické řádky vyřazuje ze statistik trvalá značka `context.excluded` (kap. 12, `recompute_setup_outcomes.py --exclude`). Stop-first platí uvnitř jedné svíčky (#257), `closed_ts` = ts baru, který úroveň zasáhl. Když živá dávka cyklu na poslední vyhodnocený bar nenavazuje (výpadek streamu, restart, opožděné cykly), `SetupEngine` dotáhne bary z partic `derived/{sym}/bars` (`parquet_store.read_bars` , jen minuty do `now − 2` , rozdělanou minutu ne; běžná minuta partici nečte); táž minuta z živé dávky vyhrává. Chybí-li minuta běžícího trhu (`marketclock.is_market_closed`) a pozdější bar už existuje, je to **díra**: vyhodnocení čeká (log INFO „čeká na doplnění") a engine si díru **sám vyžádá** (`request_bars` → `__main__.backfill_setup_gap` : IBKR historical UTC dnů díry přes zápisovou stráž `write_backfilled` , jedno doplnění naráz, jedna žádost na díru, log `Doplnění díry … pro vyhodnocení setupů: N barů`). Re-backfill stall detektoru na to nestačí — bez pohybu spotu nečítá, takže při zamrzlém spotu (3. 9.) nepřijde; při výpadku celého IBKR díru plní doplnění z tasty. Na **každou** díru čeká engine nejdéle `PATH_GAP_WAIT` = 15 min (posun cesty čekání nuluje, další díra dostane vlastní čekání i žádost), pak ji přejde s `WARNING` „bary … se nedoplnily … výsledek neověřený"; přepočet takový výsledek později opraví nebo označí. Bez jediného dalšího baru setup jen čeká, nic se nevymýšlí. **Timeout** po settle vlastní expirace (`settle.expiry_settle`) = close baru, který v settle končí (v létě 19:59 UTC), `closed_ts` = settle. Důvod: výstupní cena platí v okamžiku settle a čas nezávisí na cyklu ani restartu, takže setup zůstane v seanci a obchodním týdnu své expirace (1031 se dřív uzavřel v pondělí po Labor Day a spadl do brzd dalšího týdne). Chybí-li bary až k settle, čeká se jako u díry, pak close posledního známého baru; bez jediného baru 0 R s `WARNING` . Víc setupů uzavřených v jednom kroku (dohnání díry, restart) se zapisuje **v pořadí barů**, ne vzniku — série stopů směru (#302) i cooldown kontra-setupu (#252 C) na pořadí závisí. Po restartu se cesta aktivních setupů přehraje od baru vstupu z partic, MFE/MAE tak zahrnou i minuty výpadku. Verze mechaniky se nemění (v5 měla takto hodnotit od začátku), historii srovná `scripts/recompute_setup_outcomes.py` (kap. 12). Svátky a zkrácené seance zná rozvrh od #1308 (ADR-0046); díru dělá jen mimořádné zavření. **Týmž krokem** (`walk_setup_path`) hodnotí od #1345 i sondy T9 (`probes.py` : cesta od baru vstupu, díra z partic `bar_reader` + čekání `PATH_GAP_WAIT` , timeout v settle expirace sondy, `closed_ts` = bar zásahu nebo settle), paper účet (`paper.py` : spot se nehodnotí nikdy, cesta orderu od vzniku / baru fillu, díra z partic + čekání, v settle seance výstup za close baru končícího v settle s `closed_ts` = settle, čekající order se v settle zruší) a replay backtestu i walk-forwardu (`scripts/backtest_setups.replay` a `replay_probes` : timeout v settle místo `active` #1369, díra v datech se přejde a spočítá ve sloupci `gaps`). Výsledky se se setupy srovnávají 1:1.

Odolnost: **reconnect nesmí umlknout** (#770): `_supervise()` je odolná smyčka nad `_try_connect()` — výjimka v iteraci (padlý odběratel stavu, selhaná resubskripce) se zaloguje a jede se dál, selhaná resubskripce jde rovnou na reconnect; **watchdog**

běží záměrně mimo supervisora a kříší mrtvou smyčku (čítač `ConnectionManager.supervisor_restarts` — nenulová hodnota je nález, do logu jde jako vzkříšení supervisora), po `RECONNECT_STALL_ALERT_S` hlásí `connection_stall` (18. 8. byl engine 8 h offline bez jediného řádku). ConnectionManager watchdog (heartbeat 30/15 s + exponenciální reconnect + plná resubskripce — **vč. spot tickeru a realtime barů podkladu** přes `on_resubscribe`), spot fallback last → marketPrice → close (start i o víkendu), discovery s timeoutem a retry (sec-def farm výpadky), výjimka v cyklu nikdy neshodí smyčku, pacing guard historical requestů (≤60/10 min, dedup, priorita).

Provozní upozornění při zavřeném trhu (#1307)

Engine běží i o víkendu, v denní pauze CME (po–čt 16:00–17:00 CT) a o svátcích: sweep, OI archiv i hlídače. Mimo seanci ale TWS nedodává Greeks ani kompletní kotace, IBKR po páteční uzávěrce neposílá cenu a CME nepublikuje OI. To není porucha. Do #1307 to hlídače nerozlišovaly: 26. 9. 2026 chodilo `oi_refresh_failed` pro ES i NQ à 30 min, `spot_fallback` po nočním reconnectu DXLink a `strikes_stalled` po sobotním rollu.

Všechny hlídače níže berou jeden predikát „očekávají se data?" z rozvrhu `compute/marketclock.is_market_closed` (ADR-0023, DST-korektně; stejný vzor jako signály #968 a hlídač tasty streamu #1228). Od #1308 (ADR-0046) zná rozvrh i svátky a zkrácené seance z pravidel NYSE (`compute/settle` : `us_market_holidays` , `is_early_close`): o Vánocích, Novém roce a Velkém pátku je Globex zavřený, o ostatních svátcích obchoduje do 12:00 CT, zkrácená seance (den po Thanksgiving, Štědrý den, 3. 7.) končí v 12:15 CT a settle i US RTH v 13:00 ET. O Vánocích tak neodejde ~26 upozornění `feed_crosscheck` , o Thanksgivingu ~12. Hranu zavřeno → otevřeno zjišťuje pipeline jednou v `run_minute` , před sweepem.

Upozornění	Při zavřeném trhu	Po otevření (neděle / po pauze 17:00 CT)
`oi_refresh_failed` , `oi_missing`	Obnova ani retry neběží, selhání jde jen do logu. Zůstává jediné čtení na začátku nového UTC dne: runtime čte OI podle UTC dne, víkendová mapa by jinak OI neměla.	Chybějící OI (nebo nefinální snímek po okně) se čte hned — hrana otevření natáhne čítač retry. Upozornit smí jen obchodní den CME po publikačním okně: v neděli CME nepublikuje, snímek neděle se neobnovuje a selhání jde do logu. První upozornění po víkendu tak přijde nejdřív v pondělí po 07:00 CT.
`spot_fallback`	Nepřepíná se. Fallback z doby před uzávěrkou drží do návratu IBKR.	Ticho IBKR se měří od okamžiku otevření, ne od pátečního ticku.
`greeks_stalled` , `strikes_stalled`	Detektory se nekrmí.	Hrana otevření vynuluje repair kola a backoff scheduleru (aktivní i sekundární řetěz) a stav obou detektorů. Čítače BS fallbacku zůstávají: kontrakt bez TWS greeks dostane BS dopočet z čerstvých kotací hned prvním sweepem.
`bars_stalled` + obnova streamu	Detektor se nekrmí (pre-open: indikativní kotace se hýbou, TRADES bary nevznikají). Stav se nenuluje.	Stall ze seance skončí `bars_recovered` a re-backfillem díry.
`greeks_bs_fallback` + remediace (#877,	Hodiny epizody se pozastaví (`BsFallbackWatcher.observe(market_closed=True)`): bez alertu, bez	Nový otevřený úsek: připomínka nejdřív po 15 min otevřeného

Upozornění	Při zavřeném trhu	Po otevření (neděle / po pauze 17:00 CT)
#1309)	hodinových připomínek a bez resubscribe/reconnectu — TWS mimo seanci model greeks nepočítá. Epizoda nekončí: ohlášení i počet remediačních pokusů nuluje jen skutečný návrat za otevřeného trhu, takže trvalá porucha dostane nejvýš 2 zásahy za celou epizodu, ne každý Globex večer (#877 C). Podíl pro `/status` se měří dál.	trhu (první sweepy jedou z BS, než TWS model naběhne), délka v textu = součet otevřených úseků („… min otevřeného trhu"); zbývající pokus remediace nejdřív 30 min × pořadí pokusu od otevření. Epizoda ohlášená před zavřením skončí zprávou o návratu, pokud greeks chodí.
`vol_concentration` (#208, #1309)	Nehledá se. Objem kotací je zmrzlý z poslední seance a po sobotním rollu se nový sekundární řetěz plní postupně (26. 9. 00:05 UTC „7900C 19,7× medián" nad 10 snímky s páteční volume). Sekundární sweep běží dál.	Beze změny, první minutou po otevření.
`feed_crosscheck`, `chain_fallback`, `feed_probe`	Křížová kontrola vrací `quiet` a nuluje série. dxFeed posílá snímek posledních hodnot při resubskripci i přechodu seance, takže „tasty čerstvé" tu nic nedokazuje.	Série od nuly, prahy beze změny.
`connection_stall`, `disconnect` (API)	Jen log (watchdog enginu), API hranu výpadku nenatáhne.	Trvá-li výpadek, `connection_stall` nese celou délku a `disconnect` odejde s prvním statusem po otevření.
`competing_session`, `subscription_error`	Chyby 10197 a 354 jdou do diagnostiky (`/status`), do alertovacího prahu ne, takže nespotřebují ani cooldown.	Práh se plní od nuly; drží-li mobil feed i po otevření, ohlásí se to do pár minut.
`degraded_start`, `instrument_error`	Jen log. Sobotní roll v 00:00 UTC se kryje s denním odpojením IBKR.	Trvající výpadek ohlásí `disconnect` ; setup instrumentu se po cooldownu zkusí znovu a jeho selhání se ohlásí.

Při otevřeném trhu se nic neumlčuje („Otevřený trh = vidět vše"). Jediná změna v seanci jsou OI alerty. Jsou **hranové**: jednou za (symbol, den) a znovu až po úspěšném čtení. A čekají na **publikační okno**: v noci před 07:00 CT dodává IBKR předpublikační čísla, takže selhání jde jen do logu. Kdyby se ohlásilo, spotřebovalo by hranu dne a skutečná porucha „okno proběhlo, OI pořád nedorazilo" by se neohlásila. Retry běží à 30 min dál, další selhání jdou do logu.

Svátky rozvrh nezná (ADR-0023 bod 4): v den celodenního zavření CME vypadá trh jako otevřený. Tam platí datové proxy (spot fallback nepřepne, když mlčí i tasty; křížová kontrola je `quiet`, když mlčí oba) a hranové alerty. OI, Greeks a striky tak stojí nejvýš jedno upozornění na druh. **Dvě cesty ale svátek nepokrývá:**

- `feed_crosscheck` „Oba zdroje mlčí … uvnitř US RTH" (#1228) se opakuje po cooldownu 15 min, protože `outside_us_rth` svátky nezná. Celodenní svátek (Vánoce 25. 12.) to je ~26 upozornění. Thanksgiving 26. 11. (pauza od 12:00 CT) dá ~12 a 27. 11. (uzavření ve 12:15 CT) ~11.
- `spot_fallback` přepne, když DXLink po reconnectu pošle snímek posledních hodnot a tasty tím vypadá živě.

Řešení čeká na rozhodnutí v #1307.

Log: `Trh ES otevřel (rozvrh CME) — repair a backoff stav ze zavřeného trhu vynulován`, `oi_refresh_failed ES: trh je zavřený (rozvrh CME) — bez upozornění, jen log: …`, `oi_missing ES: CME pro tento den OI ještě nepublikoval (před oknem nebo víkend) — bez upozornění, jen log: …`, `degraded_start ES: trh je zavřený (rozvrh CME) — bez upozornění, jen log: …`, `IBKR spojení chybí N min, trh je ale zavřený`

`(rozvrh CME) — bez upozornění`. Testy: `engine/tests/test_instruments.py` (minutové sekvence pátek → pondělí po okně, neděle 16:59 a 17:00 CT, denní pauza, konec DST 1. 11., Vánoce), `test_scheduler.py`, `test_subscription.py`, `test_spot_fallback.py`, `test_crosscheck.py`, `test_connection.py`, `api/tests/test_crud_alerts.py`.

Obchodní den místo kalendářního (#1309)

`trading_session_date` vrací v sobotu a v neděli před otevřením Globexu kalendářní den bez seance a OI archiv i partice mají klíče podle UTC dne včetně víkendu. Kdo na den váže US open, settle, publikaci OI nebo srovnání „proti předchozímu dni", ptá se jediného predikátu `compute/settle.is_trading_session` (po–pá mimo svátky NYSE; frontendový protějšek `instrument/tz.isTradingSessionIso`; kalendář svátků z pravidel, #1308, ADR-0046). Sdílí ho epizody sentimentu (ADR-0037), scénáře a verdikty dne, publikace OI (`_oi_published`), backfill EM respect, pokrytí hlubokých barů (`deepbars.task_is_covered`), PDC verdiktu dne a scénáře (`scenario_auto._settle_close`, hranice `settle_ts` — po zkrácené seanci 13:00 ET, #1595), okno remediace BS (`marketclock.outside_us_rth`), dny Forward GEX (`gexforward.trading_days_until_friday`) a ve frontendu EOM expirace (`expiry.ts`), termín scénáře a „včerejšek" Briefingu, grafu a deníku (`tz.previousTradingSessionIso`, PDC v `api/briefing.previousSessionRange` s hranicí `settleMs` — protějšek `_settle_close`, #1596):

- **ΔOI** (`GET /oidelta`, `/chain`, `/replay` → `oi_prev`, T6): základem je poslední archivovaný **obchodní** den (`OIEodRepository.latest_trading_day_before`), pondělí se srovnává s pátkem. Víkendové klíče archivu jsou kopie pátku (sobota, neděle před otevřením) nebo už pondělí (neděle po otevření), takže pondělí proti neděli dávalo 21. 9. 2026 u většiny expirací ΔOI = 0 a hlas ΔOI ve verdiktu dne ES (0DTE) „převaha put" místo „převaha call". `/oidelta` navíc počítá změnu jen na stranách, které nesou oba archivy (obálka se liší: sekundár 160 striků, aktivní ~560); součty OI jsou z aktuálního archivu celé. O víkendu ukazuje pátek proti čtvrtku.

- **FA validace a ranní kalibrace α** (#232) běží jen v obchodní den a párují volume/netflow posledního obchodního dne s ΔOI do dneška. Den se řeže začátkem denní přestávky 16:00 CT (`marketclock.daily_close_utc`: v létě 21:00 UTC, od konce US letního času 22:00 UTC; do #1460 pevně 21:00 UTC, takže v zimě by poslední hodina Globexu vypadla). Sobotní běh dřív ukládal bod „pátek → sobota" (ΔOI ≈ 0) a dedup dne pak zablokoval poctivý bod „pátek → pondělí". Starší bod `fa_validation` s víkendovým `next_day` (19 řádků od 24. 7. 2026, naposledy 25. → 26. 9.) dedup nepočítá (`FaValidationRepository.exists`), takže pondělní běh bod pátku přepočítá a upsert víkendový řádek přepíše; ostatní zůstávají jako audit (tabulku nic jiného nečte). `fa_alpha_history` `next_day` nemá, víkendový bod (18. a 25. 9. 2026, medián 0 → α beze změny, #1172) proto dál blokuje bod téhož pátku.

- **FA validace a kalibrace α jen nad finálním OI snímkem** (#1314, `InstrumentPipeline._run_fa_jobs`): dřív běžely hned po prvním archivu dne v 00:00 UTC, tedy nad OI před publikací CME (07:00 CT) — ΔOI ≈ 0 (`doi_abs_sum` 0 u ES/NQ 22.–24. 9.) a denní dedup pak bod zamkl. Teď se počítají až nad snímkem se dvěma shodnými čteními po publikačním okně (`oi_final`); do té doby se jen propíše uložená α do runtime. Symbol bez kalibrace po bodu s mediánem ≤ 0 už nedostane `fa_alpha` s α 0 (jen řádek historie s `alpha_after` 0 = nekalibrováno) — tak vznikla NQ α = 0 a FA vrstva NQ byla vypnutá. Historii opraví `scripts/recompute_fa.py` (dry-run → report `{data}/reports/recompute-fa-*.md`, `--apply` po potvrzení): přehraje tytéž funkce den po dni nad finálními archivy (filtr stran níže tedy zdědí), dny se snímkem pořízeným před publikací přeskočí jako neověřitelné, nic nemaže. Report má u α starý/nový počet stran a sekci „Body v produkci bez nového přepočtu" (body, které přehrání nevytvoří — `--apply` je nechá beze změny); logy vyřazených stran vypisuje na INFO. Přehrání počítá zdrojem mechaniky kódu, proto srovnává jen body a stav α téhož zdroje (`source`, #1590): body midpointu po bumpu v6 nejsou „staré body" tiskové α.

- **Jen strany změřené po publikaci** (#1605, `measured_oi_pair`, `OIEodRepository.measured_values_for`; varianta B+, vlastník 19. 10. 2026): finalita potvrzuje jen univerzum IBKR čtení, proto kalibrace α i FA validace berou jen strany, jejichž všechny řádky (přes série) jsou změřené po publikačním okně svého dne, a to v obou dnech:

  - **strana D+1** jen z IBKR měření po publikaci D+1 (`captured_ts ≥ oi_publication_utc(D+1)`);
  - **strana D** z IBKR měření po publikaci D, nebo z tasty řádku (`captured_ts` NULL) s `tasty_captured_ts ≥ oi_publication_utc(D)` (`include_tasty`). Tasty obnoví hodnotu i čas jen při změně OI, takový řádek tedy nese publikaci D. Tasty řádky s časem před publikací D (07:00 CT = 12:00 UTC v létě, 13:00 UTC v zimě) vypadnou. **Limit (#1626, varianta A, vlastník 19. 10. 2026):** `Summary.openInterest` nese publikaci D už kolem 01:45–01:55 UTC dne D

(měřeno 19. 10. 2026: hodnota po 02:10 UTC = IBKR po publikaci ve všech porovnaných kontraktech) a po 12:00 UTC se obvykle nemění, takže se znovu nezapíše. Strana D z tasty proto projde jen výjimečně (změna OI přes den) a B+ v praxi dává totéž co B — data se nekazí, strana bez měření je „nevíme". Na straně D+1 by stejné pravidlo znamenalo „ΔOI ≠ 0": vypadly by skutečné nuly a α by se vychýlila nahoru. FA validace navíc vyřadí stranu D z tasty, která nemá řádek ve snímku D (leží mimo univerzum IBKR, volume je neznámý, ne 0); log ji počítá jako „D tasty bez volume".

Strana se smíšenými sériemi se počítá, jen když je změřená každá série. Strana, kterou archiv nese z tasty bez obnovy po publikaci nebo z IBKR čtení před publikací, je „nevíme", ne ΔOI 0 (#1172): klíče se zužují na průnik obou dnů (u FA validace volume i OI před/po), takže chybějící hodnota nedá ΔOI = −oi_before. U IBKR řádku se `tasty_captured_ts` nečte: IBKR razítkuje začátkem cyklu, ne časem zápisu, a řádek tasty → IBKR nese oba časy; IBKR řádek chrání stráž v SQL. Log bodu uvádí počet stran s |net| ≥ 25 a kolik jich vypadlo bez měření po publikaci (rozpad D+1 / D) a kolik započtených stran nese D z tasty („D tasty"; FA validace obdobně obchodované strany). Expirace bez takových stran se neukládá ani nezamyká (log „bez stran změřených po publikaci v obou dnech") a zkusí se znovu — bod se tak častěji dopočítá až při rollu v settle (nová pipeline má `oi_final=False`, finalita večer znovu spustí FA joby nad archivem rozšířeným o doarchivaci obálky); `recompute_fa.py` přehrává archiv z konce dne, shodne se proto s večerním bodem. Do #1605 nesly takové strany předchozí publikaci a ΔOI přesně 0: NQ měl od 5. 10. 2026 medián přesně 0 v 7 z 8 bodů.

- **Kalibrace α a FA vrstva od mechaniky v6** (#1403; ADR-0057 bod 6, #1460, #1580). Pod v5 se nic z toho nemění.
  - Kalibruje se jen řetěz, který byl **celou seanci D sekundárem**, tedy první expirace po D. Náhradní řetěz se nehledá: D+2 píše netflow dne D až po rollu v settle (~20:05 UTC) a jeho bod by nesl 55 minut toku. Pod v5 vyhrává nejvzdálenější expirace s bodem, od 7. 10. tedy D+2 (#1580).
  - Netflow jen se zdrojem `aggressor`. Den před bumpem (midpoint) bod nedá.
  - **Den s mezerou trade větve bod nedá:** runtime aktivního i sekundárního řetězu zapíše minutu, kterou za otevřeného trhu neměřil, do `derived/{sym}/{exp}/netgap/` (i první minutu po restartu). Kalibrace čte seanci D od 17:00 CT předchozího dne (dvě UTC partice) do řezu. Bod nedá ani řada netflow, která končí víc než 5 min před řezem (engine spadlý přes konec dne značku do seance D nezapíše).
  - Pravidlo platí **bez výjimky**: **roll v settle mezeru nedělá** (varianta C, ADR-0057 doplněk 20. 10.).
    - Řetěz D+1 si nechá tracker sekundáru. Nová pipeline ho převezme v další iteraci orchestrátoru. Do té doby do něj tečou tisky přes trasy, které pod tisky nesou tracker svého řetězu, a plán tasty subskripcí drží TimeAndSale D+1 dál (čekající předání).
    - Pásmo nového aktivního D+1 pokryje i striky sekundáru (přes roll jen roste, výjimka z ADR-0002 bod 3), takže tisky chodí na všech stranách, které sekundár měřil. Širší pásmo drží celou seanci D+1; delší je jen sweep křídel.
    - Net navazuje bez mezery, včetně tisků od posledního zápisu sekundáru (každý N-tý cyklus). První minuta aktivního D+1 je měřená, když vzorky pokryjí hranici; výpadek v neuzavřeném okně sekundáru udělá mezeru z prvního řádku.
    - CumΔ symbolu navazuje z flow aktivního řetězu a nese tok až od vzorku minuty rollu. Mezera rollu v CumΔ pod v6 zaniká.
    - Log předání: „Roll ES: aktivní řetěz 20261020 převzal tracker sekundáru (net N stran) — tisky bez mezery (#1460)".
    - Nevznikne-li pipeline hned po rollu (spojení, chyba setupu, cooldown), tracker propadá s varováním „pipeline nevznikla hned po rollu — tracker sekundáru … propadá". První minuta nového trackeru je mezera a den bod nedá, stejně jako po restartu procesu.
  - `fa_alpha` i `fa_alpha_history` nesou `source` (aditivní ADD COLUMN přes `ensure_schema` enginu i API, na PostgreSQL `IF NOT EXISTS`; NULL = midpoint). Stav jiného zdroje se bere jako nekalibrovaný: první bod z tisků jde přímo (α se při bumpu resetuje sama, bez ručního `set_state`) a historie zůstává. Stav s 0 dny bere první bod přímo i pod v5.
  - **FA vrstva jede až po 5 dnech kalibrace z tisků** (`FA_MIN_DAYS`, `fa_calibration.live_alpha`), do té doby α 0, i bez stavu. Výchozí `GEXLENS_FLOW_OI_ALPHA` platí jen pod v5; pod v6 bez nastavené α (start před prvním obnovením, chyba DB) platí 0 (`EngineRuntime.fa_alpha`). Log „FA ES vypnutá do kalibrace z tisků: n/5 dnů"; stavová lišta frontendu ukáže „FA nekalibrováno n/5" z `/fa/alpha` (`source`, `min_days`) proti `/status.cumdelta_source` (#1632). Frontend v režimu FA

kreslí minutu se snímkem bez FA řad (oiest, levelsfa, gexprofilefa) jako mezeru — nedrží poslední FA profil ani nepřejde na měřené OI (ADR-0057 bod 4).

- **Verdikty dne** (#1091, #1595, ADR-0035 dodatek 2026-10-17): zapisuje **jen engine** — `DayVerdictWriter` (`verdict_writer.py`) v minutovém cyklu každého symbolu s pipeline, v obchodní seanci od jejího začátku (17:00 CT předchozího dne) do US openu: do open − 15 min každých 5 min (minuty :00, :05, …), v [open − 15 min, open) každou minutu. `BriefingVerdictRepository.upsert_verdict` drží jeden řádek (`session_date`, `symbol`; `ON CONFLICT` na `uq_briefing_verdicts_session_symbol`): `created_at` = první zápis, `updated_at` = poslední, přepis mění jen `verdict`, `score`, `votes`, `rules_version`, `updated_at`; platí poslední stav před openem. Jediná stráž `settle.verdict_writable(session, now)` = `is_trading_session` ∧ now < `us_open_ts` (9:30 ET — táž hranice jako základ pohybu ve vyhodnocení, #1585); ověřuje ji úložiště **skutečným časem zápisu**, ne minutou cyklu — cyklus 9:29, který doběhne po openu, dostane `VerdictFrozenError` (log info) a nezapíše. Víkend a svátek NYSE: žádný řádek; `BriefingVerdictCollector` o víkendu neběží a víkendový verdikt nevyhodnocuje. Neúplný výpočet (selhalo čtení API, runtime nemá levels nebo tendenci — první minuta po restartu; tendenci jen při `GEXLENS_TENDENCY_ENABLED`) uloží seznam chybějících vstupů do `missing` (např. `["gamma útes: HTTP 500"]`) a přepíše jen neúplný řádek — úplný nikdy (#1595 e1). Engine, API a frontend se nasazují i vracejí společně (ADR-0035 dodatek bod 9). Bez platné ceny (≤ 0, NaN) se minuta nezapíše. Pod mechanikou v6 (#1600) pipeline místo staré ceny předá None: hlas overnight je „bez dat", řádek neúplný (`missing`: „spot: poslední cena … stará N s") a automatický scénář nevznikne (oddíl „Degradovaný start a co ještě přebírá tasty", ADR-0057 doplněk 23. 10.). Čtení vstupů z API má strop 20 s (`VERDICT_READ_TIMEOUT_S`, běžně ~2,4 s na symbol) — po něm se minuta nezapíše (log warning), aby visící API nezdrželo minutový cyklus sběru dat. Vstupy (PDC = RTH close, spot enginu, aktivní expirace, svíčky) a řez verze 3 podle data viz ADR-0035 dodatek. `POST /briefing/verdicts` (zápis z Briefingu do seance 16. 10. 2026) je odstraněn. Scénáře dne viz #1173 v kap. 4.

Testy: `engine/tests/test_instruments.py` (archiv pá–po, vol koncentrace), `test_runtime.py` a `test_bsfallback.py` (BS hlídka: víkend, pauza, nedělní otevření, max 2 remediace přes denní pauzy), `test_favalidation.py` (víkendový řádek neblokuje pondělí), `test_facalibration.py`, `test_marketclock_rth.py`, `test_gexforward.py`, `test_scenario_auto.py`, `test_briefing_verdicts.py` (upsert verdiktu a stráž: od openu, víkend, svátek, řádek s výsledkem), `test_verdict_writer.py` (kadence, stráž časem zápisu, neúplný výpočet, cena, víkend a svátek, zimní čas), `test_settle.py` (`verdict_writable`), `test_dayverdict_golden.py` (parita s dosavadní TS implementací), `api/tests/test_api.py` (`/oidelta` pondělí proti pátku), `test_briefing_api.py`.

6. Datové formáty a persistence

Parquet (`GEXLENS_DATA_DIR`; retence — ADR-0029)

Od ADR-0029 (v1.2) se **`snapshots/` a `derived/` nemažou nikdy** (`GEXLENS_KEEP_LEARNING_DATA_FOREVER=true`, default) — jsou to nenahraditelná učicí data samoučící smyčky (#794); IBKR historii řetězce zpětně nedá. Noční purge (`RETENTION_DAYS`, ADR-0022: 90 dní) tak reálně maže jen `ticks/`. Objem keep-forever režimu ≈ 6 GB/rok (ES+NQ); `GEXLENS_DISK_LIMIT_GB` (default 20) je alert na revizi, volné místo hlídá #773.

Partice	Schéma
`snapshots/{sym}/{expiry}/{YYYY-MM-DD}.parquet`	ts_min, strike, right, bid, ask, last, volume, iv, delta, gamma, theta, vega, oi, stale_age
`ticks/{sym}/{YYYY-MM-DD}.parquet`	ts, conId, price, size, side
`derived/{sym}/{expiry}/levels/{date}.parquet`	ts_min, flip, call_wall, put_wall, centroid, total_gex
`derived/{sym}/flow/{date}.parquet`	ts_min, flow_delta, cum_delta, futures_cvd*, **source** (v1.4, ADR-0032/ADR-0057: `midpoint` do mechaniky v5, `aggressor` od v6 (#1403); NULL = partice před #615 fází 3 = midpoint), od #1071 (v1.6) **pokrytí trade větví za minutu**: `printed_volume`, `unknown_volume`,

Partice	Schéma
	`structured_volume`, `fallback_volume`, `dropped_no_delta` (NULL = trade větev neběžela nebo partice před #1071; měří se i v režimu midpoint — říká, co by dxFeed pokryl; `/status.cumdelta_coverage` je totéž od startu enginu). Čte `scripts/compare_cumdelta_sources.py` (sloupce „pokrytí tisky" a „fallback RTH"; jen řádky midpoint, i net hybridu z `netflow`, #1590) a replay backtestu a walk-forwardu (zdroj podle mechaniky, kap. 12)
`derived/{sym}/{expiry}/printvol/{date}.parquet`	**Podíl objemu mimo tisk** (#1007, v1.4): per kontrakt a bar `volume_delta`, `printed` (Σ tisků TimeAndSale od minulého baru), `structured` (zbytek bez tisku — nohy spreadů, bloky). **NULL** = trade větev pro instrument neběžela (tasty odpojené, symbol bez univerza); nula by lhala „100 % struktura". Jen řádky s přírůstkem. Od v6 je přírůstek objem nad maximem seance (#1602). Bar bez přírůstku tisky nespotřebuje; jde o kotaci z cache mezi sweepy křídel, zamrzlou kotaci, pokles kumulativu nebo návrat k maximu. `printed` je Σ tisků od posledního baru, který počítadla vyprázdnil. To je bar s přírůstkem, nebo bar, který stav objemu zakládá (první bar seance, po restartu, po nekrmeném cyklu, po předání v rollu). Tisky se počítají až do okamžiku baru po sweepu, tedy i tisky po řezu vzorku. Je-li mezerou trade větve minuta baru nebo minuta baru bez přírůstku mezi tím, je řádek NULL; výpadek mezi vzorkem a barem téže minuty ale řádek nezneplatní (`printed` nižší, `structured` vyšší) — metrika kvality dat, ne vstup obchodního rozhodnutí (ADR-0057 doplněk 18. 10.); push `printvol.{sym}.{exp}`, sekce `printvol` v `/replay`
`derived/{sym}/bars/{date}.parquet`	ts_min, open, high, low, close, volume, **source** — **z purge vyňaté, drží se navždy** (`GEXLENS_KEEP_BARS_FOREVER=true`); ES i NQ mají ~2 roky historie od 2024-07-28 (backfill `scripts/backfill_bars.py`). `source` (#617): `NULL` = partice před #617, `ibkr` = živá cesta, `ibkr_hist` = **doplněno z IBKR historical** (#1055, v1.6 — platná cena, ale engine v tu minutu neměřil; do 8. 9. 2026 se zapisovalo jako `ibkr`), `tasty_candle` = **rekonstruováno** z dxFeed; pyarrow čte starší šestisloupcové soubory novým schématem s `NULL` (ověřeno). Frontend z něj staví historii přes hranici dne (#788, `GET /bars?date=` den po dni, 404 = víkend, 5 děr v řadě = konec archivu). **Partice = UTC den baru** (#1002, v1.3): do 3. 9. 2026 engine zapisoval půlnoční bar 23:59 i rekonstruovaný blok 22:00–23:59 dne D−1 do partice sousedního dne → minuta dvakrát, objem dvojnásobný v oknech přes půlnoc; staré partice opraví `scripts/fix_bar_partitions.py --apply` (spouštět při zastaveném enginu; bez `--apply` jen vypíše plán), API a news-engine navíc deduplikují obranně
`derived/{sym}/{expiry}/oiwalls/{date}.parquet`	**OI zdi** (#851): oi_call_wall / oi_put_wall + `share` (podíl na OI strany; frontend pod 0,2 nekreslí)
`derived/{sym}/features/{date}.parquet`	**Minutový feature log** (#796): vstupní vektor setup detektoru + ATR + band metriky — trénovací matice smyčky #794; minuta, jejíž dávka cyklu nemá bar, řádek nemá (#1346; dřív O = H = L = C ze spotu)
`trades/{sym}/{YYYY-MM-DD}.parquet`	**Surové opční TimeAndSale printy z dxFeed** (#795): ts, streamer_symbol, price, size, aggressorSide, spread_leg, eth. Mimo retenci; podklad budoucí klasifikace agresora (#615). Flag `GEXLENS_TASTY_TRADES_RECORD` (default true)

Partice	Schéma
`derived/{sym}/{expiry}/netflow/{date}.parquet`	Kumulativní klasifikovaný net objem per strana; sloupec `source` nese zdroj znaménka jako `flow/` (`midpoint` v5, `aggressor` v6, NULL = partice před #1460 = midpoint), navázání po restartu bere jen řádky zdroje běžícího enginu (#1460) — podklad FA odhadu OI a ranní kalibrace α. Řádek má každá sledovaná strana, i s netem 0 (#1582): explicitní 0 = naměřená nula, chybějící řádek = strana se nesleduje. Od mechaniky v6 minuta s mezerou trade větve CumΔ (ADR-0057 bod 3) netflow řádky nemá vůbec — net v ní ještě nese tisky částečné minuty; čtenáři berou poslední měřený řádek strany. Píše **aktivní i sekundární řetěz** (#1182): aktivní ES/NQ je vždy 0DTE bez ΔOI do D+1, kalibrace proto bere netflow sekundáru (expirace po dni netflow, od mechaniky v6 jen řetěz, který byl sekundárem celou seanci D, #1580); `fa_alpha_history.expiry` říká, ze kterého řetězu bod vznikl.
`derived/{sym}/{expiry}/netgap/{date}.parquet`	**Minuty mezery trade větve** řetězu (#1460, ADR-0057 bod 6; jen mechanika v6): `ts_min` cyklu, ve kterém aktivní nebo sekundární řetěz za otevřeného trhu neměřil tisky — netflow se tehdy nepíše. Den bez mezery partici nemá. Kalibrace α podle ní vynechá den s mezerou i u sekundáru, jehož řádek `flow` se zahazuje a jehož netflow chodí jen každý N-tý cyklus.
`derived/{sym}/{expiry}/oiest/{date}.parquet`	FA odhad OI (netflow×α, #232)
`derived/{sym}/{expiry}/gexprofile(fa)/…` + `gexfield(fa)/…`	Dyn profily/pole; `…fa` varianty nad FA odhadem
`derived/{sym}/{expiry}/charmprofile/…`, `vannaprofile/…` (+ `…field`)	Dyn Charm/Vanna plochy (#204)
`derived/{sym}/{expiry}/greekssource/{date}.parquet`	Zdroj greeks per minutu (model/computed, #547)
`derived/{sym}/{expiry}/oimissing/{date}.parquet`	Striky bez OI (šrafura, #465)
`derived/{sym}/catchup/{date}.parquet`	Příznak dohánění po startu (#518)
`derived/{sym}/gexforward/{date}.parquet`	**Forward GEX** (#519): bloky per budoucí obchodní den (day, grid, values, dropped_expiries, dropped_share, iv_fallback_share); jen poslední stav, přepočet po OI archivu
`derived/sentiment/{SYM}/{date}.parquet`	1min řada SentIndexu per symbol (ADR-0026; ploché soubory bez symbolu = ES legacy)

Zápis je **atomický** (temp + rename) — po pádu procesu nikdy nezůstane částečný soubor; osiřelé `.tmp` se uklízí při dalším zápisu. Writer po restartu navazuje na rozepsaný den.

PostgreSQL

Tabulka	Účel
`oi_eod(symbol, expiry, trading_class, strike, right, date, oi, iv, delta, gamma, theta, vega, close_prem, und_price,`	**Věčný** denní snímek řetězce — od #519 nese vedle OI i IV/greeks/závěrečnou prémii/ref. spot z ranního průchodu (NULL = model nedodal). Od #736 je v klíči `trading_class` (série se neslévají — E4C/EW4/EW…; `''` = souhrn/legacy, historie čitelná dál; čtení Σ přes série dává konzumentům stejná čísla jako dřív, `values_for(trading_class=…)` pro kalendář #513). `captured_ts` = čas IBKR čtení (finalita #463), `tasty_captured_ts` = čas poslední změny hodnoty z tasty (#828;

Tabulka	Účel
`captured_ts, tasty_captured_ts)`	nezměněná hodnota ho neposouvá, čerstvost nedokazuje). Původ hodnoty určuje jen `captured_ts`: vyplněný = IBKR (i když řádek předtím zapsal tasty — IBKR zápis `tasty_captured_ts` nemaže, řádek pak nese oba časy), `captured_ts` NULL a `tasty_captured_ts` vyplněný = tasty řádek; oba NULL = IBKR řádek z doby před #463 (srpen 2026) bez času čtení; tasty nikdy nepřepíše řádek s `captured_ts` (#1605). Žádná retence, žádné delete API
`gamma_cliff`	Denní odpad gammy po expiraci + metriky následující seance (#576, fáze měření): `next_range_atr`, `next_setups` a od #1115 `next_outside_share` = podíl minut následující Globex seance (do settle) s `band_depth ≤ 0` (třídy outside/no_zone, totéž pravidlo jako stínová brána #1060) z feature logu `derived/{sym}/features/`; NULL = feature log seance chybí, doplňuje se při dalším běhu
`map_state`	Stav „tenká mapa" per (seance, symbol) (#1245, fáze měření): medián a p10 |total_gex|, medián gammy u ceny a dominance zdí v US RTH, `thin_share` = podíl minut ve stavu thin, `sample_minutes`, `version`. Historie pro **relativní prahy** (25. percentil 20 seancí téhož symbolu); backfill z `levels` / `walldom` / `gexprofile` / `bars` partic při prvním běhu (gamma u ceny jen kde je profil). Vypnutí `GEXLENS_MAP_STATE_ENABLED=false`
`em_respect`	Respektování pásma EM per (seance, symbol) (#872, ADR-0028 vzor): zdroj EM (straddle / close prémie), close a dotyky vůči pásmu, `range_vs_em`. `negative_gamma_share` = podíl minut se spotem pod flipem **aktivního řetězu** (expirace ≥ seance s nejvíc minutami levels v seanci do settle, #1452; dřív první expirace s jakýmkoli souborem — NQ 17. 9. 2026 tak měl NULL místo podílu z 1 286 minut), `negative_gamma_minutes` = z kolika minut (spot i flip) podíl vznikl (0 = žádná společná minuta: levels chybí, aktivní řetěz nemá flip nebo bary — podíl NULL; NULL = nevyhodnoceno: nečitelná partice, nebo řádek před #1452 mimo okno backfillu 45 dní, ~36 seancí na symbol z 19. 7.–7. 9.). Sloupec přidává `EmRespectRepository.ensure_schema` ALTERem při startu enginu; startovní backfill přepočte řádky bez počtu minut jednou (první start po #1452: 66 řádků, ~2 min v minutovém cyklu — nasazovat v pauze Globexu nebo při zavřeném trhu, engine dřív než API).
`briefing_verdicts`	Verdikt dne per (seance, symbol) (#1090, #1091, ADR-0035 §3 a dodatek 2026-10-17): `verdict` (long / short / none / wait_news), `score`, `votes` JSON `[{name, vote, reason}]`, `rules_version`, `missing` JSON (chybějící vstupy výpočtu: `[]` úplný, NULL řádek z Briefingu před #1595; aditivní sloupec přes `ensure_schema` enginu i API, #1595 e1), `created_at` (první zápis), `updated_at` (poslední, vždy před US openem seance); unikátní `uq_briefing_verdicts_session_symbol`. `rules_version` 4 = mechanika v6 (#1403): pravidla jako v3, vstup tendence verze 5 z CumΔ jen z tisků (ADR-0057 bod 8); statistiky se čtou podle verze. Zapisuje **jen engine** před US openem (kap. 5 „Verdikty dne", #1595); po settle + 15 min doplní `outcome_*` (`open`, `us_open`, `close`, `move_pts`, `move_em`, `hit`, `computed_at` — aditivní sloupce přes `ensure_schema`). Řádky do seance 16. 10. 2026 psal Briefing s jinými vstupy pod touž `rules_version` 3; pět řádků zapsaných po openu před #1585 (id 1, 2, 4, 7, 11) zůstává
`feed_comparison`	Shadow porovnání IBKR × tastytrade per (minuta, kontrakt, pole) — jen po dobu sběru M7 fáze 1 (#613). **Historie do 22. 8. smazána 1. 9.** (#965)
`feed_comparison_daily`	Zhuštěné souhrny shadow sběru (#965): denní řádky + celkové řádky (`session_date IS NULL`) per symbol × pole s `n`, mediánem a p95
`sentiment_daily`, `sentiment_waves`, `sentiment_episodes`, `news_*`, `signals`,	SentimentLens (per symbol od ADR-0026). `sentiment_episodes` (#565, ADR-0037) = korekční epizody pokus/negace, plně derivované — WavesJob je přepočítává full-replace z `close_z`; sloupec `params_version` nese verzi prahu D / horizontu H (v1 = placeholder 1 σ / 10 dní), změna parametrů = nová verze v `gexlens_engine.compute.sentwaves` a přepočet při příštím běhu jobu.

Tabulka	Účel
`signal_outcomes`, `track_record`	Přeměření: `uv run python scripts/measure_sentiment_episodes.py --era both --out …` (prod DB jen čtení; URL z `GEXLENS_PG_PASSWORD`, port 55432). `news_reactions` je od #998 (ADR-0031) **jeden řádek per event × symbol** se sloupci per okno (`ret_<w>`, `range_<w>`, `vol_z_<w>` jen minutová okna, `cont_<w>`) a per fázi (`deferred_*`, `regime_*`, `computed_at_*`) — 268 → ~56 MB, bez retence (učicí data). Starý tvar (řádek per okno) news-engine při startu odmítne s odkazem na `scripts/migrate_news_reactions_wide.py` (jednorázově, po záloze PG; `--dry-run` napřed); stará tabulka zůstane jako `news_reactions_legacy_<datum>` a maže se ručně až po ověření provozu
`release_moves`, `release_previews`, `release_hypotheses`	Ohlášené releasy (#1296, ADR-0044), news-engine. `release_moves` PK (`cluster_ts`, `symbol`) = naměřená fakta za shluk USD releasů (FF High/Medium ve stejné minutě) × ES/NQ: `family` (CPI, NFP, FOMC, PPI, PCE, RETAIL, ISM_SERVICES), `headline` + `headline_event_id`, `surprise_sign` (NULL = actual/forecast chybí), `vol_ref_bp` (medián denního rozsahu 20 seancí před seancí releasu / close před ním), `exc_15m_bp`, `ret_15m_bp`, `ret_60m_bp`, `tod_med_15m_bp` (jen releasy od registrace hypotéz, pro M1), `measured_at`; ~~320 řádků za 26 měsíců, +~~10 za měsíc, píše jen `ReleaseMovesJob` (upsert). `release_previews` PK (`cluster_ts`, `symbol`, `stage` T60/T15) = každý odhad v upozornění před releasem (`n`, `expected_p50_bp`, `expected_p75_bp`, `vol_now_bp`, `sent`; `sent = false` = etapa přeskočená po pozdním startu) a zároveň dedup etap přes restart. `release_hypotheses` PK (`hypothesis`, `symbol`) = stav registru (H1, H3, M1): `n`, `hits`, `wilson_lb/ub`, `status` testing/verified/rejected, `decided_at_n`, `outcomes` JSON; přepisuje se po každé změně živého řádku, ale **vyhodnocený úsek** (releasy do posledního kontrolního bodu, u rozhodnuté hypotézy do rozhodnutí, a stav) se přebírá z předchozího řádku — rozhodnutí se zpětně nemění. **Tabulku nemazat ručně** (zmrazení by se ztratilo). Definice a kritéria hypotéz v DB nejsou (kód `gexlens_news/release_hypotheses.py` + ADR-0044). Založí je `ensure_sentiment_schema` (jen aditivně) — **před nasazením záloha PG**
`setups`	Setupy vč. `context` JSON (od #575 nese band_sharpness/band_sharpness_pct/band_depth; od #952 i `band_metrics_version` = 2 — hloubka pásma nad Major se mapuje na (1, 2] místo saturace na +1, v1 a v2 se nesmí sdružovat; po rebuildu spustit `scripts/backfill_band_metrics.py`, idempotentní podle verze) a `mechanics_version` (v5 od #859: setupy z doby zamrzlého Max Painu (#826) se nehodnotí — nemažou se, jen se verzí vyřazují ze statistik)
`paper_accounts`, `paper_events`, `paper_orders`, `paper_order_changes`	Paper účet (#1187 fáze 1, ADR-0040): účet 1 (start 50 000 $ v jednotkách plného kontraktu, `halted` = kill switch), události (vklad/výběr/poznámka), ordery (side, qty, typ, entry/stop/cíl, status `working/open/closed/cancelled`, fill/exit ceny a čas, `exit_reason` stop/target/manual/settle/kill, `pnl_usd`, `fees_usd`, `r_multiple`, `risk_usd`, `context` JSON, `journal_entry_id`). Fily dělá engine per symbol po cyklu pipeline (`gexlens_engine.paper.PaperBroker`) proti 1min barům; uzavřený obchod zapíše do `journal_entries` / `journal_trades` (typ `obchod`, tag `paper`).
`setup_params`	Verzované prahy šablon setupů (#794 fáze 2, ADR-0033): append-only, poslední řádek platí; `created_by` (`engine` seed / `ui` / `script`), povinná `note`, `mechanics_version`, `params` JSON. Engine při prvním startu založí seed z `.env` + defaultů (od té chvíle **store přebíjí** `.env` klíče `GEXLENS_SETUP_*`), novou verzi přečte po NOTIFY nebo v k-tém cyklu. Setupy nesou `params_version` (NULL = před store).
`adhoc_view`	Most UI → engine pro ad-hoc pohled (#521 C), viz kap. 12
`watchlist`, `alerts`, `annotations`, `settings`	CRUD přes API

7. API reference

Interaktivní dokumentace: `http://127.0.0.1:8010/docs` (OpenAPI; dev stack `:8011`).

REST

Endpoint	Popis
`GET /health`, `GET /status`	Liveness; agregovaný stav pipeline (`lines_utilization` je od #630 měřená špička; `symbols` = symboly s pipeline, např. „ES,NQ“ — Briefing podle něj ukáže „Engine pro tento symbol verdikt nepočítá“, #1595). Pole `chain_source` / `spot_source` nesou aktivní zdroj dat (#614), `feed_crosscheck*` verdikt křížové kontroly (#517 A), `connection_offline_for_s` délku výpadku IBKR (#770), `tasty_rate_limited` / `tasty_heals` / `tasty_budget` stav DXLink subskripce (#863, #936, #982), `tasty_kpi` KPI stability streamu per seance (#1214: výpadky, rate limit, RTH minuty bez eventu, tasty mrtvá strana, podíl greeks; uzavřené seance v `data/reports/tasty-kpi.jsonl` a v logu `tasty KPI …`), `cumdelta_source` (`midpoint`, od mechaniky v6 `aggressor`, ADR-0057) + `cumdelta_coverage[symbol]` (v1.4, ADR-0032: `printed_volume`, `unknown_volume`, `structured_volume`, `fallback_volume`, `dropped_no_delta`, `printed_share`; od #1460 `printed_share_total` = podíl tisků se stranou z celého přírůstku objemu včetně struktury; od v6 oba podíly jen z tisků spárovaných s měřeným přírůstkem objemu, `printed_volume` nese všechny tisky včetně minut bez baru kontraktu, a bez spárovaného objemu (start do fallbacku řetězu) je podíl `null` = neměří se, ne 0 (pod midpointem 0 jako dřív; ADR-0057 doplněk 18. 10.); a `trade_branch` = stav trade větve aktivního řetězu podle ADR-0057 bod 3: `running` / `gap` (otevřený trh, stream odpojený, univerzum mlčí nad práh hlídače 3 min RTH / 10 min Globex nebo trasy tisků patří předchozímu řetězu — od v6 minuta s NULL CumΔ; od v6 rozhoduje tracker, pro který se trasy stavěly, takže po rollu s předáním běží aktivní D+1 hned, ADR-0057 doplněk 20. 10.) / `closed` / `off` = bez tasty větve), `map_state[symbol]` stav „tenká mapa" (v1.10, #1245: `thin`, podmínky `thin_gamma` / `weak_walls` / `fused` jako true/false/null = bez dat, `reasons`, `gex_abs`, `gamma_abs`, `spread_pct`). **Chybějící klíč znamená „neměří se"**, ne „je to v pořádku" — od #756 chodí status i bez jediné pipeline a `connection` nese skutečný stav spojení
`GET /push/status`	Push na Telegram (#1175, #1284), bez tokenu: `configured`, `quiet_hours`, `daily_cap`, `sent_today`, `last_sent_at`, `last_error`, `enabled` (hlavní vypínač), `master` { `setting`, `label`, `help` } a `groups[]` { `key`, `label`, `topics[]` { `key`, `setting`, `label`, `help` = řádky tooltipu, `enabled` = efektivní hodnota po dědění}}. Token bota ani chat id nevrací. Čte ho Settings → Notifikace i `scripts/lib/OpsAlert.ps1`

Endpoint	Popis
`POST /engine/reconnect` `{"target": "ibkr"|"tasty"|"both"}`	Ruční přepojení (#950): zapíše razítko do `settings`, engine ho vyřídí při nejbližším pollu (kap. 5). Bez tokenu jako `/settings` — nová třída expozice nevzniká, kdo dosáhne na port, může spojení rozbít už přes `PUT /settings/ibkr_port`
`GET /gexforward/{symbol}`	Forward GEX bloky per budoucí den (#519)
`GET /search?q=`, `POST /adhoc/{symbol}`	Našeptávač: CME katalog + akcie/ETF/indexy (`kind` futures/equity) + volný ticker mimo katalog (#206); `/adhoc` přijme kořen i akcii (gramatika ADR-0041) a pošle `pg_notify(gexlens_adhoc)`, engine pohled založí do sekundy (dřív poll à 30 s)
`GET /bars/{symbol}?date=`	Lehké 1min OHLCV bary seance (#674/#678) — bez /replay balíku
`GET /news?from&to&category&importance&kind&limit&symbol`	Feed zpráv pro obrazovku **News**: nejnovější první, `limit` 1–1000 (výchozí 200, UI 100) — strop uřízne nejstarší. Karta nese `reactions_bp` a `reaction_contaminated` (symbol = `symbol`) a `significance` (stupeň významnosti, ADR-0045) a `topic_value` (index tématu k okamžiku zprávy; výpočet je O(řádky × eventy kategorie) za 2 dny zpět, 100 řádků ~1 s, 6h okno ~12 s — pro velká okna nepoužívat). Graf ho od #1290 nečte Řádek nese i `content_tier` (tier obsahu 1–3 / null, ADR-0059; totéž `/news/upcoming` a `/news/{id}`).
`GET /news/markers?from&to`, `GET /news/markers?ids=`	**Zprávy pro markery grafu** (#1290). Rozsah `[from, to)` **bez stropu počtu**; délka nejvýš 26 h (okno seance má 24 h, v den přechodu DST 25 h), delší, chybějící `from` / `to` nebo `from ≥ to` = 422 — frontend načítá po seancích [17:00 CT D−1, 17:00 CT D). `ids` = 1–100 id čárkou (proklik z upozornění), neexistující id se vynechá; `ids` spolu s rozsahem = 422. Kompaktní sloupce `id, ts_event, kind, category, importance, title, summary, sentiment_dir, sentiment_score, forecast, previous, actual, surprise_z` + `surprise_direction` u scheduled + `significance` (stupeň významnosti 0–3 / null z `compute/news_significance.py`, #1305, ADR-0045 — i u `ids`, proklik vrací i nevýznamné zprávy; frontend podle něj filtruje „Významné" a řadí dialog, pravidla nekopíruje); bez `body`, `raw`, reakcí a `topic_value`; řazení vzestupně. Odpověď jde přímo přes `JSONResponse` (řádky jsou po převodu čisté JSON typy). Frontend (`useChartNews`): den jedním dotazem, cache per datum seance (ES i NQ sdílí); živý den WS `news` (dávky po 2 s) + každou minutu `from = poslední úspěšné dotažení − 30 min` (zacelí i výpadek REST delší než okno); celý den znovu po reconnectu i po (znovu)zahájení odběru (vypnutí/zapnutí News, návrat z Daily či z jiného dne); historické dny (#788) líně po jednom. Chyby per den, maže je úspěch téhož dne; dotažení nebo push beze změny nemění stav (žádný re-render), „teď" pro nadcházející se posune jen s vydaným plánovaným eventem. Markery uzavřených seancí se cachují per den (`closedDayMarkers`), při živé změně se přestaví jen živý den. Měřeno 25. 9. 2026 na produkčních datech (read-only, `scripts/measure_news_markers.py`): špičková seance 16. 9. 4 971

Endpoint	Popis
	zpráv → PG 22 ms, odpověď ~~0,3 s, JSON 1,9 MB / gzip 0,43 MB; běžná seance (~~3 tis.) ~0,2 s a 0,3 MB; `ids` ~12 ms
`GET /news/breaking?at&hours&limit`	**Karta Breaking news** (E-6.28b, ADR-0059 bod 4 a 8): významné zprávy za `hours` (1–72, výchozí 12), nejnovější první, nejvýš `limit` (1–200, výchozí 50). Výběr `gexlens_news.breaking.is_breaking` při čtení: efektivní tier 1–3 z prvního doručení a kopií `news_event_sources` viditelných v čase, článek (tier 3) jen s příjmem ≤ 60 min po publikaci. Položka: `id`, `ts_event`, `title`, `confirmed` (tier 1–2), `is_key`, `group`, `theme`, `sources` (první doručení + kopie, `delay_s` = příjem − `ts_event`) a `impact.{ES,NQ}` = `state` (`running` do uzávěru posledního baru okna, zpráva v 14:30:20 do 14:36:00 / `fixed` / `closed` zpráva při zavřeném trhu nebo těsně před ním bez barů po zprávě či s odloženou reakcí / `no_data` mezera v barech při otevřeném trhu), `elapsed_min`, `ret_bp` (změna od posledního close před zprávou, okno 5 min, `compute_reactions`), `range_bp`, `excursion_bp` + `excursion_direction` (ADR-0043 od minuty zprávy, okno roste do 5; bez hodiny barů pro σ `null`), `contaminated` (K1, jen eventy známé v čase). **Počítá API** z `derived/{ES,NQ}/bars` (`api/src/gexlens_api/breaking_card.py`): partice v paměti podle mtime (nejvýš 24), bary jen kolem zpráv (hodina před, okno 5 min; pětidenní základ přes zavření jen u zprávy těsně po otevření), zafixované dopady v paměti podle obsahu barů (přepsaný bar = přepočet). Měřeno v kontejneru 10. 10. nad produkčními daty: 50 položek ~90–110 ms, první dotaz po startu ~1,8 s (načtení partic). Bez `at` živě včetně rozpracované minuty; `at` (ISO s pásmem, jinak 422) = replay z dat dostupných v `at` (uzavřené minuty, příjem i kopie ≤ `at`); kategorie a importance jsou ale dnešní (klasifikace nemá čas v `news_events`). Top-level `market_closed` = trh zavřený v `as_of`
`GET /oidelta/{symbol}/{expiry}`	ΔOI posledních dvou archivovaných **obchodních** dnů + top movers (#674; víkend se přeskakuje a změna se počítá jen na stranách obou archivů — #1309)
`GET /journal`, `POST/PATCH/DELETE /journal/*`	Deník tradera (#673, fáze A)
`GET /setups/params`, `POST /setups/params` `{params, note, created_by?}`	Parameter store setupů (ADR-0033): platná verze + historie + defaulty; POST založí novou verzi (jen změněné klíče, zbytek defaulty; neznámý klíč/typ = 422, `note` 3–500 znaků po ořezu mezer, jinak 422) a probudí engine NOTIFY. Autonomie stupeň 1: zapisuje člověk, ne smyčka. **Stádia buněk (#1323) tudy neměnit:** vynechané `shadow_cells` / `trial_cells` se převezmou z platné verze, jiná hodnota = 422 s odkazem na `POST /setups/stage` (typicky starý snímek parametrů — načíst znovu). `trial_budget_setups` 1–20, `trial_budget_r` 0,5–6, jinak 422. Settings → Risk management posílá parametry bez klíčů stádií a bez náhradního důvodu.
`POST /setups/stage` `{cell, stage, budget_setups?, budget_r?, note, created_by?}`	Knihovna setupů (#1323, ADR-0038 dodatek 2026-10-01): změna stádia **jedné** buňky `TICKER:šablona` (ticker instance v kanonickém

Endpoint	Popis
	tvaru, ADR-0041: `NQ`, pinovaný `NQZ6` je jiná buňka) na `auto` / `shadow` / `trial` → nová verze `setup_params` + NOTIFY. Server čte platnou verzi a mění jen tuto buňku; **začátek zkoušky nastaví sám** (teď) a orazí ji svou `SETUP_MECHANICS_VERSION` — opakované `trial` = obnovení s rozpočtem od nuly. Engine zkoušku uplatní až od `started_at` (cyklus s časem zaokrouhleným na minutu ji tak převezme v příští minutě) a jen na téže mechanice; zvednutí mechaniky zkoušku ukončí (Auto, bez zápisu). `budget_setups` (celé číslo 1–20) a `budget_r` (0,5–6) jen u `trial`, chybí = `trial_budget_*` z parametrů. 422: špatný formát buňky, neznámá šablona, rozpočet mimo meze nebo u jiného stádia, krátký `note`, neplatné `stage`. 409: stádium se nemění (Auto → Auto, Stín → Stín) nebo store ještě nemá verzi. 201 = tělo jako POST `/setups/params`. Volá UI Setupy → Knihovna (dialog s povinným důvodem).
`GET /setups/{symbol}?date&status`	Stránka tabulky setupů: posledních 200 podle `created_ts DESC, id DESC` (id rozhodne shodný čas vzniku — 31 dvojic v produkci) + `total_count` = všechny řádky se stejnými filtry (`count_for`; null při nedostupné DB). Řádek nese `after_settle` (#1324, `born_after_settle`) a `excluded` (#1346, důvod značky `context.excluded`, např. `vstup_mimo_bary`, jinak null): UI takový řádek ztlumí se štítkem („po settle, mimo souhrn" / „vstup mimo bary, mimo souhrn") a filtr „Jen obchodovatelné" ho skryje. **Není podklad statistik** — agregace nad stránkou je klouzavé okno (#1319). `date` = UTC den vzniku (Deník jím hledá setup u zapisované minuty).
`GET /setups/summary?symbols=ES[,NQ]&all_versions&sim_account_usd&sim_risk_pct`	Souhrn setupů z **celé historie** (#1319): čistá funkce `compute/setup_summary.py` nad `SetupsRepository.summary_facts` (bez stropu, jen potřebné sloupce + `context`). Skupiny `all` / `tradeable` / `shadow` / `unruled` (podle `context.tradeable`; bez `tradeable` + `contracts` = před pravidly): `count`, `active`, `closed`, `wins`, `win_rate`, `sum_r`, `avg_r`, `gross_usd` / `fees_usd` / `net_usd` na 1 kontrakt (R × stop × `paper.POINT_VALUES`; poplatek = uzavřené × `fee_per_contract_usd`), `ev_r`, `ev_usd`. Dále `shadow_reasons` (podle `trade_block`), `account` (jen tradeable: R × `max_loss_usd` − `fee_usd`, `net_pct`, max DD chronologicky podle uzavření; null bez risk kontextu), `today` (seance `trading_session_date`; uzavřený dle `closed_ts`, aktivní dle vzniku), `band_gates` (#1060), `regimes` (šablona × `gex_regime`, cíl vs. stop), `performance` (denní ΣR, Sharpe celkem/30, max DD v R, volitelná USD simulace mikro kontrakty #679 při obou `sim_*` > 0), `legacy_count`, `total_count`, `unpriced_symbols`, `after_settle_count` (#1324: setupy zvolených verzí vzniklé po settle vlastní expirace, `compute/setups.born_after_settle`; ze všech čísel souhrnu jsou vyřazené, v DB zůstávají), `excluded_count` (#1346: setupy zvolených verzí se značkou `context.excluded` a ne po settle — vznik nad zamrzlým spotem; vyřazené stejně). Obojí drží jeden predikát `compute/setups.counts_in_stats` (`SetupFact.in_stats`), který čtou souhrn, Knihovna (`cells`) i `SetupsRepository`. Výchozí jen

Endpoint	Popis
	`SETUP_MECHANICS_VERSION`, `all_versions=true` přidá starší; poplatek a účet z platné verze parametrů setupů. Prázdné `symbols` = 422, nedostupná DB = 503 (souhrn se nevydává za prázdný). Čtou ho Setupy (1 symbol) a Stats (portfolio watchlistu pro Výkon, aktivní symbol pro režimy). **Knihovna (#1323):** skupina `trial` (obchodovatelné s `gate_overridden` — podmnožina `tradeable`, ne další sčítanec), v `shadow_reasons` důvod `user`; `cells` = buňky ticker × šablona (`compute/setup_library.library_cells`, vždy jen aktuální mechanika bez ohledu na `all_versions`): tickery z `symbols`, které mají setup v okně brány, plus **vždy** tickery s nastaveným stádiem (`configured_tickers`, i mimo `symbols` — jejich řádky API dočte, souhrn sám zůstává jen pro `symbols`), šablony `TEMPLATE_REGISTRY` bez `disabled_templates`; pole `cell`, `ticker`, `template`, `template_number`, `stage` (nastavené), `effective_stage` (vyčerpaná zkouška nebo zkouška jiné mechaniky = `auto`), `trial` (`started_at`, `budget_setups`, `budget_r`, `mechanics_version`, `setups` vč. otevřených, `sum_r`, `spent` = rozpočet vyčerpán), verdikt brány **teď** `gate_verdict` / `gate_n` / `gate_lb` (`risk.affordable_rows` + `template_gate`, okno `gate_window_start` — táž cesta jako engine), `avg_r`, `avg_net_r` (R − round-trip mikra ADR-0030 / (stop × bod mikra), `setup_summary.net_r`), `net_usd` (Σ čistý P/L vzorku brány v reálných mikro dolarech: kontrakty × (R × stop × bod mikra − round-trip), kontrakty z `context.contracts`, starší řádky dopočte `position_size`; `setup_library.cell_net_usd`), `n_needed` (`risk.samples_needed`: $((1{,}645 + 0{,}84) \cdot \sigma / 0{,}2)^2$, nikdy pod `template_gate_min_samples`; null pod 2 vzorky nebo při σ = 0 pod minimem brány), `sessions`, `per_session`, `window_capacity` (⌊ `per_session` × `template_gate_days` ⌋ — víc vzorků okno brány při dnešním tempu nepojme), `sessions_to_decision` (0 = stačí, null = nejde odhadnout nebo `window_capacity` < `n_needed`); řazeno podle `gate_n / n_needed` sestupně, bez odhadu na konci. `brakes` = stav brzd účtu teď (`library_brakes`: `brake_state` nad `realized_since(week_start)` aktuální mechaniky, **napříč symboly** — nezávisí na `symbols`): `session`, `day_r`, `week_r`, limity `daily_brake_r` / `weekly_brake_r` / `max_template_stops_per_day`, `block` (`daily_brake` / `weekly_brake` / null), `template_stops` (stopy šablony za seanci, jen nenulové). Knihovna posílá všechny tickery watchlistu + aktivní symbol.
— risk parametry (#1185)	Součást téže verze parametrů: `account_equity_usd` (50000), `risk_pct` (1), `risk_max_pct` (2), `fee_per_contract_usd` (10), `daily_brake_r` (3), `weekly_brake_r` (6), `max_template_stops_per_day` (2), `template_gate_enabled` (true; od #1323 bez zaškrtávátka v UI, jen nouzově přes API), `template_gate_min_samples` (30), `template_gate_days` (60), stádia buněk (#1323) `shadow_cells` ([]), `trial_cells` ({buňka: {started_at, budget_setups, budget_r, mechanics_version}}) a výchozí rozpočet zkoušky `trial_budget_setups` (10), `trial_budget_r` (3). Engine u každého setupu zapíše do `context`: `risk_rules_version`, `contracts`,

Endpoint	Popis
	`risk_budget_usd`, `max_loss_usd`, `fee_usd`, `affordable`, `tradeable`, `trade_block` (`stop_over_budget` / `stop_over_cap` / `daily_brake` / `weekly_brake` / `template_stops` / `gate`), `template_gate` (+ `_n`, `_lb`), `realized_day_r`, `realized_week_r`. Brzdy čtou uzavřené setupy napříč symboly (`tradeable` = true). `_load_realized` čte širší okno (týden, nebo okno brány ≈ 84 dní) a `brake_state` si z něj vymezí den (`session_bounds`, do otevření Globexu 17:00 CT) i týden (od `week_start` = neděle 17:00 CT, #1322). `realized_since` vynechá setupy mimo statistiky (`counts_in_stats`: vznik po settle vlastní expirace #1324, značka `context.excluded` #1346) — do brzd, `n` brány ani čerpání zkoušky nevstupují. Brána šablon má klíč šablona × symbol (#1325, ADR-0038 dodatek): `affordable_results` bere jen setupy vlastního symbolu (ticker instance, ADR-0041 bod 3: pinovaný `ESZ6` má vlastní buňku a začíná na n = 0) se stopem v rozpočtu za `template_gate_days` seancí; starší řádky bez kontextu dopočítá z entry/stop a hodnoty bodu vlastního symbolu (`runtime.multiplier`). `risk_rules_version` 2 = brána per šablona × symbol, 1 = do #1325 (brána míchala řádky obou symbolů). Alert `risk_brake` (přepínač Telegramu „Brzda ztráty"), setup alert nese `tradeable` — stín do pushe nejde. UI: Settings → Risk management (POST téže cesty). **Stádia (#1323, `risk_rules_version` 3):** pořadí bloků sizing → brzdy → `user` (buňka ve Stínu) → `gate`; aktivní Zkouška přebije jen verdikt `block` / `insufficient` (`gate_overridden` = true), sizing ani brzdy ne. Čerpání `risk.trial_usage` (setupy buňky s `gate_overridden` od `started_at`: uzavřené z `realized_since`, otevřené z paměti instance — po restartu z kontextu), vyčerpaná zkouška = Auto bez zápisu; selže-li čtení výsledků, zkouška bránu nepřebije. `_load_realized` čte i od začátků zkoušek tickeru. Kontext navíc: `user_stage`, `gate_overridden`, `template_gate_avg_r`, `template_gate_avg_net_r`, `template_gate_n_needed`, u buňky se zkouškou `trial_started_at`, `trial_budget_setups`, `trial_budget_r`, `trial_setups`, `trial_sum_r`; `trade_block` může být `user`. Alert `setup_stage` / `trial_spent` (přepínač „Konec zkoušky setupu", skupina Setupy a burza) jen na přechodu do vyčerpání — jednou na zkoušku i po restartu; payload `symbol`, `template`, `message`, `trial_*`, `ts`. Setup alert `created` nese `template`, `user_stage`, `gate_overridden`, `trial_*` a text má šablonu jako `T7 trend_continuation`, stádium (AUTO / STÍN / ZKOUŠKA k/N) a řádek `Důkaz: ØR … (čistě …) · n a/b · brána …` — při dolní mezi ≤ 0 nebo bez ní (vždy u přebité brány) začíná `edge neprokázán:`. Zkouška platí od `started_at` a jen na mechanice, na které začala (`TrialCell.in_force`): `gate_overridden` ⇒ `created_ts` ≥ `started_at`, takže každý přebitý setup se do čerpání započte. UI: Setupy → Knihovna (zvonek `setup_stage` prokliká tamtéž).
`GET /gammacliff/{symbol}`	Dnešní odpad gammy + historie útesů (#576)
`GET /briefing/verdicts?symbol&days`, `GET /briefing/verdicts/stats?symbol&days`	Verdikt dne (#1090, #1091, ADR-0035 §3). Zapisuje **jen engine** (kap. 5 „Verdikty dne", #1595); `POST /briefing/verdicts` je odstraněn → **405** (starý bundle v otevřené kartě ho spolkne). GET

Endpoint	Popis
	vrací `{"verdicts": [...]}` — řádky posledních `days` dní (výchozí 30) včetně výsledku `outcome_*` (engine po settle + 15 min); Briefing z nich bere řádek seance `session_date` aktivního symbolu při každé obnově (à 60 s); `/stats` (výchozí 120 dní) hit-rate per verdikt a per složka hlasování, Wilsonova dolní mez, brána n ≥ 30 — **jen aktuální verze pravidel** (`rules_version` v odpovědi, `VERDICT_RULES_VERSION` : 3 do bumpu v6, pak 4; verze se nesčítají, ADR-0057 bod 8) a **jen řádky s `updated_at` před US openem své seance** (#1588). Řádky zapsané po openu před #1585 (id 1, 2, 4, 7, 11; 9. 9.–2. 10. 2026) zůstávají v DB i ve výpisu, do statistiky nejdou.
`GET /coach/summary?symbol&days`	Shrnutí kouče (`compute/coach_summary.py`): `lines` (4–6 vět z weekly + hours + setups) a `watch` (max 4 body pro Briefing).
`GET /coach/setups?days&symbol` , `GET /coach/hours?days&symbol`	Kouč v2 (#1201, `compute/coach_setups.py`): setupy aktuální mechaniky za N dní (`closed_between`) → příznaky (`counter_regime` , `outside_band` , `unaffordable` , `low_rr` , `timeout` , `bad_window`), profil denní doby (hodiny Europe/Prague + segmenty seance z ET hranic), doporučení avoid/focus (n ≥ 30, Wilson LB); `/coach/hours` totéž zvlášť pro obchody deníku a setupy. Setupy nově nesou v kontextu `session_segment` a `hour_local` .
`GET /coach/review?date&symbol` , `GET /coach/weekly?date&symbol`	Kouč v1 (#933, `compute/coach.py`): obchody deníku typu `obchod` (ruční i paper) s příznaky (`no_stop` , `after_brake` , `revenge` , `big_loss` , `no_setup` , `low_rr` , `early_exit` — z barů po výstupu, `overtrading`), cenou v R, skóre disciplíny, shrnutím; týdenní report po–pá s cenou příznaků, rozpadem po hodinách UTC a 1–3 pravidly. Čte `journal_between` + `bars_session` ; nic nezapisuje.
`GET /paper/account` , `GET /paper/orders?symbol&status&limit` , `POST /paper/orders` , `DELETE /paper/orders/{id}` , `POST /paper/kill` , `POST /paper/resume` , `POST /paper/events` , `GET /paper/events`	Paper účet (#1187, ADR-0040). `POST /paper/orders` `{symbol, side, qty, order_type market/limit/stop, entry_price (u market referenční cena), stop_price, target_price?, setup_key?, setup_id?, note?, context?}` → 201 `working` ; **409** `{block, reason, max_contracts?}` při `kill_switch` , `position_exists` (jedna pozice/order na symbol), `daily_brake` / `weekly_brake` (z realizovaných paper obchodů, prahy z parametrů setupů) a `stop_over_budget` / `stop_over_cap` (equity × risk_pct, strop risk_max_pct); 422 u neplatných úrovní / neznámého symbolu (hodnoty bodu v `compute.paper.POINT_VALUES`). `PATCH /paper/orders/{id}` `{stop_price?, target_price?, clear_target?}` posune úrovně (validace, rozpočet → 409) a zapíše `paper_order_changes` (kouč: `stop_widened_points` v kontextu deníku). `DELETE` nastaví `close_requested` — engine zruší čekající / zavře pozici na open dalšího baru. `POST /paper/kill` zastaví účet a zavře vše; `POST /paper/resume` odblokuje. Alerty `kind=paper` (placed/filled/closed/cancelled/kill; přepínač Telegramu „Paper účet"), WS `paper.{symbol}` .
`GET /calendar/expiry?date=`	Kalendář expirací (#1189): fáze kvartálního týdne (`normal` / `roll` / `opex_week` / `expiry_day` / `post_opex`), `quarterly_expiry` , `roll_date` , `soq_ts` (9:30 ET), `vix_expiry` , značky do grafu (`markers` :

Endpoint	Popis
	roll, quarterly_expiry, monthly_opex, vix_expiry). Čistá kalendářní matematika bez DB. Engine posílá alert `expiry_calendar` (roll date 9:30 ET, pondělí OPEX týdne 8:00 ET, 5 min po SOQ; přepínač Telegramu „Kalendář expirací"). Standardní kvartální třída propadá v SOQ (`compute.settle.expiry_settle_ts`); týdenní série nového kontraktu na kvartální datum (EW3/QN3) až v 16:00 ET (#1366).
`GET /fa/alpha`	Kalibrovaná α FA odhadu per symbol (#232): `{"min_days": 5, "alphas": [{symbol, alpha, days, source, updated_at}]}` . `source` = zdroj netu kalibrace (`midpoint` / `aggressor` ; stav před #1460 bez zdroje = `midpoint` , ADR-0057 bod 8), `min_days` = `FA_MIN_DAYS` — pod tisky (v6) jede FA až od tolika dnů kalibrace zdroje, který engine hlásí v `/status.cumdelta_source` (bod 6, #1588).
`GET /gexplane/{...}`	Dyn Charm/Vanna plochy (#204)
`GET /sentiment/*?symbol=`	Sentiment per symbol (ADR-0026): index/daily/state/waves
`GET /stats/releases/hypotheses`	Předem registrované hypotézy o reakci na releasy (#1296, ADR-0044): `registered_at` , `criteria` (kontrolní body, hranice) a `hypotheses[]` { `id` , `label` , `rule` , `in_preview` testing/verified/never, `symbols{ES,NQ}` { `historical` {hits, n}, `hits` , `n` , `wilson_lb` , `wilson_ub` , `status` , `decided_at_n` , `next_checkpoint` , `outcomes[]` , `computed_at` }}. Jen čte `release_hypotheses` a registr v kódu; bez řádku = `testing` s n = 0 (čerstvá DB není chyba). Čte ho Stats → Releasy
`GET /instruments` , `GET /instruments/{sym}/expiries`	Dostupné symboly/expirace (ze storage); expirace nesou `detail` (trading classes) a `multiplier` opčního kontraktu (#1400): futures z `ticker.FUTURES_MULTIPLIERS` (ES 50, NQ 20, …), akcie/ETF/index 100, neznámý futures kořen `null` . Jediný zdroj hodnot pro prémie v $ ve frontendu i pro `paper.POINT_VALUES`
`GET /instruments/{sym}/days`	Uložené dny s expirací per den (Daily pohled)
`GET /profile/{sym}/aggregate?date`	Σ profil: OI/volume sečtené přes všechny expirace dne per strike (registrováno PŘED /profile/{sym}/{expiry})
`GET /snapshots/{sym}/{expiry}?date&mode&scale&norm&from&to&raw`	Heatmap matice jako **Arrow IPC stream**; `raw=true` = surová partice
`GET /levels/{sym}/{expiry}?date`	Časová řada flip/walls/centroid
`GET /profile/{sym}/{expiry}?date&ts&variant&oi_weight&spot`	Strike profil k okamžiku
`GET /flow/{sym}?date`	CumΔ + OptVol + Vol řady (OptVol = přírůstky proti průběžnému maximu kontraktu, `volume_increments` , #1602); s `from` / `to` (#483) i okenní souhrn `window` . Minuta bez tisků (`cum_delta` NULL, ADR-0057 bod 3) dá v okně (from, to] `window.cum_delta: null` — rozdíl přes mezeru je neúplný; hranice okna v mezeře bere poslední měřenou úroveň (#1588).

Endpoint	Popis
`GET /replay/{sym}/{expiry}/{date}`	Kompletní denní balík (levels/flow/bars JSON + snapshoty base64 Arrow + `oi_prev` pro ΔOI vs. včera)
`GET /replay/{sym}/{expiry}/{date}?resolution=daily`	Daily pohled (#1206): snapshoty jen poslední minuty, řady zredukované na poslední stav a pole `daily` (denní OptVol / Δ Flow / Evo OI / CumΔ + OHLC, stejné vzorce jako UI kromě minuty po mezeře v objemu, viz níž) — stovky kB místo 20–40 MB. `daily.cum_delta` = poslední **měřená** úroveň seance (`null` , když seance nic neměřila), `daily.cum_delta_gap_minutes` = počet minut bez tisků (NULL, ADR-0057 bod 3; pod v5 0) — dřív NULL na konci seance shodil balík na HTTP 500 uložené v cache až do restartu (#1588). **OptVol / Δ Flow** (`daily_totals` , #1602) = přírůstky denního kumulativního volume proti **průběžnému maximu** kontraktu v seanci (`volume_increments`): IBKR kumulativ kmitá (862 ↔ 863) i propadá (pauza Globexu, výjimečně přes noc) a přírůstek proti poslední hodnotě započítal návrat znovu; NULL (tasty fallback) ani chybějící strike nejsou 0 a první výskyt kontraktu jen založí stav. Počítá se při čtení ze snímků, takže oprava platí i zpětně (ES 15. 10. Opt Vol C+P 501 501 → 349 944, 7. 10. 1 623 419 → 305 961). Catch-up minuta (#518) i minuta po mezeře v objemu (strike bez objemu v předchozí minutě osy: NULL z tasty fallbacku, díra ve sběru; rozhodnutí B 21. 10., #1632) se do Daily počítají (objem výpadku), minutový panel (`ensureDerived`) je kreslí jako 0 a přírůstek jen posune maximum — Daily tak za den s mezerou nese víc než součet panelu. Engine počítá totéž pravidlo pod mechanikou v6 (`CumDeltaTracker.add_volume` , `setups._flows` , `tendency._flows` , replay; kap. 5 krok 4, ADR-0057 doplněk 22. 10.), minutu po mezeře tam jako 0. **Verze v URL:** frontend volá `?resolution=daily&v=N` (`DAILY_BUNDLE_VERSION` v `frontend/src/replay/loader.ts` a v `api/src/gexlens_api/main.py` , shodu hlídá pytest) — uzavřený den jde s `Cache-Control: immutable` na rok a prohlížeč ho drží podle URL, takže **každá změna obsahu Daily balíku** (součty, `cum_delta` , řady) **verzi zvedá**. API odpoví `immutable` a uloží do `_DAILY_BUNDLE_CACHE` jen dotaz se svou verzí; jiná nebo chybějící verze (otevřený starý frontend, návrat jen jedné služby) dostane `no-cache` . API bez této stráže (před #1602) ji neumí: při nasazení jde **API před frontendem** (pořadí deploy skriptu i `depends_on`) a návrat API pod #1602 jen spolu s frontendem, jinak si prohlížeč staré součty uloží pod novou URL na rok
CRUD `/watchlist` , `/alerts` , `/annotations?symbol&date` , `/settings`	PostgreSQL persistence
`POST /internal/status` , `POST /internal/publish`	**Ingest z enginu** — vyžaduje hlavičku `X-GEXLens-Token` (#542). Od #949 tu API vyhodnocuje **provozní alerty** `AlertEngine.observe_connection` / `observe_disk` (výpadek spojení s IBKR, obsazení disku přes limit) — ze **snímku** statusu, ne z těla requestu (engine posílá jen změněné klíče); obě hlášky jsou hranové. Výpadek spojení se při zavřeném trhu podle rozvrhu

Endpoint	Popis
	CME nehlásí a hrana se nenatahuje — trvá-li i po otevření, ohlásí ho první status po otevření (#1307, kap. 5). Do té doby byl `AlertEngine` mrtvý kód; pravidla `price_cross` / `cum_delta_jump` / `dominant_strike_change` odstraněna jako překonaná (`LevelProximityWatcher`), `POST /alerts` je přestává přijímat, CRUD `/alerts` zůstává
`PUT /settings/{key}`	Zápis nastavení + `pg_notify` na kanál watchlistu (#992) — engine se probudí do sekund
`GET /backup/postgres`	Stream `pg_dump -Fc` — vyžaduje `X-GEXLens-Token` (#542)

WebSocket `/ws/live`

Protokol: klient pošle `{"action":"subscribe","channels":["status","price.ES","levels.*"]}` (podpora trailing wildcard), server vrací ack a pushuje `{"channel":..., "data":...}` . Backpressure: fronta 100 zpráv per klient, při zaplnění se zahazují nejstarší framy. Kanály: `status` , `price.{sym}` , `snapshot.{sym}.{expiry}` , `levels.*` , `flow.*` , `alerts` , `news` . Upozornění na zprávy (`news_anomaly` , `news_preopen` , #1291; `release_preview` , #1296) nesou navíc `ts_event` (začátek shluku, otevření Globexu, resp. čas releasu) a `event_ids` ; zvoneček z nich dělá proklik do grafu přes `GET /news/markers?ids=` (#1290). Fronta 100 framů může při dávce klasifikace zprávy zahodit — graf to dohání minutovým dotažením od posledního úspěšného dotažení − 30 min.

Handshake kontroluje hlavičku `Origin` (#542): CORS se na WebSocket nevztahuje, takže bez téhle kontroly by živý positioning četla libovolná stránka otevřená v prohlížeči. Povolen je same-origin (Host stránky za nginx), localhost v libovolném portu a cokoli v `GEXLENS_ALLOWED_ORIGINS` . Klienti bez hlavičky `Origin` (engine, curl) projdou. Stropy: 64 souběžných spojení, 256 kanálů na spojení.

8. Frontend

- **Heatmapa**: data → offscreen bitmapa (překreslení jen při změně dat/módu), pan/zoom = GPU `drawImage` ; výkonové cíle a jejich měření měřičem #1409 jsou v ADR-0047 bod 6 (baseline `docs/research/1381-vykon-baseline.md`); overlay canvas kreslí vektory (cena/svíčky, levels, walls, sessions, crosshair, anotace). Měřená vrstva a Dyn podklad mají každá vlastní rastr, přepočítá se jen ta, které se změnily vstupy: Mode s podkladem přepočítá jen měřenou (#1420). Kontury počítá web worker, hladiny i flip jednou úlohou nad sdíleným blurem; úloha odchází před kresbou overlay, overlay se kreslí hned s daty a po doručení kontur jednou znovu (#1420).
- **Replay**: `/replay` se stáhne jednou, `apache-arrow` dekóduje snapshoty, celý den se předpočítá v paměti (vč. profilu per minuta) — přetáčení je čisté krájení typed arrays. Timestampy se normalizují (`canonicalTs` — Arrow epoch vs. JSON ISO).
- **Stav**: React kontexty `AppState` (status z WS + REST initial fetch, view, téma, alerty) a `Crosshair` (sdílený všemi panely).
- **OI fallback**: při nulovém OI staví heatmapu z volume (engine mezitím posílá alert `oi_missing`).
- **Měřič výkonu grafu** (`frontend/src/perf/meter.ts` , #1409, E-2a.1a): ve výchozím stavu vypnutý a vypnutý nic nealokuje (neběží rAF ani `PerformanceObserver` ; instrumentovaná místa jen větví, ~35 ns na operaci). Zapnutý sbírá délku snímku (fps, p50, p95), dlouhé úlohy ≥ 50 ms (počet a maximum za minutu), operace `drawStatic` , `drawDynamic` , `buildModeGrid` , `renderGrid` , flush minuty (`minuteFlush` = výběr minut v timeru, bez appendu), fáze dne (`dayFetch` , `dayDecode` , `dayBuild`), vlastní čas renderu komponent `renderApp` , `renderHeatmap` , `renderPanels` , `renderProfile` (hooky a memo výpočty těla komponenty bez potomků — rozpad renderu Reactu bez profiling buildu, #1419) a spany do vykreslení: `dayLoad` (fetch → mřížka dne), `symbolSwitch` (klik → mřížka nového symbolu se skutečnými daty; `useDayData.dataSymbol` musí sedět na symbol, první render po přepnutí nese ještě stará data), `modeScale` (přepnutí → rastr i hotové kontury nové mřížky), `minuteUpdate` (flush → mřížka s novou minutou, včetně appendu a sestavení dne) a `gestureFrame` (krok panu nebo zoomu → vykreslení obou vrstev overlay; gesto zapisuje pohled nejvýš jednou na snímek přes rAF a zoom X a pásmo Y persistuje jednou na konci gesta — puštění tlačítka, 200 ms klidu kolečka, #1419). fps se počítá z délek snímků; pauza rAF ve skryté

záložce se do snímků nezapočítá. Drží kruhový buffer 30 s se záznamy s pohledem (symbol, timeframe, Mode). Počty od zapnutí jsou deterministické (výkonový rozpočet v CI, E-2a.13). UI nemá (overlay, Diagnostika ani „Seklo to“ nevznikají, vlastník 12. 10., #1411); ovládá se z konzole prohlížeče: `gexlensPerf.setPerfEnabled(true)`, po měření `copy(JSON.stringify(gexlensPerf.perfExport()))`, jen souhrn pro komentář na GitHubu `gexlensPerf.perfExport(undefined, { records: false })`. Export nese verzi buildu `build` = commit z CI image (build arg `VITE_BUILD_SHA`), lokální build `dev`. Režie zapnutého: ~0,3 µs na operaci a ~0,1 µs na snímek; export 30 s je ~17 ms.
- Wiki/manuál: statické HTML v `frontend/public/manual/` (generované z MD, viz níže) — servíruje ho vite dev i nginx.

9. Vývojové prostředí

Prerekvizity: uv (stáhne Python 3.12 sám), Node.js ≥ 20, Docker (pro PG integrační test lokálně volitelně).

```
uv sync --all-packages                     # Python workspace (engine + api)
uv run ruff check .; uv run ruff format .  # lint/format
uv run mypy engine/src engine/tests api/src api/tests
uv run pytest                              # PG integrační test se přeskočí bez GEXLENS_TEST_PG_DSN

cd frontend; npm ci; npm run lint; npm test; npm run build
```

Dev servery: `make run-api` (uvicorn :8000), `make run-frontend` (vite :5173), `make run-engine` (vyžaduje TWS). CORS povoluje i :5173.

Regenerace manuálů (MD → HTML pro in-app wiki → PDF): `powershell scripts/build-manual.ps1` (vyžaduje Edge; PDF vzniká headless tiskem).

Konvence: feature branch `feat/{issue}-slug` / `fix/...`, PR s `Closes #N`, merge po zeleném CI. Výpočty vždy s golden testy v `engine/tests/golden/` (ručně spočtené hodnoty, výpočet dokumentovaný v `description`).

Oddělená prostředí DEV a PROD (#568)

Vedle produkčního stacku (`compose.yml`, :8080) existuje dev stack (`compose.dev.yml`, projekt `gexdev`, :8081) s vlastním PG volume (`gexdev_pgdata`) a vlastní kopií parquet dat (`data-dev/`). Cíl: vývoj se nikdy nedotkne produkčních dat, která nejdou znovu pořídit (věčný OI archiv, setupy, track record).

Skript / ikona	Co dělá	Souběh s prod
`scripts/start-prod.ps1` (ikona **GEXLens**)	produkce; shodí dev-live, pokud běží	—
`scripts/start-dev.ps1` (ikona **GEXLens DEV**)	dev bez enginu: PG + API + frontend nad kopií dat	**povolen** — market data účtu se nedotkne, prod dál sbírá
`scripts/start-dev.ps1 -Live` (ikona **GEXLens DEV+Engine**)	plný stack proti TWS	**zakázán** — skript nejdřív shodí produkci (jeden účet); po dobu běhu prod nesbírá
`scripts/start-dev.ps1 -LiveTasty` (`start-gexlens-dev-live-tasty.cmd`)	dev engine **jen s tastytrade** (#623): IBKR vypnuto, žádné výpočty ani zápisy — stream chainu do cache s minutovým heartbeatem v logu enginu	**povolen** — tasty snese souběžné streamy (ADR-0027), produkce nepřijde o minutu; news-engine se nestartuje (mluví s TWS)
`scripts/seed-dev.ps1`	obnoví dev PG z nejnovější zálohy + zrcadlí `data/` → `data-dev/`	povolen

Pravidla:

- **Produkce pouští výhradně `main`.** `start-prod.ps1 -Build` odmítne stavět z jiné větve nebo ze špinavého stromu (`-Force` = vědomé obejití). Bez `-Build` se jen startují dřív postavené image. Dev pouští libovolnou rozpracovanou větev.
- **Image se staví v CI, ne doma (#1139, 13. 9. 2026):** po mergi do `main` workflow *Images* postaví `ghcr.io/kechicz/gex-python` (engine, api, news-engine) a `ghcr.io/kechicz/gex-frontend` (tagy `latest` + `sha-<7>`, label `org.opencontainers.image.revision`). Doma se image jen stahují (`docker compose pull`), lokální build zůstává nouzový (`deploy-engine-offhours.ps1 -Build`, `start-prod.ps1 -Build`). Balíčky jsou **veřejné** (repo je veřejné, veřejné balíčky jsou zdarma bez limitu; soukromé by narazily na 500 MB) — stažení nevyžaduje přihlášení. Jednorázově po prvním pushi: na stránce balíčku (GitHub → Packages → gex-python / gex-frontend → Package settings → Change visibility → Public). Do image jde jen to, co Dockerfile kopíruje; `.env*`, `data/`, `docs/` vylučuje `.dockerignore`.
- **Nasazení po mergi:** `git checkout main && git pull`, pak `.\scripts\deploy-engine-offhours.ps1` (stáhne image, počká na revizi HEAD, restartuje v okně). Starší cesta `.\scripts\start-prod.ps1 -Build` staví lokálně. Před nasazením, které sahá na schéma DB, vždy `.\scripts\backup-postgres.ps1` — izolace dev to nenahrazuje, je to druhá vrstva. Výchozí cíl záloh je od #439 (26. 8.) `D:\Programy\GEX\zalohy-pg` (dumpy zabíraly 3,6 GB na systémovém C:), vlastní složka přes `-Target`. Každý dump skript ověří celým čtením `pg_restore --file=/dev/null` (~1 min CPU kontejneru); při selhání ho přejmenuje na `.invalid` (mimo rotaci) a skončí chybou. Denně ho pouští úloha GEXLens backup-postgres (kap. 13.6, #1566).
- Dev frontend nese v sidebaru oranžový badge **DEV** (build arg `VITE_GEXLENS_ENV`), ať se okna prohlížeče nespletou.
- Dev stack je jednorázový: rozbitý dev = `docker compose -f compose.dev.yml down -v`, smazat `data-dev/`, `seed-dev.ps1` znovu.
- Dev engine má výchozí `clientId 2` (`GEXLENS_DEV_IBKR_CLIENT_ID`), aby se v TWS nepotkal s produkční jedničkou.
- `-LiveTasty` si bere konzervativní strop 2 000 subskripcí (`GEXLENS_TASTY_MAX_SUBSCRIPTIONS`; měřeno 6 008 bez degradace, ADR-0027) — kapacita je pravděpodobně per účet, dev nesmí ujídat produkci. Vyžaduje dev tasty grant v `.env.dev` (#696). Pozor: cross-feed logika (shadow #613, fallback #614) se v tomto režimu ověřit nedá — potřebuje oba feedy vedle sebe.

10. Testy a CI

- **Python** (~160): jednotkové + golden (GEX, levels, heatmap módy, walls, CumΔ, profil), mock-based integrační (scheduler, hot zóna, runtime), PG integrační (v CI přes service kontejner), **e2e smoke** — deterministický referenční den přes celou pipeline engine→storage→API proti golden hodnotám.
- **Frontend** (~58): jednotkové (geometrie, barvy, contours, slice), komponentové (jsdom + testing-library, PointerEvent polyfill), Arrow round-trip loaderu, **e2e render smoke** (App nad /replay balíkem), vizuální regresní snapshoty renderu.
- **CI** (GitHub Actions, na každý PR): python job (ruff, mypy strict, pytest + PostgreSQL service), frontend job (eslint, prettier, vitest, build). Výkonnostní testy s tvrdým limitem běží jen lokálně (`CI` env skip).

Bezpečnostní kontroly v CI

Job `security` na každém PR: gitleaks (celá historie — pozor, test s realisticky vypadajícím tajemstvím spadne i po přepsání souboru, dokud je v historii větve), pip-audit, npm audit. Lokálně `pwsh scripts/security-scan.ps1`.

---

11. Známé limity účtu a otevřené body

Z ADR-0001 (měřeno živě na účtu):

Limit	Hodnota	Dopad
Tick-by-tick	**5**	Bez použití od ADR-0032 (3. 9. 2026): klasifikaci agresora dodá dxFeed `TimeAndSale` bez limitu; IBKR tick-by-tick zóna zrušena.

Limit	Hodnota	Dopad
streamy		
Market data lines	**100**	Přidělení IBKR pro API (TWS „Maximum Allowed", 9. 10. 2026, #1479; sonda #1477 dostala krátkodobě tick i u 250 linek, spolehlivé to není). Původní údaj „≥ 150" v ADR-0001 neplatí — dávka 80 jede blízko stropu (~95–100/100), `batch_size` **proto nezvyšovat**. Strukturální řešení přinese #616.
FOP OI	**tick 588 nedodává nikdy; tick 101 funguje**	**VYŘEŠENO (issue #65, ADR-0001 v3):** `IbOIFetcher` používá generic tick 101 pro OPT i FOP a čte hodnotu podle strany kontraktu (opačná strana = validní 0.0). Retry à 30 min + volume fallback zůstávají jako pojistka.

ADR-0002: obálka strikes je grow-only (křídla se neztrácejí), strop šířky s alertem. ADR-0003: multi-instrument orchestrace řízená watchlistem.

Sekundární datový zdroj — tastytrade/dxFeed (M7)

Naměřené limity a pasti feedu: **ADR-0027** (6 000+ symbolů na subskripci, REST ≥ 6 req/s, povinné KEEPALIVE, dekádová kolize futures candle symbolů). Přístup výhradně **OAuth2 scope** `read` — nikdy `/sessions`, nikdy `trade` (ADR-0025); granty oddělené pro dev a produkci. Shadow mód (#613) porovnává oba feedy do `feed_comparison`, nic nepublikuje; vyhodnocení `scripts/feed_comparison_report.py`.

Konfigurace je jen přes `.env`, v Settings tastytrade není. Od fáze 2 (#614) se ale aktivní fallback **ukazuje v hlavičce** jantarovým chipem — tiché přepnutí zdroje zakazuje ADR-0025 pravidlo 5.

Pozor na klíče, které engine nečte. Model `Settings` má `extra="ignore"`, takže neznámý klíč se **tiše zahodí**. `GEXLENS_TASTY_CLIENT_ID` z `.env.example` nepoužívá nikdo (refresh flow posílá jen `refresh_token`

- `client_secret`) a `GEXLENS_DEV_TASTY_*` čte napřímo z prostředí jen sonda `scripts/tasty_probe.py`. Když nastavení „nezabírá", ověř nejdřív, že ten klíč engine vůbec zná — seznam je v kapitole 4.

Vydání a odvolání grantu (#620)

Granty jsou **dva na jednom účtu** — zvlášť pro dev a zvlášť pro produkci (ADR-0025). Jde je odvolat nezávisle, takže zabití dev přístupu nechá produkci běžet dál.

Vydání

1. V tastytrade **Manage → Create Grant**, potvrdit druhým faktorem.
2. Zaškrtnout **výhradně scope** `read`. `trade` se nezaškrtne **nikdy** — nejde o důvěru v kód, ale o to, že právo, které aplikace nemá, nelze zneužít. `openid` jen tehdy, pokud ho autorizační tok vyžaduje.
3. **Client secret se ukáže jen jednou.** Kdo ho v tu chvíli neuloží, musí vydat grant znovu — zpětně se přečíst nedá.
4. Hodnoty zapsat do `.env` (produkce), resp. `.env.dev` (dev), pod klíči `GEXLENS_TASTY_CLIENT_SECRET` a `GEXLENS_TASTY_REFRESH_TOKEN`. Nikdy do repa ani natvrdo do compose. Před zápisem ověř, že soubor **končí novým řádkem** — append bez něj přilepí klíč k předchozí hodnotě.
5. Restartovat dotčený stack. Refresh token neexpiruje, access token (15 min) si engine obnovuje sám.

Ověření, že grant platí

co	kde	očekávané
spojení	`GET /status`	`tasty_connected: true`
subskripce	log enginu	`DXLink subskripce kompletní: N symbolů`

co	kde	očekávané
politika přístupu	`pwsh scripts/security-scan.ps1`	sekce *tastytrade přístup* bez nálezů

Odvolání

V **Manage** odvolat příslušný grant. Engine tím přijde o obnovu access tokenu, tastytrade větev přestane fungovat a s ní i křížová kontrola (#517 A), oba fallbacky (#614) a OI fill (#664) — IBKR cesta běží dál. Aktivní zdroj je vidět v hlavičce jantarovým chipem, takže ztráta nezůstane tichá (ADR-0025 pravidlo 5).

Rotace při podezření na únik: nejdřív **odvolat**, teprve pak vydávat nový. Opačné pořadí nechává uniklý token platný po celou dobu, kdy se vyplňuje `.env`.

Trvalé × dočasné je od #763 rozdělené. Do té doby hlídal jeden flag `GEXLENS_TASTY_SHADOW` obojí, takže nejpřirozenější možná úvaha — „měření doběhlo, vypínám ho" — tiše vypnula i křížovou kontrolu, oba fallbacky, OI fill a publikaci ceny bez pipeline. Nově:

- `GEXLENS_TASTY_ENABLED` drží **trvalou** větev (default `true`),
- `GEXLENS_TASTY_COMPARISON_WRITE` jen **dočasný** zápis do `feed_comparison`.

Až doběhne vyhodnocení M7 fáze 2, vypíná se **druhý** z nich. Monitor běží dál a dodává tally detektoru i fallbackům; ověřuje to test, který porovnává tally se zapisováním a bez něj — musí vyjít shodně, jinak by se aplikace po konci měření začala rozhodovat jinak.

Provozní stav tasty větve (reconnecty DXLink, handshake, počet sledovaných symbolů, zapsané řádky) končí **jen v logu kontejneru** a v tabulce `feed_comparison`; v `/status` je z něj `chain_source` a `spot_source`. Samostatná obrazovka je zadaná v #706.

Křížová kontrola feedů (#517 fáze A)

Heartbeat testuje jen TCP spojení a stall detektory (ADR-0015) měří pasivně stáří dat — incident 26.–27. 7. (15 h zmrzlé ATM greeks při tekoucích cenách) nezachytila ani jedna vrstva. Chyběla **nezávislá reference**: mlčí data, nebo mlčí trh? Běžící tasty větev ji dodává zadarmo, takže detektor nedělá **žádný request na IBKR a nebere žádnou market data line** — jen čte, co `compare_minute` stejně počítá.

Rozhoduje se z rozpadu kontraktů podle čerstvosti obou stran:

Situace	Verdikt	Chování
IBKR mlčí ∧ tasty data má	`ibkr_suspect`	**alert** — tasty vylučuje „tichý trh"; farmu od mrtvých subskripcí rozliší až sonda fáze B
oba mlčí	`quiet`	ticho — nikdo neobchoduje, není co hlásit
tasty mlčí ∧ IBKR data má	`tasty_suspect`	**jen stav a log, bez alertu** — viz níže
oba dodávají	`ok`	—

Proč zrovna 70 % / 3 minuty. Sweep obchází kontrakty po dávkách `batch_size`, takže **každou třetí minutu** vyskočí podíl „IBKR mlčí" na ~58 % a hned zase spadne — rotační artefakt, ne porucha. Okamžitý podíl je proto jako signál nepoužitelný a naivní práh by alertoval 24/7. Měření na 3 016 minutách čisté shadow historie (13.–16. 8. 2026) — nejdelší souvislá série minut nad prahem:

práh	nejdelší série	počet epizod
50 %	2 min	696
60 %	2 min	42
70 %	**2 min**	**2**
80 %	1 min	1

Série tří minut v řadě nenastala při žádném prahu ani jednou. Podmínka „M minut v řadě" tedy rotaci odfiltruje úplně; `_MINUTES` snížené na 2 by naopak plašilo.

Proč `tasty_suspect` nealertuje. Přehrání téže historie detektorem dalo 41 epizod „tasty mlčí ∧ IBKR data má" — všechny v hodinách **21–06 UTC**. Příčina je konstrukční: dxFeed je **event-driven** (v klidu neposílá nic, okno stáří vyprší), IBKR sweep **poll-driven** (vrací poslední kotaci pořád dokola). V noci a v denní pauze CME (16:00–17:00 CT) je to normální stav, ne porucha — v alert kanálu by z toho bylo 41 planých poplachů. Stav se proto jen poznamená do `/status` a jednou na začátku epizody do logu.

Kontrolní měření po téhle úpravě: **3 053 minut čisté historie → 0 alertů.**

Při zavřeném trhu podle rozvrhu CME (víkend, denní pauza) je verdikt vždy `quiet` a série se nulují (#1307): dxFeed po resubskripci a při přechodu seance pošle snímek posledních hodnot, takže „IBKR mrtvé ∧ tasty čerstvé" tam vzniká i bez poruchy.

Stav je v **Settings → Stav enginu** (řádek *Křížová kontrola feedů*) a v `/status` jako `feed_crosscheck`. Když tasty větev neběží, pole ve statusu **chybí** — UI to ukáže jako „neměří se", což je jiný stav než `ok`.

Fallback na tastytrade (#614 fáze 2a a 2b)

Když IBKR přestane posílat data, přebírá je tastytrade. Typická příčina není porucha, ale **souběh s mobilem**: market data jsou per uživatel, takže přihlášení do IBKR aplikace v telefonu přetáhne feed k sobě (error 10197) a engine zůstane připojený — jen mu přestanou chodit ticky.

Co	Kdy nastoupí tasty	Kdy se vrátí IBKR
Spot podkladu (2a)	30 s bez ticku	60 s souvislých dat
Opční řetěz — kotace, greeks, OI (2b)	verdikt `ibkr_suspect`, tedy 3 min	5 čistých minut v řadě

Při zavřeném trhu podle rozvrhu CME se spot nepřepíná a po otevření se ticho IBKR měří od okamžiku otevření (#1307, kap. 5 „Provozní upozornění při zavřeném trhu").

Řetěz se spouští **verdiktem křížové kontroly**, ne vlastním prahem — dědí tak kalibraci měřenou na 3 016 minutách místo nového odhadu. Návrat vyžaduje `state == "ok" AND streak == 0`: samotné `ok` nestačí, protože detektor ho vrací i pro minutu nad prahem, která zatím nenaplnila sérii do alertu. Stavy `quiet` a `insufficient` sérii jen nulují — tichý trh není důkaz uzdravení.

Co fallback vědomě NEdodá: kumulativní denní objem. Tasty ho ve stejné sémantice nemá, a dosadit nulu by znamenalo skokový záporný přírůstek přes celý řetěz. Ve snímcích je objem `NULL` — díra, kterou je vidět, místo čísla, které lže. Pod mechanikou v5 (zdroj `midpoint`) proto po dobu fallbacku řetězu **stojí CumΔ a net objem**. Od v6 (ADR-0057, zdroj `aggressor`) jdou CumΔ i net objem z tisků tastytrade dál a chybí jen rozklad objemu na tisky (printvol, pokrytí). Text alertu `chain_fallback` se řídí stejnou podmínkou. Heatmapa, GEX profil, zdi i flip běží dál.

Přepnutí se hlásí alertem a jantarovým chipem v hlavičce; `/status` nese `chain_source` a `spot_source` (`ibkr` | `tasty`). `spot_source` se agreguje pesimisticky — stačí jeden instrument na fallbacku a status hlásí `tasty`, protože výpadek market data je vlastnost účtu.

Slepé místo „mrtvá tasty větev selže tiše" uzavřelo #764: `tasty_suspect` sám o sobě dál nealertuje (41 planých epizod v noci), ale rozlišovač `GEXLENS_CROSSCHECK_CHANGE_THRESHOLD` (kap. 4) pozná, že se trh hýbe (IBKR hodnoty se mění) a tasty přitom mlčí → alert `feed_backup_dead`. Záloha, která umřela za běhu, se tak pozná dřív než při výpadku IBKR.

Degradovaný start a co ještě přebírá tasty (v1.17, #1153, ADR-0025 dodatek 14. 9.)

Fallback výše řešil jen **běžící** pipeline. Od 14. 9. přechod na tasty platí i tehdy, když IBKR vypadne dřív, než se pipeline založí (Error 1100 = TWS bez spojení s IBKR, nebo Gateway vyhozená souběhem s mobilem — API port odmítá):

- **Cache discovery** `data/derived/discovery_cache.json` — per symbol front futures kontrakt + všechny expirace a striky z posledního úspěšného IBKR discovery (přepisuje se při každém úspěchu). Při selhání discovery nebo bez API socketu se z ní pipeline založí („degradovaný start", alert `degraded_start`, log `Degradovaný start ES (#1153): …`), úvodní spot dá tasty, řetěz/OI/svíčky fallbacky. Záznam se nepoužije, když je starší než 14 dní, front kontrakt expiroval nebo nemá dnešní expiraci → pipeline pak čeká na IBKR (čistá instalace, roll během výpadku).
- **Cyklus pipeline bere spot z tasty** (`spot_override`), takže GEX ani hlídač barů nepočítají nad cenou z doby výpadku. Pod mechanikou v6 (#1600, ADR-0057 doplněk
  23. 10.) má spot cyklu stáří (`compute/spot.resolve_spot`): IBKR `ticker.last` (tick ≤ 30 s, mid ne) → close posledního 5s baru (≤ 60 s) → tasty mid hlášený jako čerstvý (poll ≤ 30 s, jen s `GEXLENS_TASTY_SPOT_FALLBACK`). Když není živý žádný zdroj, verdikt dne je neúplný (`missing` : „spot: poslední cena … stará N s"), automatický scénář nevznikne a tendence nehlasuje z polohy ceny; GEX výpočet běží s nejčerstvější známou cenou. Log `Spot ES: žádný zdroj nemá živou cenu` (jen při otevřeném trhu a ne v prvních 2 min po startu a otevření) / `živá cena zpět`. Pod v5 platí override jako dosud.
- **Svíčky podkladu**: při stall (3 min bez real-time barů při živém spotu) engine každou minutu doplní chybějící minuty z dxFeed Candle (`source = tasty_candle` , banner „doplněno" v UI); jedno pomocné DXLink spojení naráz (souběžný fetch ES+NQ nechával druhý bez svíček).
- **Tasty subskripce řetězu** se plánují z kontraktů, které pipeline drží, a pro symboly z konfigurace i watchlistu bez pipeline; kontrakt bez IBKR kotace je pro křížovou kontrolu mrtvá IBKR strana (jinak by po restartu bez IBKR hlásila „sledováno 0 kontraktů" a fallback řetězu se nezapnul).
- **Stáří tasty hodnot = živost streamu** (`GEXLENS_TASTY_CHAIN_MAX_AGE_S` je práh posledního eventu STREAMU): dxFeed posílá jen změny, deep OTM strike nezměněný 20 min je na živém streamu aktuální. Kontrakt bez dxFeed Greeks dostane BS greeks z mid (`greekssource = computed`), bid 0 zůstává ve snímku (limitně nulové greeks) — jinak ze strike profilu mizelo OI těch striků.
- **IBKR volání mají stropy**: discovery podkladu 3×30 s, discovery řetězu 30 s, kvalifikace kontraktu 20 s a po timeoutu 60 s fail-fast (do 14. 9. `qualifyContractsAsync` bez stropu držel OI archiv i celý orchestrátor).

Diagnostika: `/status.spot_source` , `chain_source` , `feed_crosscheck` , `tasty_symbol_breakdown.chain` (má odpovídat počtu držených kontraktů, 0 = fallback nemá z čeho brát); log `Cyklus ES …: N snapshotů` má za fallbacku odpovídat obálce (ES 160, NQ 96; `greeks 0/160` v tomtéž řádku počítá jen IBKR modelGreeks, tasty greeks jsou ve snímku).

Start bez běžícího TWS (#756)

Engine na IBKR čeká **nejvýš** `GEXLENS_STARTUP_CONNECT_WAIT_S` (default 60 s) a pak rozjede zbytek i bez něj. Do #756 tu bylo nekonečné čekání, za kterým zůstalo úplně všechno — schéma DB, pipelines i tastytrade větev — takže po startu Windows bez spuštěné TWS neběželo nic.

Co platí po vypršení stropu:

- pipeline se zakládá jen **degradovaně** z cache discovery + tasty spotu (v1.17, #1153 — viz výše); bez cache se nezakládá; existující se **neruší** — `plan.stop` se řídí watchlistem, ne spojením,
- tasty se odebírá i pro instrumenty **bez** pipeline, jinak by fallback neměl z čeho stavět,
- cenu publikuje samostatná smyčka, která pro symbol s běžící pipeline umlkne,
- `/status` chodí i bez pipelines a `connection` nese **skutečný** stav — „engine běží" a „IBKR je připojen" jsou dva různé stavy.

Pipeline se založí sama, jakmile se spojení objeví. Restart enginu není potřeba.

---

12. Diagnostika a údržba

Situace	Postup
Engine offline	`docker compose logs engine` — hledej stav ConnectionManageru; ověř TWS (API zapnuté, port, Trusted IP). Warning 2110/2103 = výpadek TWS↔IB, vyřeší se sám.
Prázdné GEX/walls	Zkontroluj `oi_eod` pro dnešek: `docker compose exec postgres psql -U gexlens -c "select date, count(*) from oi_eod group by 1 order by 1 desc limit 5"` — pokud dnešek chybí, engine archiv opakuje à 30 min při otevřeném trhu (CME publikuje OI ráno; o víkendu a v denní pauze se neopakuje, #1307).
Ticker z watchlistu nesbírá	`docker compose logs engine
Vysoké `Repair` / `Stale`	Konkrétní kontrakty bez dat — často nelikvidní křídla; zvyš `BATCH_TIMEOUT_S` nebo zmenši obálku.
Disk roste	Retention běží nočně; ručně: smaž staré partice v `./data` (nikdy `oi_eod`).
Panel Sentiment plochý na nule (ES/NQ)	Ověř váhy: `docker compose exec postgres psql -U gexlens -c "select symbol, category, predictor, round(weight::numeric,3) from news_weights order by 1,2,3"` — od ADR-0036 (#1150) je váha vždy v [0,25; 2,0], mince = 1,0; samé nuly znamenají image před #1150. Řadu zpětně spraví `docker compose exec news-engine python -m gexlens_news recompute-sentindex --from YYYY-MM-DD [--to YYYY-MM-DD]` (přepíše partice `derived/sentiment/{SYM}/` i svíčky `sentiment_daily` toutéž mechanikou jako živý job, σ/close_z od `--from` dál dopočte znovu; dnešek jen do „teď"). Bez eventů se skórem (`select count(*) from news_events where sentiment_score <> 0 and ts_event > now() - interval '1 day'`) je problém v klasifikaci, ne ve vahách.
Reset prostředí	`docker compose down`, smaž `./data` (přijdeš o 14denní okno, ne o OI archiv ve volume `pgdata`), `docker compose up -d --build`.
Málo dat po restartu	Writer navazuje na rozepsaný den — mezera zůstane jen za dobu výpadku.
Souběh s mobilem: graf stojí déle než ~4 min	`/status` → `spot_source` / `chain_source` mají být `tasty`, `tasty_symbol_breakdown.chain` > 0; log `grep -E "Degradovan|Fallback řetězu|Rekonstrukce|Setup .* selhal"`. `Setup … nedorazila cena podkladu` = tasty nemá čerstvý spot symbolu (front future není v subskripci — od #1163 se odebírá i pro watchlist); `0 snapshotů` s `chain_source: tasty` = chybí tasty chain subskripce (`chain: 0`). Cache discovery: `python -c "import json;print(json.load(open('data/derived/discovery_cache.json')).keys())"`. Nerestartovat engine bez příčiny — restart během výpadku IBKR = degradovaný start (~2–3 min).
Potřebuju log enginu z doby PŘED restartem/deployem	Log kontejneru zmizí s jeho recreate. `scripts/deploy-engine-offhours.ps1` ho od #1056 (v1.6) ukládá sám do `data/logs/engine-<YYYYMMDD-HHMM>.log` (a `…-crashed.log` před rollbackem; bez uloženého logu se nenasazuje; retence 30 dní). Při **ručním** `docker compose up -d engine` / `--force-recreate` udělej totéž předem: `docker logs gex-engine-1 > data/logs/engine-$(Get-Date -Format yyyyMMdd-HHmm).log 2>&1`. 7. 9. (#1054) se bez toho hodinu řešil falešný poplach.
Docker roste / disk D: dochází / Docker Desktop se nerozjede	`pwsh scripts/docker-cleanup.ps1` (rollback tagy, build cache > 7 d, dangling; `-WhatIf` jen ukáže) → když VHDX zůstane velký, `pwsh -File scripts/compact-docker-vhdx.ps1` jako správce při zavřeném trhu (12. 9. 2026: 70,9 → 24,9 GB). Nikdy `docker volume prune` ani reset Docker Desktopu — Postgres je uvnitř VHDX. Podrobně kap. 13.4. **Kompaktace VHDX automaticky**

Situace	Postup
	(24. 9. 2026, rozvrh #1277): jako správce `pwsh scripts/register-compact-vhdx-task.ps1` (znovu po každé změně rozvrhu) → elevovaná úloha „GEXLens compact-vhdx" (po/st/pá 23:05 v pauze Globexu + sobota 10:00, #1277; navíc ji spouští `docker-cleanup.ps1` po úklidu image, když je VHDX > 40 GB); skript `compact-docker-vhdx.ps1 -IfNeeded` sám hlídá práh ≥ 10 GB k vrácení a zavřený trh, po kompaktaci ověří, že 5 kontejnerů gex-* naběhlo (log `data/logs/compact-vhdx.log`). Kompaktace nic nemaže — retence: 3 rollback tagy na službu (`-KeepTags`), 14 dumpů PG (`backup-postgres.ps1 -Keep`).
Díra ve snapshotech, ale bary v okně jsou	Podívej se na `source` barů v `derived/{sym}/bars/`: `ibkr_hist` = doplněno z IBKR historical při dalším startu → **engine v tu dobu neběžel** (vypnuté PC, zastavený kontejner), ne stall řetězu. Stall vypadá obráceně: bary `ibkr` tečou, snapshoty chybí. Do #1055 (8. 9. 2026) se backfill tvářil jako `ibkr` a 2,5 dne vypnutého PC vyvolalo falešný poplach (#1054).
Změna portu TWS	Settings v aplikaci (platí do sekund i bez spojení, #992), **a zároveň** `.env` + `docker compose up -d engine` — hodnota v DB přebíjí `.env`, takže samotná změna `.env` skončí skokem zpět na starý port (engine to hlásí `WARNING: Nastavení připojení ze Settings UI (DB) přebíjí .env`).
Zaseknuté spojení, restart kontejneru nechci	Settings → Stav enginu → **Přepojit IBKR** / **Přepojit tastytrade** (#950) — 1–2 min díra, mimo US RTH. Uložení nastavení beze změny hodnot nepřepojuje.
Docker po samovolné aktualizaci WSL neběží / hlídač Dockeru (#1552)	Windows Update nainstaloval 13. 10. 2026 v 03:10 UTC WSL 3.0.3.0 mimo pauzu, VM WSL se restartovala a Docker Desktop zůstal zastavený 4 h 49 min. Prevence a náprava jsou dvě úlohy Task Scheduleru (kap. 13.4): **„GEXLens update-offhours"** aktualizuje WSL a Docker Desktop jen v pauze Globexu, **„GEXLens docker-watchdog"** každých 5 min restartuje zastavený Docker Desktop. Ručně: `docker desktop status`, `docker desktop restart`, pak `docker ps` (5 kontejnerů gex-*) a `/status` (engine online, connected). **Před záměrným ručním zastavením Dockeru** hlídače vypnout: `Disable-ScheduledTask -TaskName 'GEXLens docker-watchdog'` (potom `Enable-ScheduledTask`); deploy (příkazový řádek) a kompaktaci a aktualizaci (zámek údržby `%LOCALAPPDATA%\GEXLens\docker-maintenance.lock`) pozná sám. Logy obou úloh: `%LOCALAPPDATA%\GEXLens\logs`.
Po restartu Docker Desktop frontend vrací 502 (Settings nejde uložit, watchlist prázdný)	Do #993 nginx držel starou IP služby api; nově se resolvuje za běhu (do ~10 s). U staršího image `docker compose restart frontend`, trvale rebuild frontendu.
Upozornění o výpadku dat o víkendu nebo v denní pauze (`oi_refresh_failed` à 30 min, `spot_fallback`, `strikes_stalled`, `connection_stall`, `disconnect`, `competing_session`, `degraded_start`)	Do #1307 hlídače zavřený trh nerozlišovaly. Od #1307 se při zavřeném trhu podle rozvrhu CME jen loguje (kap. 5 „Provozní upozornění při zavřeném trhu"). Chodí-li i tak, ověř čas v CT (DST) a jestli není svátek. Ten rozvrh nezná: OI, Greeks a striky se ohlásí nejvýš jednou, `feed_crosscheck` „Oba zdroje mlčí … uvnitř US RTH" ale à 15 min.
Alert `greeks_stalled` každý den po settle	Do #959 hlídka běžela i nad expirující řadou, která se po settle přestane kotovat (93 % chybějících Greeks, sekundární řada plná). Nově se po settle dané expirace nehlásí.
`subscription size is too big` v logu /	Strop 25 000 položek `symbol × event` na spojení (#982, kap. 4). Zkontroluj `/status.tasty_budget` — které složky se ořezaly; ad-hoc pohled má rezervu `GEXLENS_TASTY_ADHOC_RESERVE_ENTRIES`. Heal se na tuhle chybu záměrně nespouští.

Situace	Postup
`tasty_budget.size_exceeded` roste	
Reddit 429 v logu news-engine	Limit per IP napříč subreddity (#941) — kolektor jede round robin, jeden subreddit za cyklus à 300 s, retry při 429. Ojedinělé 429 jsou normální; trvalé = zkrátil se `GEXLENS_NEWS_REDDIT_RSS_FEED_DELAY_S` nebo interval.
Signály vznikají mimo obchodní dobu / bez outcomes	Do #968 vznikaly i se zavřeným trhem (21 z 39, žádný neměl outcome — bez barů není z čeho měřit). Generování je vázané na `compute/marketclock.is_market_closed`; expirace a dopočet outcomes běží dál. Latentní past: `_evaluate_outcomes` čte jen 3 dny zpět.
Databáze roste, `pg_database_size` v GB	Největší tabulky vypisuje alert `disk_space` (#773). `feed_comparison` je od #965 zhuštěná (kap. 3); `trades/` a `snapshots/` v parquetu jsou keep-forever záměrně (ADR-0029).
PostgreSQL na 100–200 % CPU, API pomalé, `pg_stat_activity` ukazuje 3 backendy s týmž dotazem	Jeden dotaz + 2 paralelní workery. 23. 9. 2026 (#1257) to byl job news-engine s `NOT IN (SELECT event_id FROM news_classifications)`: s `work_mem` 4 MB se hash ~260 k řádků nevešel, plánovač spadl na nehashovaný SubPlan (`EXPLAIN` bez slova `hashed`) a běh trval 23 min. Dotazy jsou od #1257 `NOT EXISTS`, `compose.yml` dává postgresu `-c work_mem=64MB`; runaway dotaz zruš `select pg_cancel_backend(<pid>)` (jen SELECT). Při „něco je pomalé" nejdřív `docker stats` + `pg_stat_activity`, ne tipovat.
Graf zamrzl, engine hlásí connected	Souběh s mobilem (error 10197) — market data jsou per uživatel. Od #614 přebírá data tastytrade: do 30 s cena, do 3 min celý řetěz; v hlavičce svítí jantarový chip. Zamrzne-li i tak, ověř tasty větev: `docker compose logs engine
CumΔ stojí, zbytek jede	Očekávané chování při fallbacku řetězu (#614) — tasty denní objem v sémantice IBKR nedodává, tak se nevymýšlí. Skončí návratem na IBKR.
V logu není ani řádek `tasty`	Chybí grant v `.env` (`GEXLENS_TASTY_CLIENT_SECRET`, `_REFRESH_TOKEN`) nebo je vypnutý `GEXLENS_TASTY_ENABLED` (od #763; zastaralý `GEXLENS_TASTY_SHADOW`, je-li nastaven, ho přebíjí a engine to hlásí varováním), viz kap. 11.
`Error 200, reqId 5/6` po startu	Známé (#734): `BRFUPDN` a `DJ` nemají celofeedovou pásku. Není to porucha, zprávy tečou z `BRFG` a `DJNL`.
Zvonek `greeks_bs_fallback` / vysoké `greeks_bs_share` ve /status	TWS přestala dodávat model greeks a engine je dopočítává BS modelem z mid (#547) — data tečou dál, jen z jiného modelu (přesně bouře #862: 29 h na BS, vyléčil až restart TWS). `/status.greeks_bs_share` nese podíl per symbol; alert přijde po 15 min epizody nad 20 %. Při plném fallbacku (>80 % ≥ 30 min) zasáhne engine sám (#877, varianta C): 1. pokus vynucená obnova subskripcí (bez díry), 2. pokus reconnect — **výhradně mimo US RTH**; každý pokus hlásí zvonek. Vypnutí zásahů: `GEXLENS_BS_FALLBACK_RECONNECT=false` (alert zůstává). Nepomůže-li ani reconnect, restartuj TWS.
`DXLink ERROR … subscription rate is too high` v logu	Server odmítl dávku subskripcí a odmítnuté symboly by tiše mlčely. Od #863 se engine léčí sám: rozestup dávek se zdvojí (0,25 s → strop 2 s) a po 10 s klidu pošle znovu — od #936 **jen mlčící symboly** (`cache.silent_symbols`, event starší 10 min nebo žádný; typicky jednotky dávek), plný resubscribe jen když mlčí většina (výpadek); plný heal za RTH trhal limit znovu a nekonvergoval (79 healů / 2 h, 5 881 rate limitů / den). Rozestup se po 120 s klidu vrací k základu. `/status.tasty_rate_limited` počítá odmítnutí, `tasty_heals` healy; roste-li trvale i s pomalým tempem, je problém jinde (kapacita účtu — nebo strop položek, viz `size_exceeded`). Transportní `ping_timeout` je 120 s (#916): default 20 s byl kratší než plná subskripce na stropu rozestupu (~44 dávek à 2 s ≈ 90 s) a klient spojení zabil dřív, než doběhla → smyčka reconnectů a fallback #614 bez dat; mrtvé spojení hlídá protokolový KEEPALIVE (60 s) + backoff.

Situace	Postup
`tasty_symbols` výrazně nad plán ADR-0025	`/status.tasty_symbol_breakdown` nese rozpad per účel (`chain` / `extended` / `wide` / `underlying` / `total`) — podle něj se pozná, která složka přerostla (typicky souběh řetězů více expirací během dne; v noci klesá).
Chyby 354/300 s `neznámý kontrakt` v subscription_error_recent	Do 27. 8. anonymní: náhrobky reqId vzorkovaly jen `mktData`, kdežto tick-by-tick hot zóny se registruje pod `AllLast` / `BidAsk`. Nově nesou popisek i `[tickType]` — podle něj poznáš rotaci hot zóny (ATM se pohnul) od výpadku kotací.
Stínové CumΔ z TimeAndSale (#615 fáze 3)	Paralelní měřicí řada z dxFeed `aggressorSide` — živé CumΔ ani detektory se NEmění. Minutové řádky v `derived/{sym}/cumdelta_dx/` (od ADR-0032 doplňku 4. 9. celý sbíraný řetěz bez zón; `flow_hot` zůstává 0), `/status.tasty_dx_shadow` nese denní `unknown_side_share`, `volume` a od #1070 (v1.6) `seeded_from_ts` — po restartu uprostřed seance se kumulativ **navazuje z posledního řádku partice** (do 8. 9. 2026 začínal od nuly a den měl tolik řetězů, kolikrát engine startoval; řádky téže minuty se upsertují). **Štítek řádku** N = stav k N: od v6 (#1586) se minuta uzavírá na hranici podle nástěnných hodin, obsah je [N−1, N) jako u živého flow a hranice bez řádku hlásí warning „Stín CumΔ: … min bez řádku“; pod v5 perioda 60 s + doba zápisu driftuje (obsah posunutý o 0–60 s, co 2–5 h chybí jeden štítek — doplní se posledním kumulativem, ADR-0057 bod 7). **Živá trade větev** (#615 fáze 3, v1.4): `CumDeltaTracker.add_dx_trade` přijímá tytéž tisky; pod mechanikou v5 (zdroj `midpoint`) jen měří pokrytí (`/status.cumdelta_coverage` : `printed_share` , `structured_volume` , `fallback_volume`); od v6 (#1403, ADR-0057, zdroj `aggressor`) dávají znaménko toku i čistého objemu jen tisky se stranou a bar větev znaménko nedává (objem bez tisku, tisk bez strany i bez delty jdou mimo tok). Zdroj určuje mechanika, klíč `GEXLENS_CUMDELTA_SOURCE` je zrušen (#1460), návrat jen pinem image; řada `flow/` nese sloupec `source` . Spread legy se nerozlišují (3. 9. 2026, ADR-0027 doplněk): CME příznak `spreadLeg` pro FOP nenese, řada „outright“ a `spread_volume_share` zrušeny. Vypínání: `GEXLENS_TASTY_DX_CUMDELTA=false` .
Kandidát T9 „strop nad hlavou“ (#577, fáze 1)	Tichý sběr výskytů do PG `setup_probes` : vstup do tlumící zóny zespodu (t9_ceiling, hypotetický LONG hrana→střed) a výpad dolů (t9_exit, SHORT o šířku zóny); kotvy z geometrie pásma #575 (žádné body/ATR — poučení #434), akceptace 5 min, vyhodnocení **po cestě ceny** týmž `walk_setup_path` jako živé setupy (#1345: díra z partic, timeout za close baru končícího v settle expirace sondy). Rozhoduje čistá funkce `compute/setups.detect_damping_ceiling` (mimo `detect_all` , regresní zámek `test_setups_regression.py`); tutéž spouští `python scripts/backtest_setups.py --probes --data <kořen dat>` nad historií (bary + `gexprofile/` , bez DB) — report per instrument, šablona a směr. Do `setups` , alertů ani track recordu nejde nic; fáze 2 při ≥ 30 výskytech na instrument.
Ad-hoc pohled (#521 C)	Tabulka `adhoc_view` (PG) je most UI→engine; frontend prodlužuje `requested_ts` à 1 min, engine bez prodloužení pohled po ~3 min uklízí a subskripce vypadnou diffem. Data jdou výhradně z tastytrade (žádné IBKR linky); `/status.tasty_adhoc` nese aktivní produkty, rozpad subskripcí má složku `adhoc` . Od v1.4 pohled **ustoupí produktu s plnou pipeline** (`refresh` ho uklidí i s řádkem `adhoc_view`) a UI nepinguje, dokud nezná watchlist — 4. 9. vznikl ad-hoc NQ po restartu enginu (pipelines ještě prázdné) a přežíval díky pingům při každém otevření stránky. Partice ad-hoc symbolu uklízí běžná 14denní retence.
Upozornění na zprávy (#1291, ADR-0043)	Dva joby v news-engine ve smyčce reakcí (à `GEXLENS_NEWS_REACTION_INTERVAL_S` , po klasifikaci), oba do kanálu `alerts` . **`AnomalyJob`** → `news_anomaly` (shluk s významnou zprávou × mimořádná výchylka, per ES/NQ): nic nezapisuje, `news_reactions` nečte; bary čte z `derived/{ES,NQ}/bars` , zprávy z `news_events` ; významnost ze sdílené čisté funkce `gexlens_engine/compute/news_significance.py` (#1293, ADR-0045) nad `kind` , `importance` , `category` — FF impact podle měny i strop nekurátorovaných sociálních sítí nese už importance z

Situace	Postup
	klasifikátoru v2, `raw.impact` ani `raw.curated` job nečte. Konstanty v `gexlens_news/clusters.py` a `reactions.py` (shluk 2 min, okno 5 min, p97 denní doby ± 30 min z 20 seancí a p97 po normalizaci σ poslední hodiny, cooldown 15 min, lookback 30 min), nová env proměnná není. Log: `Baseline výchylek ES (seance …): N seancí, … vzorků` 1× za seanci a symbol (studený běh ~3 s na hostiteli + ~20 ms/partici v kontejneru), `Reakce na zprávy ES: N shluků při otevřeném trhu nejde změřit` (chybí bary nebo baseline — v prvních ~45 min po otevření normální), `Reakce na zprávy: N upozornění (…)`. Dedup a watermark jen v paměti: po restartu se shluky hotové před startem nehlásí. **`PreopenJob`** → `news_preopen` (zprávy za víkend, per ES/NQ) 4 h a 15 min před otevřením Globexu (neděle 17:00 CT; léto i zima 20:00 a 23:45 Praha, týden nesouladu DST 19:00 a 22:45); mimo tyto etapy nic nečte. Poslední bar (začátek zavření, close) z `derived/{sym}/bars`, úrovně z `derived/{sym}/{expirace}/levels/{den}.parquet` nejbližší expirace ≥ příští seance (pondělní 0DTE). Stav etap a ohlášených zpráv v PG `settings` klíč `news_preopen_state` (píše jen news-engine, API ho zapsat nedovolí; zapisuje se před publikací → restart v neděli večer souhrn nezopakuje, riziko je ztráta, ne duplicita). Log: `Předobchodní upozornění (otevření …, etapy main): N upozornění`, bez barů WARNING `… zavření podle rozvrhu`. Do výčtu a sklonu jdou jen zásadní zprávy (`news_significance.is_key` : kalendář nebo zpráva s importance 3, ADR-0045), ostatní významné a šum jen počtem; bez zásadní zprávy nic. Svátek uprostřed týdne upozornění nemá a svátek s celodenním zavřením v pondělí (Vánoce/Nový rok v pondělí, poprvé 12/2028) dostane souhrn k nedělnímu otevření, které nenastane (rozvrh svátky nezná, ADR-0023; ADR-0043 bod 5). Oba payloady nesou navíc `ts_event` (začátek shluku / otevření, ISO UTC) a `event_ids` (významné zprávy) pro proklik (#1290). Přeměření včetně simulace víkendů: `uv run python scripts/measure_news_anomaly.py --days 14 --out … [--preopen-weeks 8]` (replay skutečných jobů, prod DB jen čtení; URL z `GEXLENS_PG_PASSWORD`, port 55432)
Klasifikace zpráv v2 a reklasifikace historie (#1293, #1305, ADR-0045)	**Klasifikátor** `gexlens_news/classifier.py` (čisté funkce, golden test `news-engine/tests/test_classifier.py` — 45 typických makro titulků, vzory falešných shod): importance a kategorie jen z titulku, 3 jen pro události, 2 téma jako předmět, zmínka/názor 1; strop podle feedu `feed_of(source, raw)` — Bluesky bez `raw.curated` a Reddit 1, agregátory (`rss_news` s `raw.feed` na Yahoo — headline feedy `^GSPC` a `^IXIC`, do 23. 9. 2026 `rssindex` — nebo MarketWatch, `rss_user`) o stupeň níž; kalendář `scheduled_importance` (USD/All High 3, Medium 2, EUR/GBP/JPY rozhodnutí CB s High 2, jinak 1 — tutéž funkci volá collector i `ffhistory`); `fed_rss` FED/3 statement/projekce/minutes, jinak FED/2 (i svědectví v Kongresu); `ecb` 2 jen rozhodnutí o sazbách (tisková zpráva „Monetary policy decisions“, statement a tisková konference jako v kalendáři, ADR-0045 bod 3), jinak 1, kategorie z titulku jako u zpráv o ECB odjinud (FED → OTHER, FED je americký); `whitehouse` pravidla v2 bez stropu (ADR-0059 bod 5, #1515); téma cel (`TARIFF_TOPIC`, 2 a téma karty Cla) i z úředních formulací aktů bez slova tariff — uložená `additional duties` (`imposing …`, `subject to the …`, `… on`, `… imposed`), `from importation into the United States`, `adjusting imports of` ; úzké vzory, „assume additional duties“ ani „FDA bars importation“ clo nejsou (#1518; na událost 3 je nezvedá — reakce n = 2 čisté shluky nebyla výrazně nad mediánem hodiny, `range_5` 0,5–1,3×); `bls` podle feedu releasu (`feed_of` → `bls:{release}`): CPI a PPI MACRO_INFLATION/3, zaměstnanost MACRO_LABOR/3, JOLTS a ECI MACRO_LABOR/2, směr 0 — směr releasu nese řádek kalendáře konvencí řady (#1516); `bea` podle řady v titulku (`BEA_RELEASE_CLASSES`): „GDP (…“ MACRO_GROWTH/3, „Personal Income and Outlays“ MACRO_INFLATION/3, ostatní vydání 1 s kategorií z titulku — roční i obchodní bilance a běžný účet, které kalendář FF hodnotí Low impact (vlastník 11. 10.), směr 0 jako u BLS (#1517). Job čte `raw.feed/link/curated/impact/impactName` (`classification_job.RAW_COLUMNS`) a do WS dávky přidá `significance`. **Kadence (#1496, E-6.28a):** nové zprávy klasifikuje smyčka à 1 s (`classification_loop`, `RuleClassificationJob.run_new`)

Situace	Postup
	dotazem `id > watermark` přes PK bez anti-joinu — kategorie a importance do ~1 s od zápisu ze všech cest (collectory news-enginu i IBKR pásky z enginu); watermark po startu = `max(id)`. Pojistka `run` (anti-join, ~0,5 s na produkci) zůstává v `reaction_loop` à `GEXLENS_NEWS_REACTION_INTERVAL_S` a dojede řádky commitnuté s nižším `id` po posunu watermarku i vše před startem. Obě cesty zapisují pod zámkem a event s jakoukoli klasifikací přeskočí, souběh tedy nedá druhou pravidlovou verzi. Push `news` nese skutečný `ts_ingested`. Log à 10 min: `Rychlá klasifikace: N eventů, od příjmu medián … s, p90 … s, max … s`; chyba smyčky → `Rychlá klasifikace selhala` a další pokus za 30 s. Před #1496 byl medián od příjmu 203 s (p90 407 s). **Slovník režimu** (`ACTORS`, předseda Fedu ve `FED_TOPIC`, úřední formulace cel v `TARIFF_TOPIC`) stárne — po změně režimu upravit a doplnit golden test. **Významnost** jediná čistá funkce `gexlens_engine/compute/news_significance.py` (stupeň 0–3 / None, `is_key` = stupeň ≤ 2); volají ji shluky, předobchodní souhrn, WS dávky a API. **Kontaminace K1** (`reactions.contaminates`): okno kazí jen jiný event s importance ≥ 2 a jinou kategorií. Příznak `cont_*` zapisuje `ReactionJob` při měření z eventů, které už jsou v DB. Event zapsaný později (pozdní příjem Finnhub a IBKR, první stažení nového zdroje) dožene **hodinový přepočet reakcí posledních 7 dní** (`gexlens_news/contamination.py`, `ContaminationJob` v `reaction_loop` po reakcích, #1522). Používá tytéž funkce jako fáze `contamination` skriptu reklasifikace a přepočítává úplně, takže chytí i změnu kategorie (True → False). Log: `Kontaminace K1 (#1522): N řádků reakcí od …, změněno M (… s)`; na produkci ~27 tis. řádků za ~2 s. Starší historii (backfill) přepočte skript `--phase contamination`. **Jedno měření = jeden vzorek** (`model_stats`, #1293): vzorky téhož bucketu se stejným `ts_event` (řádky FF jednoho releasu — CPI m/m, Core CPI m/m, y/y) se v `news_model_stats` slučují do jednoho (směr = převaha, remíza se neposuzuje); job čte reakce seřazené podle času eventu. **Váhy** `news_weights` berou outcomes z klouzavého okna 90 dní podle **času zprávy** (`news_events.ts_event`), ne podle `computed_at` — reklasifikace jednorázově vyhodnotí i staré predikce, jejichž okno přestalo být kontaminované. **Reklasifikace historie** `scripts/reclassify_news_rules.py` (jen `sentiment_source = rule`; LLM, ruční korekce a stín ngram beze změny): fáze `classify` = nová verze v `news_classifications` (max+1 přes všechny zdroje, směr/síla z poslední pravidlové verze, `created_at` = RUN_AT běhu) + UPDATE `news_events.category/importance`, jen při rozdílu; **kurátoři Bluesky** pro historii = autoři (`raw.did`) s aspoň jedním postem s příznakem `raw.curated` (collector ho zapisuje od #1291, 25. 9. 2026) ∪ aktuální seznam `news_bluesky_authors` v `settings` + `GEXLENS_NEWS_BLUESKY_CURATED_AUTHORS` (handly přes veřejné resolveHandle jako collector — potřebuje síť; report vypíše počty a nepřeložené handly); fáze `contamination` = `news_reactions.cont_1/5/15/60` pravidlem K1 (deferred od prvního baru z `derived/{ES,NQ}/bars`, jen sloupec `ts_min`, ~1 400 partic). **Postup na produkci** (mimo RTH, v pauze Globexu nebo o víkendu): 1) `pwsh scripts/backup-postgres.ps1`; 2) nový image news-engine; 3) `docker compose run --rm news-engine python scripts/reclassify_news_rules.py --dry-run --out /app/data/reports/reclass-dryrun.txt` (PG spojení `read only`, ~4 min; matice importance, feedy, kurátoři, změny kategorií, objemy za 28 dní, důvody importance 3 s ukázkami, kontaminace per okno — dry-run 26. 9. 2026 **před opravami z recenze**: 52 500 ze 184 397 eventů, w5 kontaminace 48 % → 24 %; před ostrým během přeměřit); 4) `docker compose stop news-engine` a ostrý běh bez `--dry-run` (dávky `--batch` 5 000 v transakci; report vypíše **RUN_AT** pro rollback); 5) druhý `--dry-run` musí mít 0 změn v obou fázích; 6) `docker compose up -d news-engine` — první smyčka přepočte `news_model_stats` a gate (log přepočtu model stats; offline simulace finálním kódem: živé gate v0 se zavřou, otevřené zůstanou jen odložené buckety GEOPOLITICS, ADR-0045 Důsledky), v noci `news_weights`; 7) `docker compose exec news-engine python -m gexlens_news recompute-sentindex --from <nejstarší partice derived/sentiment>`; 8) vzorek H1 (#1264) začíná znovu, drift hlídka může ohlásit posun. **Pořadí nasazení**: api a news-engine (s reklasifikací) nejpozději současně s frontendem — frontend bez pole `significance` z API ukáže s filtrem Významné jen připnuté

Situace	Postup
	zprávy upozornění. **Rollback**: `… reclassify_news_rules.py --rollback <RUN_AT>` (nová pravidlová verze s hodnotami před během, obnoví denormalizaci; pozdější LLM/ruční verze přeskočí; opakování nic nemění), pak `--phase contamination` a kroky 6–7. Obě fáze jdou spustit i zvlášť (`--phase classify
Reakce na releasy a upozornění před releasem (#1296, ADR-0044)	Dva joby v news-engine. **`ReleaseMovesJob`** ve smyčce reakcí (po předobchodním upozornění): shluky USD releasů (`news_events` scheduled, FF `raw.impact` / `raw.impactName` High/Medium, headline podle `releases.SERIES_RANK` , jen rodiny CPI, NFP, FOMC, PPI, PCE, Retail Sales, ISM Services) změří po uplynutí 60 min + 2 min zápisu barů, **jen když headline shluku má actual** (release bez actual se nekonal — fantom po přesunu releasu, #1298 — nebo ho FF ještě nevyplnil; po 24 h WARNING `Release … nemá ani po 24 h actual – neměří se` , jednou za proces): výchylka 15 min, výnos 15/60 min, `vol_ref` z `gammacliff.session_ranges` (od seance − 45 dní, cache do změny dne), od registrace hypotéz (1. 10. 2026) i medián denní doby pro M1 (20 seancí barů, ±30 min, ≥ 200 vzorků — zmrazené konstanty `M1_*` registru, ne konstanty news_anomaly; ~~2–3 s na release). Živě 14 dní zpět; neúplné měření se opakuje nejvýš 1× za hodinu do 3 dní, překvapení se doplňuje do 7 dní.~~ **~~Prázdná tabulka~~** ~~`release_moves` se dopočítá sama od 28. 7. 2024~~ (1 min v kontejneru, log `release_moves je prázdná – dopočítávám historii…` ; zkouší se každý cyklus, dokud je prázdná, takže selhaný první běh se zopakuje). Ručně / po opravě kalendáře (#1298) nebo barů (#1299, #1300): `docker compose exec news-engine python -m gexlens_news backfill-release-moves [--from YYYY-MM-DD]` (idempotentní; přeměří jen shluky **před registrací hypotéz** 1. 10. 2026, živé jen s `--include-live` — WARNING v logu a vyhodnocený úsek hypotéz se ani tak nepřepíše; návratový kód 1 při chybě některého shluku). Log: `Reakce na releasy (od …): shluků N, změřeno …, bez výchylky …, bez vol_ref …, bez actual …, chyb …` , `Hypotézy releasů přepočteny: H1 ES k/n testing, …` a WARNING `Hypotéza … N releasů ve vyhodnoceném úseku by dnes dalo jiný výsledek – úsek je zmrazený` (oprava dat po rozhodnutí; nic nedělat, jen vědět). **`ReleasePreviewJob`** ve vlastní smyčce à 60 s → `release_preview` (kanál `alerts`) 60 a 15 min před shlukem s aspoň jedním High a rodinou s upozorněním, ES a NQ zvlášť: cena = poslední bar do 15 min, úrovně z `derived/{sym}/{expirace}/levels/{UTC den}.parquet` nejbližší expirace ≥ seance releasu, statistika velikosti z `release_moves` (cluster_ts < T, medián a p75 násobků, ≥ 8 releasů), stav H1/M1 z `release_hypotheses` (dovětek M1 jen u rodin M1, ne u slabších řad). Stav etap v `release_previews` zapsaný před publikací (restart nic nezopakuje; po pozdním startu odejde jen T−15). Log: `Upozornění před releasem CPI … (T60): 2 upozornění` ; WARNING `málo historie (n = 0) – chybí backfill release_moves?` . Chybějící cena, úrovně či volatilita jsou vidět v textu, upozornění neblokují. Registr hypotéz a kritéria jsou konstanty v `gexlens_news/release_hypotheses.py` včetně vlastních kopií vstupů (řady H1, rodiny a baseline M1) — změna = nová hypotéza s novým ID a datem (ADR-0044), ne úprava. **Změnu času releasu kalendář nepropíše** (#1298): přesun v rámci dne nechá starý čas (upozornění i měření podle něj), přesun na jiný den nechá fantomový řádek → falešné upozornění v původní čas (měření ho nezapočte). Výzkum a jeho přegenerování: `uv run python scripts/build_release_reactions.py --out-dir …` a `scripts/measure_release_predictability.py --out-dir …` (prod DB jen čtení)
Zdroje zpráv: registr + audit (#578)	Tabulka `news_sources` (tier core/extra/test, očekávaný denní objem, enabled, notes; seed nepřepisuje ruční editace). Audit: `GET /news/sources` — realita per zdroj včetně neregistrovaných. Nové zdroje 27. 8.: **bluesky** (Jetstream firehose bez klíčů, klientský filtr cashtags+makro+kurátoři přes `GEXLENS_NEWS_BLUESKY_CURATED_AUTHORS` , strop 30/min, vypínání `GEXLENS_NEWS_BLUESKY_ENABLED=false`) a **reddit_rss** (nativní RSS wsb+stocks hot à 300 s, `GEXLENS_NEWS_REDDIT_RSS_ENABLED=false` vypíná). **Bluesky po výpadku spojení (#1509):** reconnect posílá `cursor` = `time_us` poslední přijaté zprávy (µs unixového času, který zprávě přidělí Jetstream při příjmu). Jetstream přehraje zprávy od cursoru včetně a přejde na živý

Situace	Postup
	proud. Hraniční duplicitu (`time_us` ≤ cursor) zahodí stream. Přehrání sahá jen do retence instance; legacy Jetstream (host `jetstream2.us-east`) má výchozí `event-ttl` 24 h, veřejné instance hodnotu neuvádějí. Backoff 2 → 60 s roste jen opakovaným selháním připojení. Po pádu spojení, které doručilo data, jde první reconnect za 2 s. Strop 30/min se počítá v minutách podle `time_us` , ne podle hodin příjmu: přehrání se rozloží do původních minut, takže ho strop ořízne jen tam, kde by ořízl živý proud téže minuty (minuta s víc než 30 posty, které prošly filtrem, se ořízne i při přehrání). Log `Bluesky Jetstream připojen (…) s cursorem – výpadek N s, přehrává se` , po dohnání živého proudu `Bluesky: přehrání dohnáno – výpadek N s, přehráno N zpráv, filtrem prošlo M, zapsáno W, nad strop D` (M = prošlo filtrem i stropem, D = zahozeno stropem); při pádu nebo stopu uprostřed přehrání `přehrání nedokončeno (pád nebo stop)` s dosavadními počty. Cursor je jen v paměti procesu: restart news-enginu začíná živým proudem a posty z doby restartu chybí. **Zdroje tier 1 (ADR-0059 bod 5, #1515):** `fed_rss` (feedy `press_monetary` , `speeches` a od E-6.25a `testimony`), `ecb` (tiskové zprávy ECB), `whitehouse` (zprávy a prezidentské akty Bílého domu) a od E-6.25c `bls` (feedy releasů `empsit` , `cpi` , `ppi` , `jolts` , `eci` , Atom; jen s `GEXLENS_NEWS_UA_CONTACT` , #1516) a od E-6.25d `bea` (`www.bea.gov/news/rss` ; feed drží jen nejnovější vydání každé řady, první stažení tedy historii releasů nedá — `docs/research/1517-bea-uplnost-feedu.md` ; `pubDate` = čas releasu, `ts_source = item` ; `apps.bea.gov` zakazuje robots.txt, #1517), vše à 60 s s conditional GET, vypínání v registru. **Čas zveřejnění** (`ts_event`) zdroje tier 1 má původ v `raw.ts_source` : `item` = čas položky v UTC, je-li věrohodný (rok ≥ 2000 a ne po stažení), jinak `fetch` = čas prvního stažení položky (stabilní po dobu běhu procesu). **BLS** čas položky nebere nikdy (předbíhá release o 39 min): `calendar` = čas řádku kalendáře FF (`source = forexfactory`) téhož dne v ET s **přesným** titulkem podle mapování `collectors/bls.py` (`cpi` → `USD CPI m/m` , `ppi` → `USD PPI m/m` , `empsit` → `USD Non-Farm Employment Change` , `jolts` → `USD JOLTS Job Openings` , `eci` → `USD Employment Cost Index q/q` ; ADP ani Core CPI se nespárují); den releasu z času položky v ET; bez shody nebo bez věrohodného času položky `fetch` se stejnými pojistkami (den stažení zálohou není — stará položka bez času by v den releasu dostala dnešní čas). Čas z kalendáře se pro položku dotazuje jednou za běh a pak platí do konce běhu. Položka bez věrohodného času, kterou feed nesl už v prvním úspěšném stažení po startu, se nezapíše (WARNING `položka bez věrohodného času … se nezapisuje` , jednou za běh; testimony nese 3 položky s rokem 1899). Jednou přidělený čas stažení platí do konce běhu. Položku zdroje tier 1, kterou zdroj už zapsal (`source` , `source_uid`), `NewsWriter` znovu nezapíše, ani když se jí po restartu změní čas. Ostatní RSS beze změny (`published or fetched_at`). Při prvním nasazení se zapíše historie feedů s jejich časy (karta Breaking news i shluky upozornění vybírají podle `ts_event` , takže se jako čerstvá neukáže). Log startu vypisuje běžící collectory (`news-engine běží: N collectorů: …`), ne klíče v konfiguraci.
Tier obsahu zpráv (ADR-0059, #1486)	Sloupec `news_events.content_tier` : 1 oficiální zdroj (`fed_rss` , `ecb` , `whitehouse` , `bls` , `bea`), 2 headline feed (Alpaca s autorem Benzinga Newsdesk, Bluesky s kurátorem), 3 článek (ostatní Alpaca, `rss_news` , `finnhub` , `ibkr_*` , `rss_user`), NULL mimo tiery (kalendář, Reddit, Bluesky bez kurátora). Není to role zdroje z registru (`news_sources.tier`). Zapisuje se při ingestu z čisté funkce `gexlens_engine/compute/news_tier.py` (news-engine `NewsWriter` i engine `newsticks`); sloupec přidává při startu `ensure_sentiment_schema` (news-engine, API i engine), takže na pořadí nasazení nezáleží; zprávy zapsané starým kódem po přidání sloupce mají NULL — **ostrý backfill až po nasazení news-engine i enginu**. Historie: z hostitele `uv run python scripts/backfill_content_tier.py` (výchozí dry-run jen čtením, počty zdroj × tier; `--apply` zapíše jen rozdílné hodnoty, druhý běh 0 změn; URL `--db` , jinak `GEXLENS_NEWS_DATABASE_URL` / `GEXLENS_HOST_DATABASE_URL` / `GEXLENS_PG_PASSWORD` ; před `--apply` záloha PG). Kurátor Bluesky před prvním postem s `raw.curated` podle ADR-0045 bod 8

Situace	Postup
	(`GEXLENS_NEWS_BLUESKY_CURATED_AUTHORS` + seznam v `settings`, handly přes síť). Nový zdroj musí v `news_tier.py` dostat výslovné rozhodnutí (test pokrytí registru `NEWS_SOURCE_SEED`).
Zdroje u duplicit zpráv (ADR-0059 bod 3, #1489)	Tabulka `news_event_sources`: kopie téže zprávy z **jiného** zdroje, než má první doručení v `news_events` — `event_id` (první doručení), `source`, `source_uid`, `content_tier` kopie, `published_at` (`ts_event` kopie), `fetched_at` (kdy kopie přišla, od té chvíle je viditelná). PK `(event_id, source)`: zápis je idempotentní a drží nejdřívější kopii zdroje. Zapisují ji všechny cesty přes jednu funkci `record_news_copy` (`storage/sentiment.py`): rolling dedup news-enginu (exaktní i Jaccard, kopie se přiřadí podle `dedup_hash` prvního výskytu v okně, i po restartu z DB), konflikt na unikátním `dedup_hash` v `NewsWriter` (týž titulek téhož dne mimo 6h okno) a engine `newsticks` (IBKR páska, kterou rolling dedup nevidí). Kopie téhož zdroje je duplicita bez záznamu; payload kopie se neukládá; `ts_event` prvního doručení se nepřepisuje. Efektivní tier a nejdřívější publikace se počítají při čtení (`compute/news_tier.py`: `effective_tier`, `earliest_published`; viditelnost od `fetched_at`). Kopie bez prvního doručení v DB (jeho zápis selhal) se nezapíše — WARNING `Kopie zprávy bez prvního doručení v DB`. Engine loguje `IBKR headlines: N nových, M kopií zprávy z jiného zdroje`. Tabulku zakládá `ensure_sentiment_schema` (aditivně, **před nasazením záloha PG**); historie kopií neexistuje (dřív se zahazovaly). Objem hrubě ~140 MB za rok (ADR-0059), skutečnost změří E-6.29. `raw.merged_sources` z ADR-0016 se nepoužívá.
Výběr karty breaking news (ADR-0059 bod 4, #1491)	Čisté funkce `news-engine/src/gexlens_news/breaking.py`, počítané při čtení, nic se neukládá: `is_breaking` = efektivní tier 1–3 v čase *t* (`compute/news_tier.effective_tier`) ∧ `is_significant` (ADR-0045) ∧ článek (tier 3) jen čerstvý, příjem ≤ 60 min po publikaci (`ARTICLE_MAX_INGEST_DELAY`, varianta C, revize 10. 10.); `is_confirmed` = efektivní tier 1–2 — čerstvá zpráva tier 3 jde na kartu hned se štítkem „článek, zatím nepotvrzeno“, který zmizí s první viditelnou kopií tier 1–2 (kopie = táž zpráva, jiný titulek o téže události potvrdí až shluk); `card_group` mapuje `category` (+ zdroj `ecb`) na skupinu `macro` / `central_banks` / `geopolitics` / `companies` / `other`; `theme` = první shoda slovníku `THEMES` (jen vzory klasifikátoru, aktéři režimu z `REGIME_ACTORS`) v předmětu titulku, pak v celém titulku. Výsledky firem na kartu nejdou ani u mega caps (předregistrované měření nesplnilo kritérium, `docs/research/1491-mega-caps-earnings.md`). `card_impact` + `fixed_at` skládají dopad karty z `compute_reactions` a `measure_excursion`. Volá je `GET /news/breaking` (E-6.28b, kap. API) a skripty; UI karty staví E-6.28c. Šum karty: z hostitele `uv run python scripts/measure_breaking_noise.py [--days 30] [--as-of …]` (jen čtení PG; denně potvrzené × nepotvrzené, později potvrzené, náskok karty, zdroje a varianty zpřísnění; report `docs/research/1491-sum-karty-breaking.md`, přeměření #1492). Měření mega caps: `uv run python scripts/measure_megacap_earnings.py`. URL obou `--db`, jinak `GEXLENS_NEWS_DATABASE_URL` / `GEXLENS_HOST_DATABASE_URL` / `GEXLENS_PG_PASSWORD`.
Doplnění minutové fáze reakcí (#1494)	Běžná minutová fronta `ReactionJob` měří event jednou (pending = event bez jakéhokoli řádku `news_reactions`, ochrana #655). Do #1494 tak řádek bez minutové fáze (`computed_at_min` NULL) zůstal navždy. Vznikalo to dvěma cestami: (1) denní fronta (eventy starší 16 dní) zapsala řádek eventu, který minutová fronta ještě neměřila (backfill zpráv 17. 8., 27 968 párů doplněno 10. 10.); (2) symbol bez barů v T+60 min (výpadek feedu, restart enginu), díru v barech pak doplní `ibkr_hist`. **Od #1494 (A + D) obě cesty zavírá job sám** jednou metodou `ReactionJob.complete_minute` (doměří jen symboly bez minutové fáze, nic nepřepisuje): **D** — `_run_daily` doměří minutovou fázi eventům, které zapisuje (pojistka i pro vše starší, cesta 2 po 16 dnech); **A** — `run` jednou za hodinu (`MINUTE_RETRY_EVERY`) vezme eventy posledních 3 dní s neúplnou minutovou fází (`MINUTE_RETRY_LOOKBACK`, limit `MINUTE_RETRY_LIMIT` 200, nejnovější první, až po nových eventech), takže výpadek symbolu se doplní do hodiny od backfillu barů; pár bez barů se tak zkusí nejvýš ~72× (past #655), pak ho převezme D. Doměřování drží partice jen po dobu průchodu (`bars.CachedBars`), filtr na eventy jde už do agregace nad

Situace	Postup
	`news_reactions` ; chyba jednoho páru se zaloguje a průchod pokračuje. Log: `Minutová fáze doměřena (#1494, výpadek symbolu
Hlídka RSS feedů (#1451)	`RssCollector` drží stav selhání **per feed** (`feed_failures` : od kdy, poslední chyba — HTTP status, síť, nečitelné XML); 304 i úspěšné parsování ho mažou. Výjimku pro runner (degradace zdroje, backoff) hází jen selhání **všech** stažených feedů — mrtvý feed vedle 304 ostatních už nezpomaluje zdravé feedy. `FeedWatch` (`gexlens_news/feed_watch.py` , smyčka à 60 s) pošle `news_feed_error` , když feed selhává bez přerušení ≥ 1 h, a `news_feed_recovered` po prvním úspěchu; stav „ohlášeno" je v paměti, po restartu se trvající výpadek ohlásí znovu po hodině. Bez brány zavřeného trhu: RSS publikuje i o víkendu a hlídka reaguje na chybu stažení, ne na ticho. Telegram téma „Feed zpráv nefunguje" v kategorii zpráv (výchozí zapnuto, respektuje tiché hodiny; zvonek bez otevřeného tabu alert neudrží). Zpráva začíná krátkým jménem feedu (`feed_label`), protože Telegram slučuje alerty se shodnými prvními 80 znaky. Spouštěč: Yahoo `rssindex` vracel od 23. 9. 2026 HTTP 404 dva týdny bez upozornění (−94 % `rss_news`); nahrazen headline feedy `^GSPC` a `^IXIC` (`config.NEWS_RSS_URLS`).
Walk-forward parametrů setupů (#794 fáze 3, ADR-0034)	`pwsh scripts/walkforward-nightly.ps1` — registrováno v Task Scheduleru jako úloha **„GEXLens walk-forward"** (po–pá 23:30 místního času — spouštěče všech úloh GEXLens se zakládají přes `scripts/lib/LocalTrigger.ps1` , jinak by je Windows ukotvil na UTC a po konci letního času běžely o hodinu dřív, #1278; zmeškaný start se dožene, `scripts/register-walkforward-task.ps1` ; log `data/reports/walkforward-nightly.log` ; bez běžícího `gex-postgres-1` se tiše přeskočí, stejně tak při otevřeném trhu podle `compute/marketclock.is_market_closed` — replay je ~12–15 min plného CPU vedle enginu; v týdnech posunu letního času je 23:30 = 17:30 CT, po–čt se proto přeskočí a report vznikne jen v pátek, #1568; vědomý ruční běh za otevřeného trhu `-Force`) → `data/reports/walkforward-<datum>.md` : replay archivu produkčním detektorem pro kandidáty z `configs/walkforward_grid.json` (baseline = platná verze `setup_params`), IS 20 / OOS 5 seancí, zábrany (≥ 20 OOS seancí, ≥ 50 % foldů, Ø Δ ≥ 0,1 R/den, Bonferroni p). **Nic nezapisuje** — případný návrh je tělo pro `POST /setups/params` , odesílá člověk. **Zdroj CumΔ replaye podle mechaniky** (#1590, ADR-0057 bod 7; `--mechanics 5
Přepočet výsledků setupů z barů (#1320)	Z hostitele: `uv run python scripts/recompute_setup_outcomes.py --data data [--symbols ES,NQ] [--ids 1003,1004]` (URL `--db` , jinak `GEXLENS_HOST_DATABASE_URL` , jinak `GEXLENS_DATABASE_URL` ; z hostitele `postgresql+psycopg://gexlens:<heslo>@127.0.0.1:55432/gexlens` , URL z `.env` míří do compose sítě). Přehraje uzavřené setupy aktuální mechaniky týmž `walk_setup_path` jako engine nad `derived/{sym}/bars` , od baru vstupu (`context.entry_bar_ts` ; u starších řádků bar N−2 … N+3 s close = entry, jinak setup nad spotem za minutou N−1), a zapíše report `data/reports/recompute-setup-outcomes-<čas>.md` + `.csv` : verdikty **opravit** (jiný status nebo R, Δ R a Δ USD na 1 kontrakt), **opravit čas** (výsledek sedí, jen `closed_ts`), beze změny, **vstup mimo bary**, **neověřitelný** a po settle (`born_after_settle` , přeskočí). Vstup mimo bary = entry neleží v rozsahu barů minuty vzniku ani není close baru vstupu: setup vznikl nad zamrzlým spotem (důvod ukáže bar, jehož close entry je, a jeho stáří; ES 1004 3. 9.), nad mid kotací nebo nad jiným kontraktem — nezapisuje se, návrh je v reportu jen pro rozhodnutí uživatele. Neověřitelný = díra v barech (návrh přes díru jen v reportu), v rozhodující cestě bar z rekonstrukce tasty `tasty_candle` (jiný zdroj než IBKR; živý `ibkr` i `ibkr_hist` jsou týž zdroj), skok ceny na jiný kontrakt (#1232), nejednoznačný bar vstupu, na kterém výsledek závisí, nebo expirace ještě běží a bary úroveň nezasáhly. Kvartální datum (#1366): settle podle `context.trading_class` , u řádků před #1366 dovozený z tickeru (kořenový ticker = týdenní série 16:00 ET, pinovaný expirující kontrakt = SOQ) — důvod řádku to uvede. Výchozí je **dry-run**. `--apply` zapíše po napsání `ano` verdikty opravit a opravit čas přes `SetupsRepository.correct_outcome` : přepíše `status` , `closed_ts` , `outcome_r` a MFE/MAE ze stejné

Situace	Postup
	cesty ceny, původní hodnoty uloží do `context.outcome_correction` (`old_status` , `old_outcome_r` , `old_closed_ts` , `old_mfe` , `old_mae` , `corrected_ts` , `reason` ; opakovaná oprava drží první původní hodnoty), zbytek kontextu nechá, nic nemaže; opakovaný běh už nic nezmění. `--apply` přepočítá znovu: interaktivně zapíše to, co ukazuje report téhož běhu; neinteraktivně (`--yes`) jen spolu s `--approved <CSV schváleného dry-runu>` a jen při shodě zapisovaných řádků — mezitím doplněné bary by jinak zapsaly něco, co nikdo neviděl (rozdíly vypíše, návratový kód 1). Před `--apply` `pwsh scripts/backup-postgres.ps1` . Oprava mění vstup brány šablon, brzd a kalibrace confidence (čtou `outcome_r` z DB), proto po zápisu zkontrolovat verdikt brány dotčených buněk šablona × symbol. **Vyřazení ze statistik (#1346, `--exclude` , rozhodnutí uživatele 6. 10. 2026):** verdikt „vstup mimo bary" se kontroluje i na kvartální expiraci (vstup na settle nezávisí) a `--exclude` (vzájemně výlučné s `--apply`) mu zapíše trvalou značku `context.excluded` = `{reason: "vstup_mimo_bary", detail: <důvod z reportu>, ts}` přes `SetupsRepository.exclude` . Výsledek, časy ani zbytek kontextu se nemění, nic se nemaže; aktivní řádek se neoznačí, řádek se značkou se nemění (idempotentní, drží první čas). Potvrzení, `--yes` jen s `--approved` a shoda řádků „vstup mimo bary" se schváleným CSV platí jako u `--apply` . Proč značka, ne výpočet při čtení: kritérium stojí na barech v partici, které API ani engine při čtení historie nečtou (bind mount ~20 ms/soubor); predikát nad řádkem (`counts_in_stats` = ne `born_after_settle` a bez značky) pak čtou všichni čtenáři — souhrn (`excluded_count`), Knihovna, brzdy, brána, čerpání zkoušky, kalibrace confidence, sebekontrola a kouč; tabulka řádek ztlumí se štítkem. Gamma útes (`next_setups`) a drift news-engine je nevyřazují stejně jako vznik po settle (#1331). Postup: dry-run → schválení CSV → `pwsh scripts/backup-postgres.ps1` → `--exclude --yes --approved <CSV>` . Zápis 6. 10. 2026: ES 1004, 1049, NQ 1048, 1184, 1203.
Oprava barů dne z IBKR historical (#1320)	Když přepočet hlásí setup jako neověřitelný kvůli díře nebo `tasty_candle` v cestě: z hostitele při běžící IB Gateway / TWS `uv run python scripts/backfill_bars.py --days 2026-09-10,2026-09-14 [--symbols ES,NQ] [--replace-tasty] [--replace-wrong-contract] --dry-run` , výpis zkontrolovat, pak týž příkaz bez `--dry-run` . Bez `--replace-tasty` doplní jen chybějící minuty, s ním nahradí i minuty `tasty_candle` . S `--replace-wrong-contract` přepíše i dřívější doplnění `ibkr_hist` , jehož close se od ověřeného staženého baru liší o víc než `BACKFILL_CONTRACT_TOLERANCE` (bar jiného kontraktu zapsaný před stráží #1232, např. 9. 9. 2026 Z6 mezi měřenými U6; výpis ukáže bloky a odchylku, `ibkr_hist` v toleranci zůstává). Živé minuty (`ibkr`) nepřepíše nikdy, dřívější doplnění (`ibkr_hist`) bez této volby také ne; zápis jde upsertem `SnapshotWriter.write_bars` (přednost původu `bar_source_rank`), nic nemaže. Doplněné bary nesou `source = ibkr_hist` , stejně jako backfill enginu; hluboký backfill bez `--days` je do #1320 zapisoval jako `ibkr` . Kontrakt dne vybírají měřené bary partice, ne kalendář: nejdřív front podle expirace, pak další kvartál. Bary musí sedět na měřené minuty v toleranci `BACKFILL_CONTRACT_TOLERANCE` (stráž #1232), a to i po blocích: každý zapisovaný souvislý blok minut se porovná s nejbližší měřenou minutou téže seance po každé straně. Seance začíná 17:00 CT předchozího dne (22:00 UTC v letním čase, 23:00 v zimním), takže stranu před blokem od půlnoci UTC ověří jen měřená minuta ve večeru předchozí partice. Má-li předchozí partice měřené minuty seance dne (běžně večer), stáhne skript z ověřeného kontraktu i předchozí den: jeden request a 11 s navíc na den a symbol; do předchozí partice nic nezapíše (#1390). Engine mohl během dne přepnout kontrakt (roll se projeví až novým discovery — restart, nová seance) a medián dne by takovou partici pustil; měřená minuta za denní pauzou je jiná seance a kontrakt bloku neověří. Doplnění díry nebo `tasty_candle` stačí jedna strana, přepis `ibkr_hist` potřebuje obě (engine mohl za jedinou ověřenou stranou přepnout kontrakt a bary v partici by byly správné). Den bez měřených minut, den, kde nesedí žádný kandidát, blok bez měřené minuty téže seance, blok, vedle kterého měřil engine jiný kontrakt, ani přepis s ověřenou jen jednou stranou se nezapíše

Situace	Postup
	(vypíše se s důvodem, návratový kód 1). Dnešek a včerejšek skript odmítne: jejich partice drží engine v paměti (`BUFFER_KEEP_DAYS`) a pozdním zápisem by opravu přepsal. Port je z `GEXLENS_IBKR_PORT` (IB Gateway 4001, přebije `--port`), clientId 997, mezi requesty 11 s. Před zápisem zkopírovat dotčené partice `data/derived/{sym}/bars/{den}.parquet` stranou a zkontrolovat počet kopií. Pak přepočet setupů v dry-runu.
Sken skoků v archivu barů (#1390)	Z hostitele: `uv run python scripts/scan_bar_jumps.py --data data [--symbols ES,NQ] --db "postgresql+psycopg://gexlens:<heslo>@127.0.0.1:55432/gexlens" > report.md` (heslo = `GEXLENS_PG_PASSWORD` , URL z `.env` míří do compose sítě; na Windows s `PYTHONUTF8=1` ; zprávy se čtou v transakci read-only, partice jen čtení). Hlásí každý skok ≥ `JUMP_BP` (60 bp) mezi po sobě jdoucími minutovými bary uvnitř seance i přes její hranici: obchodní den (`trading_session_date` , den bez US seance s poznámkou), čas, bp, mezera v minutách (`zavřeno` = jen pauza, víkend nebo svátek podle `is_market_closed` ; `díra N min` = chybí N minut otevřeného trhu), zdroj baru před a po a pravděpodobnou příčinu. Pořadí příčin: **špatný kontrakt** (skok vrácený opačným skokem v téže seanci, zbytek ≤ 10 % většího skoku a medián
Ukázka výstupu MCP nad REST (E-3.1, #1432)	Z hostitele: `uv run python scripts/mcp_sample_1432.py --api http://127.0.0.1:8010 [--symbols ES,NQ] > ukazka.json` (na Windows s `PYTHONUTF8=1`). Jen `GET` na API, nic nezapisuje, žádný MCP server. Skládá prototyp nástrojů `levels_now` , `session_context` , `vol_context` a `setup_library` (#1407): každé číslo nese `value` , `unit` , `as_of` , `age_s` , `market_open` , `source` , `scope` a vedlejší čísla jednotku v `units` ; chybějící hodnota je `null` s `reason` , pole bez REST jsou v `omitted` s ID úkolu, tendence `calibrated: false` . Minutová hodnota starší než 180 s při otevřeném trhu = `error: stale` ; při zavřeném trhu čte poslední seanci s bary bez chyby. Vzdálenosti `distance_pt` a `distance_em` počítá čistá funkce `level_distance` (převezme ji MCP server E-3.7); `distance_em` je zatím `null` , protože EM dnešní seance REST nevrací. Rozbor zkreslení nad ukázkou je v #1246.
Backfill zpráv	`python scripts/alpaca_news_backfill.py --dry-run` (změří objem), pak `--from 2024-07-28` — historie headlines z Alpaca s filtrem relevance (indexové ETF + mega caps; bez něj je ~85 % zpráv small caps bez vazby na index).
Oprava HTML entit v titulcích zpráv (#1500)	Alpaca do 10. 10. 2026 ukládala titulek a perex HTML-kódované (`Fed's` , `S&P`), Bluesky boti píšou entity doslova. Nové řádky dekóduje collector a klíč dedupu (`normalize_title`) entity dekóduje sám, obojí přes `compute/newstext.unescape_entities` : pojmenovaná entita jen se středníkem, číselná i bez něj. Holý `&` zůstane, takže `S&P` i odkaz s `&region=US` se nezmění. Bluesky tím propustí i posty s `S&P 500` , které dřív filtr klíčových slov minul. Historii srovná jednorázově `uv run python scripts/fix_news_html_entities.py` (dry-run jen čtením, URL `--db` , jinak `GEXLENS_NEWS_DATABASE_URL` / `GEXLENS_HOST_DATABASE_URL` / `GEXLENS_PG_PASSWORD`). Pak **záloha PG** a `--apply` : dekóduje `title` a `summary` u `alpaca` a `bluesky` , u Bluesky i `body` , a přepočte `dedup_hash` . Kde přepočtený hash už má jiný řádek (kopie, kterou dedup minul), zůstane starý hash a řádek se nemaže; `raw` se nemění. Je idempotentní. Spustit **hned po nasazení news-enginu s #1500, v téže pauze**. Nová verze počítá v okně dedupu hash už z dekódovaného titulku, takže restart není potřeba. Do doběhnutí skriptu se ale kopie k řádkům s entitou nezapíšou (WARNING `Kopie zprávy bez prvního doručení`). Se starší verzí news-enginu by se po skriptu restartovat musel.

13. Zprovoznění od nuly — IBKR účet, TWS/Gateway

Jednorázový onboarding pro nové prostředí (převzato z issue #1, kde vznikl a byl odškrtán při prvním zprovoznění 16. 7. 2026). Bez těchto kroků se engine nepřipojí, nebo dostane jen delayed data, která záměrně odmítá (SPEC 3.1 — Greeks z

delayed dat nejsou spolehlivé).

13.1 Market data subskripce (Client Portal)

1. https://www.interactivebrokers.com → **Log In** → **Portal** (IBKR login + IB Key).
2. **Settings** → **User Settings** → **Market Data Subscriptions** → Configure (ozubené kolo).
3. **North America** → **Futures** → **CME Real-Time (NP,L2)** — pokrývá ES/NQ futures i futures opce (FOP). Levná subskripce (~1,55 USD/měs.) prokazatelně stačí (ověřeno živě, ADR-0001).
4. Zkontroluj status **Non-Professional** (jinak výrazně vyšší poplatky).
5. *(Až pro SPY/SPX — sekundární scope)*: **OPRA (US Options Exchanges)**, pro SPX index navíc **Cboe Streaming Market Indexes**.
6. Vývoj proti **paper účtu**: Settings → Account Settings → Paper Trading Account → *Share real-time market data with paper account* — jinak paper účet subskripce nevidí.

13.2 TWS nebo IB Gateway (musí běžet lokálně)

Engine se připojuje socketem na lokální TWS/Gateway (z kontejneru přes `host.docker.internal`), ne přímo na servery IBKR.

Varianta A — stávající TWS (nejrychlejší): Edit → Global Configuration → API → Settings → ✅ *Enable ActiveX and Socket Clients*, port **7496** live / **7497** paper, Trusted IPs `127.0.0.1` (vypne potvrzovací dialog), *Read-Only API* nechat **zapnuté** — GEXLens jen čte, nic neobchoduje.

Varianta B — dedikovaný IB Gateway (doporučeno pro trvalý provoz, #461):

1. Stable Windows 64-bit: https://www.interactivebrokers.com/en/trading/ibgateway-stable.php
2. Login obrazovka: režim **IB API** (ne IB Trader Workstation), Live/Paper, přihlášení s IB Key. Pozor: přihlášení se stejným username **odhlásí běžící TWS** (a naopak, kap. 13.3) — přepojovat mimo seanci.
3. Configure → Settings → API → Settings:
   - port nechat **4001** live / **4002** paper (nepřepisovat na 7496 — po #737 poběží TWS vedle Gateway a port by kolidoval);
   - **Read-Only API** zapnuté;
   - **Allow connections from localhost only vypnout** — engine se připojuje z kontejneru přes `host.docker.internal`, ne z `127.0.0.1`; s výchozím nastavením se nepřipojí;
   - **Trusted IPs** zkopírovat z dnešní TWS (Edit → Global Configuration → API → Settings) — samotná `127.0.0.1` nestačí, spojení z Dockeru přichází z jiné adresy a bez trusted IP Gateway vyhodí potvrzovací dialog při každém reconnectu. U Docker Desktop (WSL2) je to `192.168.65.254` — přesnou adresu ukáže log enginu (`Connect call failed ('192.168.65.254', 4001)`) nebo potvrzovací dialog Gateway;
   - Master API client ID nechat prázdné.
4. Configure → Settings → Lock and Exit → **Auto restart**, čas mimo seanci (např. 23:00).
5. Přepojit engine — obojí, jinak se po příštím vytvoření kontejneru vrátí starý port:
   - v aplikaci Settings → IBKR → Port `4001`, uložit (engine se přepojí bez restartu a pipeline založí v nejbližším cyklu, #455);
   - v `.env` `GEXLENS_IBKR_PORT=4001`, pak `docker compose up -d engine` (předtím uložit log kontejneru, kap. 12 / 13.5 — #1056) (restart kontejneru nestačí, env se čte při jeho vytvoření). Host v `.env` neměnit — compose ho přepisuje na `host.docker.internal`. News-engine vlastní socket k TWS nemá, nic dalšího se nepřepojuje. Pořadí je jedno, ale **obojí je nutné**: hodnota ze Settings (DB) `.env` přebíjí — kdo změní jen `.env`, uvidí connect na 4001 a o sekundu později skok zpět na 7496 (#992; engine to od té doby hlásí `WARNING` em při startu).
6. Ověřit: `Test-NetConnection 127.0.0.1 -Port 4001`, stavová lišta `connected :4001` a `● Live`; v logu enginu zkontrolovat, že chodí broker news pásky (`reqMktData` broad tape).

Naměřená zátěž (2. 9. 2026, 55 min po startu, Globex před US RTH, ES+NQ, 3 × 60 s přes `Get-Process ibgateway`):

	TWS (4. 8.)	IB Gateway	rozdíl
CPU klid (1 jádro = 100 %)	16 %	19 % medián, špičky ~38 %	srovnatelné
RAM	1 003–1 087 MB	651 MB	−40 %
CPU při konfliktu session (error 10197)	244 %	—	Gateway neřeší, jen druhý username (#737)

Gateway tedy šetří paměť, ne procesor; hlavní zdroj zahřívání PC (souběh loginů na jednom username) odstraní až druhý username. Provozní důvody (bez GUI, auto-restart, žádné okno na ploše) platí dál.

Denní a týdenní reautentizace (TWS i Gateway stejně): *Auto restart* restartuje aplikaci denně bez zadávání hesla. Týdenní cyklus IBKR začíná každé pondělí — po víkendovém resetu serverů se auto restart sám nepřihlásí a aplikace čeká na ruční login (potvrzení IB Key push); v logu `Login failed = Soft token=0 received instead of expected permanent`. Není to vlastnost Gateway, ale session na straně IBKR (dokumentace IBKR). U TWS to není vidět, protože se do ní během týdne přihlašuje kvůli obchodování tak jako tak. **V neděli po auto-restartu (nebo v pondělí ráno před seancí) potvrdit IB Key push**, jinak engine v pondělí nasbírá díru až do přihlášení.

Ověřeno v provozu (7. 9. 2026, #1016): týdenní cyklus platí přesně takhle — pondělní ruční přihlášení s IB Key bylo potřeba, zatímco denní auto restart ve 23:00 (4. → 5. 9.) i nedělní start seance (6. 9. 22:00) proběhly bez zásahu (bary ES 3.–7. 9. kompletně z IBKR, žádný `tasty_candle` fallback). Výpadek z noci 3. → 4. 9. (Gateway skončila ve stavu `PRELOGON`, engine 9 h na tastytrade) se neopakoval — byl jednorázový, ne vlastnost auto restartu. Provozní rutina je tedy **jednou týdně, v pondělí ráno před seancí**: otevřít okno Gateway, potvrdit IB Key push, zkontrolovat `connected :4001` (kap. 13.5).

13.3 Konflikt jednoho přihlášení ⚠

IBKR povoluje jedno přihlášení na username: Gateway + TWS (či mobil s trading permission, druhé PC) se stejným loginem současně = Gateway spadne. Řešení: druhý user v Client Portal (Settings → Account Settings → Users & Access Rights) — pozor, market data subskripce se platí per user. Pro start stačí varianta A.

13.4 Software na PC

- **Docker Desktop** s WSL2 backendem — celý stack (PostgreSQL, api, engine, frontend) běží přes `docker compose up -d` (kap. 3). Bez Dockeru: Python 3.12, Node.js ≥20, PostgreSQL 16 + `make run` (kap. 9).
- Volné místo: ~1 GB pro 14denní datové okno; WSL2 limit paměti viz `C:\Users\<user>\.wslconfig` (`[wsl2] memory=6GB` — pojistka proti nafouknutí vmmem). Od 9. 9. 2026 (#1105) navíc `autoMemoryReclaim=gradual` (VM vrací nevyužitou cache Windows, jinak vmmem sedí trvale na stropu) a `sparseVhd=true` (virtuální disk uvolňuje smazaná data); platí po `wsl --shutdown` nebo restartu PC.
- **Paměť procesů (#1105):** engine i news-engine logují RSS každých 10 min (`engine: RSS 1234 MB`) a engine ho hlásí do `/status.memory_rss_mb` (Settings → „Paměť enginu"). Při hledání viníka nastav `GEXLENS_MEMORY_TRACE=1` (tracemalloc, top-10 řádků kódu podle přírůstku; dražší běh, jen dočasně). Hloubka zásobníku `GEXLENS_MEMORY_TRACE_FRAMES` má default **1** (jen řádek alokace) — s 25 rámci engine 16./17. 9. 2026 neběžel (RSS 1,9 → 3,8 GB za 8 min, cykly > 240 s) a news-engine držel celé jádro; hlubší stack jen na devu. Trace platí pro engine i news-engine, po měření flag z `.env` odstranit a **oba** kontejnery recreatnout. Noc 10./11. 9. 2026 ukázala, že RSS roste mimo Python heap (glibc drží uvolněné bloky v arénách) — hlídka proto po každém vzorku volá `malloc_trim(0)` a loguje, kolik MB vrátila (`malloc_trim vrátil N MB` , vypnutí `GEXLENS_MALLOC_TRIM=0`), a compose nastavuje `MALLOC_ARENA_MAX=2` pro engine i news-engine. Vyhodnocení 15. 9.: u news-engine to stačilo (trim vrací ~~280 MB na vzorek, RSS z 3,1 GB na 0,4–0,7 GB), engine drží ~3 GB nezávisle na trimu~~ (45 MB na vzorek) — proto hlídka od 15. 9. po každém vzorku loguje i **Arrow pool** pyarrow (`Arrow pool (mimalloc) alokováno N MB, max M MB; release_unused vrátil K MB`), volá `release_unused()` před glibc trimem a podíl hlásí do `/status.memory_arrow_mb` ; vypnutí `GEXLENS_ARROW_RELEASE=0` . Čtení: velký `release_unused` = cache Arrow (partice), malý = paměť drží Python objekty a

numpy pole. **CPU a teplota** (11. 9. 2026, notebook i5-10300H přes 90 °C při špičkách): `.wslconfig` `processors=4` v sekci `[wsl2]` (VM dostane polovinu vláken, Docker buildy a testy nesaturují celý procesor), compose `cpus: 2.0` engine / `1.5` news-engine (tlumí jen nárazy, v klidu služby berou jednotky %), vitest `maxWorkers: 2`; testovací sady a buildy pouštět po jednom, ne paralelně ve více worktree. **Pozor na sekce `.wslconfig`:** `autoMemoryReclaim=gradual` a `sparseVhd=true` patří do `[experimental]` — v `[wsl2]` je WSL 2.7 hlásí „Neznámá klávesa" a ignoruje (12. 9. 2026 se tak zjistilo, že bod B z #1105 doteď neplatil; ověření: `wsl -d docker-desktop -e true` nesmí vypsat varování). Změny platí po `wsl --shutdown` (= restart stacku, jen v okně). Hygiena: dev stack (`compose.dev.yml`) nikdy nenechávat běžet vedle produkce, prohlížeč s heatmapou na jednom tabu — PC s 16 GB má po 6 GB pro Docker a IB Gateway málo rezervy.

Disk Dockeru (VHDX) a úklid (#1127). Datový disk Docker Desktopu (`D:\Programy\Docker\DockerDesktopWSL\disk\docker_data.vhdx`, cesta v Settings → Resources) **jen roste**: každý build přidá vrstvy do build cache a každý deploy rollback tag `gex-*:pre-<issue>`. 12. 9. 2026 měl 71 GB (43 GB build cache, 36 rollback tagů), build zaplnil disk D: na 0 B, containerd ve VM spadl a Docker Desktop se nerozjel („Reset to factory defaults" NIKDY — pgdata je uvnitř VHDX). Proto:

- `scripts/docker-cleanup.ps1` maže jen rollback tagy (nechá 3 na službu), build cache starší 7 dnů a osiřelé image; volumes se nedotýká. Běží po každém úspěšném deployi (`deploy-engine-offhours.ps1` krok 5c) a týdně v sobotu 08:00 (Task Scheduler „GEXLens docker úklid", `scripts/register-docker-cleanup-task.ps1`, log `data/logs/docker-cleanup.log`). Při < 15 GB volných na disku s VHDX nebo VHDX > 40 GB pošle alert `disk_low` do zvonku a skončí s exit 2.
- Smazaná data VHDX **sám nevrátí** — místo uvolní až kompakce: `pwsh -File scripts/compact-docker-vhdx.ps1` **jako správce** (diskpart), Docker při ní 2–5 min stojí → jen při zavřeném trhu; stack naběhne sám. Automaticky ji spouští elevovaná úloha „GEXLens compact-vhdx" po/st/pá 23:05 a v sobotu 10:00 (#1277, místní čas) a `docker-cleanup.ps1` po deployi; běží jen nad prahem (`-IfNeeded`), při zavřeném trhu s ≥ 20 min do otevření a počká, až doběhne deploy / walk-forward / záloha PG a aktualizace v pauze (zámek údržby `%LOCALAPPDATA%\GEXLens\docker-maintenance.lock`, #1552 — drží ho i kompaktace, takže ji pozná hlídač Dockeru). Nanečisto: `-IfNeeded -DryRun`. Před zastavením si zapamatuje běžící služby projektu `gex`; když po kompaktaci Docker nenaběhne (volání `docker` mají časový limit) nebo některá z nich neběží, zkusí jeden restart Docker Desktopu a `docker start` (jen spustí existující kontejnery, nic nerecreatuje; záměrně zastavenou službu nechá být) a pak upozorní **mimo Docker** (#1279, `scripts/lib/OpsAlert.ps1`) na Telegram, jsou-li v `.env` `GEXLENS_PUSH_TELEGRAM_TOKEN` a `_CHAT_ID` (token se nevypisuje) a povolí-li to Nastavení → Notifikace: hlavní vypínač a přepínač **Noční údržba selhala** (`maintenance`, #1284). Nastavení si skript přečte z `GET /push/status` ještě **před** zastavením Dockeru (`Get-OpsAlertPolicy`, výsledek je v logu a v `-DryRun`); když API neodpoví, Telegram se pošle (fail-open). Upozornění je vždy v logu úlohy (`data/logs/compact-vhdx.log`); okno na ploše (`msg.exe`) #1284 zrušil, takže vypnutý nebo nenastavený Telegram = jen log. Totéž upozornění pošle `deploy-engine-offhours.ps1` po rollbacku (API běží, nastavení čte až v tu chvíli). Chybu diskpartu pozná podle návratového kódu a VHDX odpojí. Limit úlohy je 60 min.
- Lokální build image jen nouzově a **po jedné službě** (`docker compose build engine`, pak `frontend`), nikdy všechny naráz, a před buildem zkontrolovat volné místo; standardně image dodává CI (#1139, kap. 3).

Aktualizace WSL a Docker Desktopu, hlídač Dockeru (#1552). Balík WSL (MSI) instaluje automatický sken Windows Update (klient `MoUpdateOrchestrator`, služba DCat Flighting Prod — stejnou cestou jako ovladače), ne Microsoft Store; zásada Store „Turn off Automatic Download and Install of updates" ho proto nezastaví. Instalace restartuje VM WSL a Docker Desktop se sám nerozjede (13. 10. 2026: výpadek 03:11–08:00 UTC). Vlastník rozhodl (13. 10., varianta V3) instalaci mimo pauzu neblokovat; místo toho:

- **„GEXLens update-offhours"** — elevovaná úloha denně 23:10 místního času (registrace jako správce `pwsh scripts/register-update-offhours-task.ps1`), skript `scripts/update-wsl-docker-offhours.ps1`. Běží jen při zavřeném trhu s ≥ 30 min do otevření (`Get-GlobexClock`) a nejvýš 20 min počká na deploy, walk-forward, zálohu PG a kompaktaci; pak drží zámek údržby (kompaktace a hlídač na něj čekají). `wsl --update`; když se verze WSL změní, `docker desktop restart` a kontrola stacku (daemon, `docker start` chybějících služeb, engine online a connected). Docker Desktop: `docker desktop update --check-only`; je-li novější verze, nejdřív záloha PG (`backup-postgres.ps1`) a `pg_restore --list` nad dumpem, přesné počty řádků všech tabulek (read-only), `docker desktop update --quiet`, čekání na novou verzi, kontrola stacku a znovu počty řádků — žádná tabulka nesmí zmizet ani ztratit polovinu řádků (menší pokles přepočítávaných tabulek jen do logu). Bez úspěšné zálohy se Docker Desktop neaktualizuje. Problém = upozornění **Noční údržba selhala** (`maintenance`,

stejně jako kompaktace) a exit 1. Jen do logu: Docker neodpovídá už před aktualizací (výpadek hlásí hlídač) a engine bez spojení s IBKR víc než hodinu před otevřením (o víkendu je IB Gateway běžně odhlášená). Limit úlohy 120 min. Log `%LOCALAPPDATA%\GEXLens\logs\update-offhours.log` (mimo repo kvůli Řízenému přístupu ke složkám). Nanečisto `-DryRun`; přepínače `-SkipWsl` a `-SkipDockerDesktop`. Ručně bez UAC: `Start-ScheduledTask -TaskName 'GEXLens update-offhours'`.

- **„GEXLens docker-watchdog“** — úloha každých 5 min bez elevace a bez okna (registrace `pwsh -File scripts/register-docker-watchdog-task.ps1`), skript `scripts/docker-watchdog.ps1`. Když `docker info` do 20 s neodpoví a neběží deploy (příkazový řádek) ani kompaktace či aktualizace (zámek údržby — příkazový řádek elevovaného procesu neelevovaný hlídač nepřečte): Docker Desktop `stopped` nebo CLI bez odpovědi → `docker desktop restart`, čekání na daemon (≤ 5 min), výpis kontejnerů gex-*, které neběží (`unless-stopped` spustí Docker sám, ručně zastavené hlídač nespouští), a spojení enginu (≤ 5 min), pak upozornění s výsledkem; přechodový stav (`starting` …) čeká do 10 min, pak restart; `running` s nereagujícím daemonem jen upozorní (restart jen při stavu ≠ running). Upozornění jde jen při otevřeném trhu podle `compute/marketclock.is_market_closed` (Python z `.venv` repa, zná svátky CME; bez něj náhradou `Get-GlobexClock` s varováním v logu), při zavřeném se jen loguje (AGENTS.md „Zavřený trh = žádná upozornění“), jedno na výpadek. Nastavení notifikací čte z API; když API s Dockerem neodpovídá, Telegram se pošle (fail-open). Limit úlohy 30 min, incident se ukládá před restartem. Stav výpadku `%LOCALAPPDATA%\GEXLens\docker-watchdog.json`, log `%LOCALAPPDATA%\GEXLens\logs\docker-watchdog.log` (zapisuje jen při výpadku). Nanečisto `-DryRun`. Před ručním zastavením Dockeru: `Disable-ScheduledTask -TaskName 'GEXLens docker-watchdog'`.
- Zbývající riziko V3: vydání WSL, které Windows Update nainstaluje dřív než úloha v pauze, shodí Docker kdykoli; hlídač výpadek zkrátí na ~5–10 min, mezeře v datech nezabrání.

13.5 Ověření a denní provoz

```
Test-NetConnection 127.0.0.1 -Port 7496   # TcpTestSucceeded: True = API poslouchá (Gateway: 4001)
```

Každé pondělí ráno: potvrdit IB Key push v Gateway (týdenní reautentizace, kap. 13.2). Každý obchodní den: TWS/Gateway běží a je přihlášený **před startem enginu**; stavová lišta aplikace ukazuje `connected :7496` (Gateway `:4001`) a `● Live` (ne Offline). Diagnostika problémů: kap. 12.

Restart nebo nasazení enginu: vždy `pwsh scripts/deploy-engine-offhours.ps1` (pauza Globexu, rollback, **uložení logu před recreate** — #1056). Ruční `docker compose up -d engine` jen s předchozím `docker logs gex-engine-1 > data/logs/engine-<YYYYMMDD-HHMM>.log 2>&1`; jinak důkazy o předchozím běhu zmizí.

13.6 Úlohy hostitele — při přenosu na jiný PC nebo server

Úlohy Task Scheduleru nejsou v Dockeru ani v gitu. Jsou zaregistrované ve Windows tohoto PC a jejich akce mají absolutní cesty ke složce repa, k `pwsh.exe` a k logům. Skripty navíc obsahují napevno některé cesty tohoto PC (krok 2 postupu). Proto se úlohy musí **znovu zaregistrovat**, když se aplikace přenese na jiný PC, když se přesune složka repa nebo po reinstalaci Windows. Na Linux serveru (#1094) se nahrazují jinak (poslední sloupec tabulky). Stav k 15. 10. 2026 (`Get-ScheduledTask -TaskName 'GEXLens*'`), backup-postgres od #1566 (23. 10.):

Úloha	K čemu	Rozvrh (místní čas)	Registrace	Log	Linux server (#1094)
GEXLens docker-watchdog	restart zastaveného Docker Desktopu, upozornění na výpadek (#1552, kap. 13.4)	každých 5 min	`pwsh -File scripts/register-docker-watchdog-task.ps1`	`%LOCALAPPDATA%\GEXLens\logs\docker-watchdog.log` (jen při výpadku)	jinak: Docker Engine je služba systemd a po pádu ho restartuje systemd; upozornění na výpadek Dockeru zvenku zatím chybí → #1094

Úloha	K čemu	Rozvrh (místní čas)	Registrace	Log	Linux server (#1094)
GEXLens update-offhours	WSL a Docker Desktop aktualizovat jen v pauze Globexu, před aktualizací Docker Desktopu záloha PG (#1552, kap. 13.4)	denně 23:10	**jako správce** `pwsh -File scripts/register-update-offhours-task.ps1`	`%LOCALAPPDATA%\GEXLens\logs\update-offhours.log`	ne (WSL ani Docker Desktop tam nejsou); automatické aktualizace systému nastavit tak, aby Docker nerestartovaly mimo pauzu
GEXLens compact-vhdx	kompaktace VHDX Docker Desktopu nad prahem (VHDX ≥ 40 GB a ≥ 10 GB k vrácení; #1127, #1277, kap. 13.4)	po, st, pá 23:05 a so 10:00	**jako správce** `pwsh -File scripts/register-compact-vhdx-task.ps1`	`data\logs\compact-vhdx.log`	ne (VHDX má jen Docker Desktop)
GEXLens docker úklid	rollback tagy (3 na službu), build cache starší 7 dnů, osiřelé image; alert `disk_low` do zvonku (#1127, kap. 13.4)	so 08:00	`pwsh -File scripts/register-docker-cleanup-task.ps1`	`data\logs\docker-cleanup.log`	potřeba, ale skript je jen pro Windows (APPDATA, VHDX, Task Scheduler) → úklid a hlídka místa na serveru #1567
GEXLens walk-forward	noční walk-forward parametrů setupů, jen report (ADR-0034, kap. 12)	po–pá 23:30	`pwsh -File scripts/register-walkforward-task.ps1` (účet s `docker` a `uv`)	`data\reports\walkforward-nightly.log`	`scripts/systemd/gexlens-walkforward.{service,timer}`; předpoklady: pásmo Europe/Prague (timer je v místním čase serveru), `uv` v PATH služby, `pwsh` v `/usr/bin`, uživatel `gex` ve skupině `docker` (#1567)
GEXLens backup-postgres	noční záloha PostgreSQL: `backup-postgres.ps1` (dump, hlavička PGDMP, celé čtení `pg_restore`, rotace 14 dumpů v `D:\Programy\GEX\zalohy-pg`) s limitem 20 min; čeká na zámek údržby (kompaktace, aktualizace), deploy a ruční zálohu; selhání dumpu = upozornění Noční údržba, přeskočení (aplikace	denně 23:50	`pwsh -File scripts/register-backup-postgres-task.ps1`	`%LOCALAPPDATA%\GEXLens\logs\backup-postgres.log`	potřeba: timer systemd s `pg_dump` a kopie mimo server (#1094)

Úloha	K čemu	Rozvrh (místní čas)	Registrace	Log	Linux server (#1094)
	vypnutá, Docker neodpovídá) jen při dumpu starším 36 h (#1566)				

Jednorázové úlohy `GEXLens-deploy-engine` a `GEXLens-deploy-engine-600` ze srpna 2026 (odložený deploy přes `scripts/deploy-engine-offhours.cmd`) už proběhly a vlastník je 15. 10. 2026 smazal; nepřenášejí se. Pravidelnou zálohu PG dělá od #1566 úloha GEXLens backup-postgres; před přenosem přesto spusť `pwsh scripts/backup-postgres.ps1`, ať je dump čerstvý. Dumpy leží na D:, stejně jako VHDX — selhání disku D: přežije jen kopie jinam (varianta B v #1566, s přenosem #1094).

Postup na novém Windows PC (Docker a stack podle kap. 13.4, ověření podle kap. 13.5):

1. Repo dej na konečné místo. Registrační skripty berou cesty ze svého umístění (`$PSScriptRoot`) a úloha si je uloží natvrdo. Po každém přesunu složky registraci zopakuj (`-Force` úlohu přepíše).
2. Cesty tohoto PC ve skriptech (registrací se zatím nastavit nedají, #1567):
   - VHDX Docker Desktopu `D:\Programy\Docker\DockerDesktopWSL\disk\docker_data.vhdx` (výchozí `-Vhdx` v `compact-docker-vhdx.ps1`). V Docker Desktopu nastav Settings → Resources → Advanced → Disk image location na `D:\Programy\Docker\DockerDesktopWSL`. Jinak kompaktace skončí `VHDX nenalezen` bez upozornění a VHDX poroste (12. 9. 2026 plný disk shodil Docker).
   - Cíl záloh PG `D:\Programy\GEX\zalohy-pg` (výchozí `-Target` v `backup-postgres.ps1` a `backup-postgres-nightly.ps1`, `-BackupTarget` v `update-wsl-docker-offhours.ps1`). Bez zapisovatelné složky záloha selže, noční úloha upozorní a update-offhours Docker Desktop neaktualizuje.
   - Docker Desktop v `C:\Program Files\Docker\Docker` (kompaktace ho po sobě spouští touto cestou).
3. PowerShell 7: úloha volá `%LOCALAPPDATA%\Microsoft\WindowsApps\pwsh.exe` (alias instalace z Microsoft Store, přežije aktualizaci verze); když chybí, použije `pwsh` z PATH v okamžiku registrace. Spustitelný soubor akce Task Scheduler v PATH uživatele nehledá, proto plná cesta; skripty pak PATH uživatele vidí (`uv` v `~\.local\bin` stačí). Hlídač, update-offhours a backup-postgres volají pwsh přes `conhost.exe --headless` (bez okna).
4. `uv sync --all-packages`: hlídač Dockeru bere rozvrh trhu se svátky z Pythonu v `.venv` repa (bez něj náhradou `Get-GlobexClock` bez svátků, s varováním v logu). Walk-forward potřebuje `uv`.
5. Proměnné v `.env`:
   - hlídač, update-offhours, compact-vhdx a backup-postgres posílají upozornění na Telegram mimo Docker (`scripts/lib/OpsAlert.ps1`), když jsou `GEXLENS_PUSH_TELEGRAM_TOKEN` a `GEXLENS_PUSH_TELEGRAM_CHAT_ID` a v Nastavení → Notifikace je zapnutý přepínač **Noční údržba selhala**; když API neodpoví, pošle se i tak. Bez proměnných se jen zapíše do logu;
   - docker úklid posílá `disk_low` do zvonku přes API (`GEXLENS_API_TOKEN`);
   - walk-forward čte `GEXLENS_PG_PASSWORD`.
6. Defender → **Řízený přístup ke složkám**: když je zapnutý a repo leží v chráněné složce (Dokumenty, Plocha), pwsh do `data\` nezapíše (`Could not find file`, událost 1123) a compact-vhdx, docker úklid a walk-forward neuloží log ani report. Repo dej mimo chráněné složky, nebo mezi povolené aplikace přidej `pwsh.exe` (alias i skutečnou cestu) a `python.exe` z `.venv`. Hlídač, update-offhours a backup-postgres proto logují do `%LOCALAPPDATA%`.
7. Bez správce zaregistruj docker-watchdog, docker úklid, walk-forward a backup-postgres. V terminálu **jako správce** (Win+X → Terminál (správce)) update-offhours a compact-vhdx.
8. Úlohy běží pod účtem, který je zaregistroval, a jen když je ten účet přihlášený (`LogonType Interactive`). Po restartu se tedy PC musí přihlásit.
9. Rozvrh je v místním čase (`scripts/lib/LocalTrigger.ps1`, #1278) a počítá s pásmem Praha (pauza Globexu 23:00–24:00). Zavřený trh si ověřují compact-vhdx, update-offhours a walk-forward (#1568): v jiném pásmu nebo v týdnech posunu letního času (pauza 22:00–23:00 SEČ, AGENTS.md) běh přeskočí. Walk-forward v těch týdnech dá report jen v pátek (23:30 SEČ je už víkendová zavírka); docker úklid v sobotu bránu nepotřebuje. Backup-postgres bránu trhu nemá

(`pg_dump` sběr nezastaví), v těch týdnech zálohuje v otevřeném trhu. Čas změníš parametrem `-At` (u compact-vhdx `-At` = sobota a `-WeekdayAt` = po, st, pá; hlídač `-EveryMinutes`).

10. Kontrola hned po registraci: `Start-ScheduledTask -TaskName 'GEXLens docker-watchdog'` (při běžícím Dockeru nic nedělá) a `Start-ScheduledTask -TaskName 'GEXLens walk-forward'` (jen report; za otevřeného trhu jen řádek „walk-forward se přeskakuje" v logu, #1568) a `Start-ScheduledTask -TaskName 'GEXLens backup-postgres'` (nový dump v `D:\Programy\GEX\zalohy-pg`, řádek „Noční záloha PG hotová" v `%LOCALAPPDATA%\GEXLens\logs\backup-postgres.log`). Za minutu `Get-ScheduledTask -TaskName 'GEXLens*' | Get-ScheduledTaskInfo`: `LastTaskResult` 0 (267011 = ještě neběžela) a vyplněný `NextRunTime`; u walk-forwardu i nový čas změny logu (hlídač zapisuje log jen při výpadku). Elevované úlohy zkontroluj ráno po první pauze. Konzolový `-DryRun` akci úlohy neověří (cesta pwsh, přesměrování logu, Řízený přístup ke složkám), jen rozhodnutí skriptu: `docker-watchdog.ps1 -DryRun`, `update-wsl-docker-offhours.ps1 -DryRun`, `compact-docker-vhdx.ps1 -IfNeeded -DryRun`, `docker-cleanup.ps1 -WhatIf`.
11. **Starý PC**, jakmile produkce běží jinde:
    1. v terminálu **jako správce** vypni všechny úlohy (`Get-ScheduledTask -TaskName 'GEXLens*' | Disable-ScheduledTask`). Bez elevace zůstanou zapnuté compact-vhdx a update-offhours. Docker Desktop totiž znovu spustí nejen hlídač (do 5 min), ale i kompaktace, která ho po sobě startuje vždy;
    2. `docker compose stop`: kontejnery s `restart: unless-stopped` pak při případném startu Docker Desktopu nenaběhnou. Jinak by se spustil druhý engine na témž IBKR účtu;
    3. ukonči Docker Desktop a zkontroluj, že je vypnuté Settings → General → Start Docker Desktop when you sign in.

---

14. Bezpečnost a nasazení na server

Výchozí stav aplikace počítá s tím, že běží na jednom PC za NATem. Přesun na VPS s veřejnou IP (#539) tenhle předpoklad ruší, proto proběhla prověrka #542. Tahle kapitola je její provozní výstup.

Tajemství

Dvě hodnoty musí být v lokálním `.env` (do repa nepatří, `.gitignore` je drží venku):

Proměnná	K čemu
`GEXLENS_PG_PASSWORD`	Heslo k PostgreSQL. Compose bez něj **nenastartuje** — žádný slabý default už neexistuje.
`GEXLENS_API_TOKEN`	Sdílené tajemství pro `/internal/*` (engine i news-engine → API) a `/backup/postgres`.

Vygeneruje je `pwsh scripts/init-secrets.ps1` — skript je idempotentní, existující hodnoty nepřepisuje a heslo rovnou přepíše i v běžícím PostgreSQL (`ALTER USER`). To je nutné: `POSTGRES_PASSWORD` se uplatní jen při prvním `initdb`, takže na existujícím volume by samotná změna v `.env` znamenala, že se služby k DB nepřipojí.

Token do UI patří jen kvůli stažení zálohy — vkládá se do pole **Settings → Záloha databáze → API token** a zůstává v localStorage prohlížeče. V image frontendu být nesmí, ten si stáhne kdokoli.

Síť

`compose.yml` nepublikuje nic na `0.0.0.0`. Adresu řídí `GEXLENS_BIND_ADDR` (default `127.0.0.1`); na serveru sem patří Tailscale IP. **Nikdy** `0.0.0.0` — Docker zapisuje publikaci do řetězce `DOCKER`, který se na Linuxu vyhodnocuje před UFW, takže port publikovaný bez adresy je veřejný i se „zapnutým firewallem".

Prohlížeč mluví jen s nginx (`:8080`), který proxuje `/api` → `api:8000` včetně WebSocketu. Port API i PostgreSQL zůstávají publikované na loopback jen pro nástroje na hostiteli (zálohy, sondy) — pro provoz je potřeba nemá.

Checklist před nasazením na VPS

1. `pwsh scripts/init-secrets.ps1` na serveru; ověřit, že `.env` má práva 600.
2. `GEXLENS_BIND_ADDR` = Tailscale IP serveru.
3. `GEXLENS_ALLOWED_ORIGINS` = adresa UI, pod kterou se bude otevírat (jinak prohlížeč zablokuje fetch a WS handshake skončí na kontrole Origin).
4. UFW: povolit jen SSH a Tailscale; ověřit `nmap` z venku, že 8080/8010/55432 nejsou vidět (kontrolovat zvenčí, ne `ufw status` — viz past s DOCKER chainem).
5. SSH: klíče, `PasswordAuthentication no`, root login zakázaný.
6. IB Gateway: **VNC nikdy veřejně**, jen přes tunel.
7. Zálohy: `scripts/backup-postgres.ps1` (nebo cron s `pg_dump`) mimo server, šifrovaně — dump obsahuje celý OI archiv a historii setupů.
8. `docker compose up -d --build`, pak ověřit, že engine publikuje (`/status` ukazuje `engine: online`) — chybějící token se pozná tak, že UI zůstane bez živých dat a engine loguje chybu hned při startu.

Linux server: IB Gateway v kontejneru a rozdíly proti PC (#1094)

Postup zprovoznění krok za krokem (netcup RS 1000 G12, Tailscale, přenos dat, přepnutí v pauze Globexu) je v issue #1094. Co k tomu má repo:

Co	Kde	Poznámka
IB Gateway jako služba	`compose.server.yml`	vždy `-f compose.yml -f compose.server.yml`; image `ghcr.io/gnzsnz/ib-gateway:stable` (IBC + Xvfb), **API na `ib-gateway:4003`** (socat relay — 4001 je uvnitř kontejneru jen localhost), nic nepublikuje, VNC vypnuté
Username a heslo	`.env` → `IBKR_DATA_USERID`; `secrets/ibkr_password` (mimo git, chmod 600)	**druhý (datový) username** účtu (#737), nikdy hlavní — souběh s mobilem by shodil Gateway
Reautentizace	`AUTO_RESTART_TIME` 23:05, `TWS_COLD_RESTART` neděle 22:30 (TZ kontejneru Europe/Prague)	pondělní IB Key push zůstává ruční (kap. 13.2); `TWOFA_TIMEOUT_ACTION=restart` login opakuje, dokud push nepřijde
Past host/port v DB	Settings UI → host `ib-gateway`, port `4003`	hodnota z DB přebíjí `.env` i compose (#446/#992); po obnově dumpu z PC tam zůstal domácí `4001`
Název projektu	`compose.yml` → `name: gex`	kontejnery `gex-*` a image `gex-engine` nezávisle na názvu adresáře (`/srv/gexlens`) — deploy skript a rollback tag s nimi počítají
Deploy	`pwsh scripts/deploy-engine-offhours.ps1 -Server`	`-Server` přidá override; zóna Chicago se hledá jako `America/Chicago` i `Central Standard Time`
Walk-forward	`scripts/systemd/gexlens-walkforward.{service,timer}`	náhrada Task Scheduleru; po–pá 23:30, `Persistent=true`
Ostatní úlohy hostitele	kap. 13.6	co z Task Scheduleru PC na serveru odpadá a co chybí (úklid a hlídka místa #1567, upozornění na výpadek Dockeru, záloha PG #1566)
Přístup k UI	`tailscale serve --bg --https=443 localhost:8080`	`GEXLENS_BIND_ADDR` zůstává loopback, `GEXLENS_ALLOWED_ORIGINS=https://<host>.<tailnet>.ts.net`; UFW nepouští nic, SSH přes Tailscale

Co	Kde	Poznámka
Docker × UFW	řetězec `DOCKER-USER` v `/etc/ufw/after.rules` zahazuje vše z veřejného rozhraní do kontejnerů	druhá pojistka vedle loopback bindu; kontrola vždy `nmap` zvenčí
Swap na 8 GB	`zram-tools`, `PERCENT=25`	naměřeno 5,3 GB (kap. 13.2 + `docker stats` 9. 9. 2026); zram kryje špičky RTH, práh pro větší plán je 85 %

Návrat zpět (rollback)

Rotace hesla nesahá na data — mění jen přihlašovací údaj, `pgdata` volume ani parquety se nedotkne. Vrátit ji lze kdykoli:

```
docker exec $(docker compose ps -q postgres) psql -U gexlens -d gexlens `
  -c "ALTER USER gexlens PASSWORD 'gexlens';"
```

Funguje vždy, i když se heslo v `.env` rozejde s databází nebo `.env` ztratíš: `psql` uvnitř kontejneru chodí přes unix socket, který heslo nevyžaduje. Stejným příkazem se dá nastavit i nová hodnota, když je potřeba sladit `.env` s běžícím serverem. Po změně restartovat stack (`docker compose up -d`) — běžící kontejnery drží spojení se starým heslem.

Při revertu změn #542 na tohle nezapomenout. Starší `compose.yml` má `POSTGRES_PASSWORD: gexlens` natvrdo, takže po návratu ke starému kódu se služby k databázi s rotovaným heslem nepřipojí. Pořadí: nejdřív vrátit heslo příkazem výše, pak revertovat kód.

Opakovaná prověrka

`pwsh scripts/security-scan.ps1` spustí gitleaks (celá historie), pip-audit a npm audit; s `-Images` navíc trivy nad postavenými images. První tři běží i v CI jako job `security` na každý PR. Nálezy trivy jsou zpravidla OS balíky base image — řeší je rebuild s čerstvou bází, ne zásah do kódu, proto v CI nejsou.

Interní dokument. Uživatelská příručka: `UZIVATELSKY-MANUAL.md` (dostupná i v aplikaci jako Wiki).

14.9 Přepnutí repozitáře na soukromé — co se rozbije a co udělat

Repo je od září 2026 veřejné a část provozu na tom stojí (13. 9. 2026, #1139). Před přepnutím na *Private* (Settings → General → Danger Zone → Change visibility) projít tento seznam; body 1–3 jsou povinné, jinak přestane fungovat nasazení nebo ochrana `main`.

co	dnes (veřejné)	po přepnutí (soukromé, plán GitHub Free)	co udělat
Image v GHCR (`gex-python`, `gex-frontend`)	veřejné balíčky, zdarma, pull bez přihlášení	balíček navázaný na repo **zůstane veřejný** (viditelnost balíčku je nezávislá) — pokud ho přepneš na private, platí limit **500 MB úložiště + 1 GB přenosu/měs.**, což dva image (~360 MB komprimovaně) s tagy `sha-*` přerostou během týdne	**1.** Buď nechat balíčky veřejné (kód je z image stejně dohledatelný jen jako bytecode; tajemství v nich nejsou, kap. 14.1), nebo při private: přidat do workflow *Images* krok `actions/delete-package-versions` (nechat `latest` + 3 poslední `sha-*`) a na každém stroji, který táhne image, `docker login ghcr.io` (PAT classic s **jen** `read:packages`, uložit do `.env` jako `GHCR_TOKEN`, nikdy do repa; `echo $env:GHCR_TOKEN | docker login ghcr.io -u kEchiCZ --password-stdin`)

co	dnes (veřejné)	po přepnutí (soukromé, plán GitHub Free)	co udělat
GitHub Actions (CI + Images)	minuty zdarma bez limitu	**2 000 min/měs.** zdarma; CI ~5 min na PR + Images ~6 min na merge ⇒ ~10–15 min na PR, tj. cca 130 PR/měs. v limitu	**2.** Sledovat Settings → Billing → Actions; při přiblížení limitu vypnout Images pro PR (dnes běží jen na main — OK) a sloučit CI joby
Ruleset `main` (PR + zelené CI + jen squash, žádný force push)	vymáhá GitHub	**rulesety a branch protection na soukromém repu vyžadují GitHub Pro** — na Free se přestanou vymáhat (zůstanou uložené, ale neaktivní)	**3.** Buď GitHub Pro (~4 $/měs.), nebo se spolehnout na `scripts/merge-when-green.sh` (merguje jen po zeleném CI) a na kázeň: nikdy `git push origin main` přímo. Automatické mazání větví po merge funguje dál
Dependabot, gitleaks, pip-audit, npm audit	běží	běží stejně (Dependabot i CI jsou na Free pro soukromá repa)	nic
Odkazy na soubory/obrázky v issues a wiki	veřejné URL `raw.githubusercontent.com`	fungují jen přihlášeným	nic (wiki v aplikaci má obrázky u sebe v `frontend/public/manual/img`)
`gh` CLI v tomto počítači	token `repo` stačí	totéž; pro balíčky navíc `read:packages` (bod 1)	`gh auth refresh -s read:packages` jen při private balíčcích

Co se **nemění**: tajemství zůstávají výhradně v `.env` (kap. 14.1), image je neobsahují, `docker compose pull` + deploy skript fungují beze změny, pokud zůstanou balíčky veřejné. Pořadí při přepnutí: nejdřív rozhodnout bod 1 (balíčky), pak bod 3 (ochrana `main`), teprve pak přepnout viditelnost repa a ověřit `docker compose pull frontend` na produkci.